



Module Code & Module Title

CS5054NT Advanced Programming and Technologies

Assessment Weightage & Type

50% Group Coursework

Year and Semester

2026 Spring

Group: L2C8

Team Name: ECN

S.N.	Students Name	College ID
1.	Sugam Khatiwada	24046096
2.	Arjun Rai	24045829
3.	Bjay Kumar Khawas	24045847
4.	Nitika Mandal	24046174
5.	Sujal Kumar Majhi	24046103

Submitted To: Mr. Sujan Subedi

Project Title: NirvachanSetu

Project Link: <https://github.com/Sugam-Khatiwada/NirvachanSetu>

Assignment Due Date: May 21, 2026

Assignment Submission Date: May 19, 2026

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

1	Introduction.....	1
1.1	Title.....	2
1.2	Purpose.....	3
1.3	Audience	4
1.4	Aims and Objectives	5
1.4.1	Aim:	5
1.4.2	Objectives:	5
1.5	List of Features	6
1.5.1	Admin Features:.....	6
1.5.2	Voter Features:.....	6
1.5.3	Candidate Features:.....	7
1.5.4	System-Wide Features:	7
2	Database design and development.....	8
2.1	Entity Relationship Diagram (ERD) Design with Business Rules	8
2.1.1	Entity Descriptions and Relationships:.....	10
2.1.2	Normalization Validation up to Third Normal Form (3NF).....	10
2.2	Database Development	17
2.2.1	Table 1: constituencies.....	17
2.2.2	Table 2: Users	18
2.2.3	Table 3: elections	20
2.2.4	Table 4: candidate_applications.....	21
2.2.5	Table 5: candidates	22
2.2.6	Table 6: votes.....	23
2.3	Relationship Diagram of the overall system.....	24
3	Wireframe and Design	25
3.1	Login.....	25

3.1.1	Research Reference.....	25
3.1.2	Wireframe	26
3.1.3	Design Development.....	27
3.2	Registration	28
3.2.1	Research Reference.....	28
3.2.2	Wire Frame	29
3.2.3	Design developed.....	30
3.3	User management.....	31
3.3.1	Research Reference.....	31
3.3.2	Wire frame	33
3.3.3	Developed Design.....	33
3.4	Candidate Management	34
3.4.1	Research Reference.....	34
3.4.2	Wire frame	35
3.4.3	Developed Design.....	36
3.5	. Elections Gallery.....	38
3.5.1	Research Reference.....	38
3.5.2	Wire frame	40
3.5.3	Developed Design.....	41
3.6	Results Analytics	42
3.6.1	Research Reference.....	42
3.6.2	Wire frame	44
3.6.3	Developed Design.....	45
3.7	Voter Dashboard	46
3.7.1	Developed Design.....	49
3.8	Electoral Candidate	50
3.8.1	Research Reference.....	50

3.8.2	Wire frame	52
3.8.3	Developed Design	53
3.9	Cast Vote.....	54
3.9.1	Research Reference.....	54
3.9.2	Wire frame	56
3.9.3	Developed Design	57
3.10	Result	58
3.10.1	Wire frame	60
3.10.2	Developed Design	61
3.11	Profile.....	62
3.11.1	Wire frame	64
3.11.2	Developed Design	65
3.12	Candidate dashboard.....	66
3.12.1	Wireframe	68
3.12.2	Developed Design	69
3.13	Application.....	70
3.13.1	Wire frame	72
3.13.2	Developed design.....	73
3.14	Profile and manifesto	74
3.14.1	Wire frame	76
3.14.2	Developed design.....	77
3.15	Competitors.....	78
3.15.1	Wire frame	80
3.15.2	Developed design.....	81
4	Java Classes and Methods.....	82
4.1	Class Diagram.....	82
5	References.....	83

5.1	Method explanation	83
5.1.1	DAOs	83
5.1.2	Utiltiy Classes	123
5.1.3	Controller Class: AdminCandidateServlet.....	126
6	Test Cases	128
6.1	Admin Login.....	128
6.2	Dashboard Refresh Functionality	129
6.3	Add New Election.....	130
6.4	Approve Candidate Application	132
6.5	Reject Candidate Application	133
6.6	Add New Constituency	135
6.7	Constituency Required Field Validation.....	136
6.8	View Election Results.....	138
6.9	Voter Login.....	139
6.10	Candidate Login.....	141
6.11	Election Creation	143
6.12	Cast vote.....	144
6.13	apply for candidacy	146
6.14	view candidates	149
6.15	view Results	150
6.16	Register as user	151
6.17	Candidate Nomination	153
6.18	Edit candidate profile	155
6.19	Edit manifesto	157
6.20	Update profile picture	159
6.21	View Competitors.	161
7	Coursework Development with Analysis	162

7.1	Team Contribution	162
7.1.1	Sujal Kumar Majhi – Voter Portal	163
7.1.2	Nitika Mandal – Admin Portal.....	166
7.1.3	Arjun Rai – Candidate Portal.....	169
7.1.4	Bijay Khawas – Authentication, Filter & Layout	171
7.2	Sugam Khakiwada – Backend (Models, DAOs, Services & Database)	174
8	Conclusion	178
9	References.....	180

Table of Figures

Figure 1: Sample record of constituencies table	17
Figure 2: sample records of users table.....	19
Figure 3: sample record of elections	20
Figure 4: Sample records of candidate_applications table	21
Figure 5: Sample records of candidates table	23
Figure 6: sample records of votes table	23
Figure 7: Relationship Diagram of the database.....	24
Figure 8 login Research Reference	25
Figure 9 login Wireframe.....	26
Figure 10 login Design Development	27
Figure 11 register Research Reference	28
Figure 12 rigister wireframe	29
Figure 13 Design developed	30
Figure 14 User management Research Reference	31
Figure 15 user management Wire frame	33
Figure 16 user management Developed Design	33
Figure 17 Candidate Management Research Reference	34
Figure 18 candidate Wire frame	35
Figure 19 candidate Developed Design	36
Figure 20 candidate Developed Design	37
Figure 21 Elections Gallery Research Reference	38
Figure 22 Wire frame.....	40
Figure 23 Developed Design	41
Figure 24 Results Analytic Research Reference.....	43
Figure 25 Results Analytics Research Reference	43
Figure 26 Wire frame.....	44
Figure 27 Developed Design	45
Figure 28 Voter Dashboard.....	47
Figure 29 Wire Frame	48
Figure 30 Developed Design	49
Figure 31 Electoral Candidate Research Reference.....	51
Figure 32 Wire frame.....	52

Figure 33 Developed Design	53
Figure 34 Cast Vote Research Reference	55
Figure 35 Wire frame.....	56
Figure 36 Developed Design	57
Figure 37 Result.....	59
Figure 38 Wire frame.....	60
Figure 39 Developed Design	61
Figure 40 Profile	62
Figure 41 Wire frame.....	64
Figure 42 Developed Design	65
Figure 43 Candidate dashboard	67
Figure 44 Wireframe.....	68
Figure 45 Developed Design	69
Figure 46 Application	71
Figure 47 Wire frame.....	72
Figure 48 Developed design	73
Figure 49 Profile and manifesto.....	75
Figure 50 Wire frame.....	76
Figure 51 Developed design	77
Figure 52 Competitors	79
Figure 53 Wire frame.....	80
Figure 54 Developed design	81
Figure 55: Class Diagram	82
Figure 56 CandidateApplicationDAO	83
Figure 57 findByUserAndElection Code.....	83
Figure 58 findByElection Code	84
Figure 59 findByStatus Code.....	85
Figure 60 findByUser Code	86
Figure 61 CandidateDAO	87
Figure 62 findByUser Code	87
Figure 63 findByElection code	88
Figure 64 indByConstituency code.....	89
Figure 65 findByElectionAndConstituency code	90
Figure 66 findActiveByElection code	91

Figure 67 countByElection code.....	92
Figure 68 ConstituencyDAO	93
Figure 69 findByDistrict code	93
Figure 70 findByProvince code	94
Figure 71 ElectionDAO code.....	95
Figure 72 findByStatus code.....	95
Figure 73 findByType code	96
Figure 74 findActiveElectionByConstituency code	97
Figure 75 GenericDAO<T,ID>.....	98
Figure 76 findById code	98
Figure 77 findAll code.....	99
Figure 78 save code	100
Figure 79 update code.....	101
Figure 80 delete code	102
Figure 81 deleteById code	103
Figure 82 beginTransaction code.....	104
Figure 83 commitTransaction code	105
Figure 84 rollbackTransaction code.....	106
Figure 85 count code.....	107
Figure 86 close code	108
Figure 87 UserDao code	109
Figure 88 findByEmail code.....	109
Figure 89 findByCitizenshipNumber code	110
Figure 90 findByRole code.....	111
Figure 91 findByStatus code.....	112
Figure 92 findByConstituency code	113
Figure 93 countByRole code	114
Figure 94 countByStatus code	115
Figure 95 findAllWithPagination code.....	116
Figure 96 VoteDAO.....	117
Figure 97 findByElectionAndVoter code	117
Figure 98 countByElection code.....	118
Figure 99 countByCandidate code.....	119
Figure 100 hasVoted code	120

Figure 101 findVotesByElection code.....	121
Figure 102 findVotesByCandidate code.....	122
Figure 103 AppConstants	123
Figure 104 PasswordUtil.....	124
Figure 105 hashPassword code.....	124
Figure 106 verifyPassword code.....	125
Figure 107 Controller Class: AdminCandidateServlet code.....	126
Figure 108 doGet code.....	126
Figure 109 doPost code.....	127
Figure 110 Login Page Admin.....	128
Figure 111 Admin Dashboard.....	129
Figure 112: dashboard before refreshing	129
Figure 113: dashboard after refreshing	130
Figure 114: creating new election.....	131
Figure 115: created new election successfully.....	131
Figure 116: approve option.....	132
Figure 117: approved successfully.....	133
Figure 118: reject option.....	134
Figure 119: rejected successfully	134
Figure 120: adding new constituency	135
Figure 121: new constituency added successfully	136
Figure 122: empty field on purpose	137
Figure 123: form submission prevented	137
Figure 124: election result page.....	138
Figure 125 Login Page Voter.....	139
Figure 126 Voter Dashboard.....	140
Figure 127 Login Page Candidate.....	141
Figure 128 Candidate Dashboard.....	142
Figure 129 Adding Election.....	143
Figure 130 Election Record	143
Figure 131 cast vote page	144
Figure 132 vote cast done	145
Figure 133 apply-candidacy page	147
Figure 134 Nomination Submitted done.....	148

Figure 135 Candidates page.....	149
Figure 136 Results page.....	150
Figure 137 Register form.....	151
Figure 138 login after register.....	152
Figure 139 User dashboard after login.....	152
Figure 140 Candidate nomination fill up	154
Figure 141 candidate nomination error since application was already submitted prior	154
Figure 142 Before editing profile	155
Figure 143 Changes made in profile.....	156
Figure 144 result after changes are made	156
Figure 145 before editing manifesto	157
Figure 146 changes made in manifesto.....	158
Figure 147 result after changes are made	158
Figure 148 before changing profile picture	159
Figure 149 changing profile picture.....	160
Figure 150 result after profile picture is changed	160
Figure 151 view competitors	161

Table of Tables

Table 1 Entity Descriptions and Relationships	10
Table 2:Constituencies Table.....	11
Table 3: Users Table	12
Table 4: Elections Table	13
Table 5: candidate_applications Table.....	14
Table 6 Candidates Table.....	15
Table 7: Votes Table	16
Table 8: Table design of constituencies.....	17
Table 9: Table design of users	18
Table 10: Table design of elections	20
Table 11: Table design of candidate_ applications.....	21
Table 12: Table design of candidates table.....	22
Table 13: Table design of votes table	23
Table 14 Admin Login.....	128
Table 15: Dashboard refresh function.....	129
Table 16: add new election	130
Table 17: approve candidate application	132
Table 18: reject candidate application	133
Table 19: add new constituency.....	135
Table 20: constituency required field validation	136
Table 21: view election result	138
Table 22 Voter Login.....	139
Table 23 Candidate Login.....	141
Table 24 Election Creation	143
Table 25 Cast vote	144
Table 26 Apply for Candidacy.....	146
Table 27 View candidates	149
Table 28 View Results	150
Table 29 Register as User	151
Table 30 Candidate Nomination	153
Table 31 Edit Candidate profile.....	155
Table 32 Edit manifesto	157

Table 33 update profile picture	159
Table 34 View Compitators	161
Table 35 Contribution Tablesss	162

1 Introduction

At the time when any sphere of the social life is engaged in the digital revolution and is transformed, the issue of the reliable, transparent, and effective election management system is more urgent than ever before. Election is the most important foundation of democracy but the traditional election which is paper-based is prone to human error, fraud, inefficiency and low accessibility. NirvachanSetu is a Java based web-application designed to overcome such challenges by digitalizing the overall election lifecycle, that is, registration of voters and candidates, casting of votes and publishing of results. The system, inspired by organizational principles of the Election Commission of Nepal, was purposely designed to manage House of Representatives (HoR) election and be architecturally scaled up to support local and provincial elections in the future. By using Java EE technologies such as Java Servlets, JSP, JDBC, and MySQL, and based upon the Model-View- Controller (MVC) design pattern, NirvachanSetu shows how advanced concepts of programming can be used to address real-world governance issues. The platform instates vigorous democratic values - one man to one vote, constituency based voting and total anonymity of voting - combined with a smooth and easy-to-use interface to the administrator, voter and candidate.

1.1 Title

NirvachanSetu: A Java-Based Web Application for Digital Election Management

1.2 Purpose

NirvachanSetu is formulated to modernize and digitalize the election management process in a secure, transparent, and efficient way. The name of the system, Setu (meaning in Nepali), represents the essence of the system, to act as a bridge between citizens and democratic participation using the power of technology.

The organizational structure and the principles of the Election Commission of Nepal are variables that stimulate the project. It is mostly about administration of the House of Representatives (HoR) elections and is scalable enough to accommodate the local and provincial elections in the future. This platform is tailored to contain the deficiencies of the traditional paper-based election systems which are usually fraught with fraud incidents, inefficiency and absence of transparency.

With the introduction of an e-governance strategy, NirvachanSetu will strive to make sure that every qualified citizen will be in a position to participate in the election process in a convenient manner, and the administrator will be able to manage the entire election lifecycle - starting with voter registration and candidate screening, up to the publication of the results - with one unified platform.

1.3 Audience

NirvachanSetu aims at target audiences listed below:

- Election Administrators: Personnel who do the setup and the administration of elections, checking of voter and candidate registration as well as the administration of constituencies, monitoring of voting activities and publishing of election results.
- Registered Voters: Accredited citizens falling within the eligible categories, which are required to be registered on the platform, verified by the admin and cast their votes throughout the election phase within the assigned constituency.
- Candidates: These are people who apply to challenge elections in a particular constituency, submit their report and manifestos to be approved and follow the election process.
- Academic Evaluators: Itahari International College and London Metropolitan University faculty members and course assessors who will evaluate the project as a part of the CS5054NT Advanced Programming and Technologies module.

1.4 Aims and Objectives

1.4.1 Aim:

To create a secure, scalable and user friendly web application based on Java that will digitize and simplify the election management process and ensure transparency, integrity and ease of participation by all the stakeholders.

1.4.2 Objectives:

Design and implement a normalized relational database schema using MySQL to describe all the necessary entities of voting, running, electing and constituency.

- To create a role-based access control (RBAC) solution, which imposes specific permissions on the administrators, voters and candidates.
- To establish secure authentication controls, such as password encryption and session control, to safeguard user information and provide only authenticated access.
- This was done to create an out of the box Admin Dashboard which will help in supporting the entire life cycle of election (including user registration approval up until result publication).
- To create a voter portal that can enable registered voters to view candidates and filter by constituency and party and cast one anonymous vote per election.
- In order to develop a candidate portal over which applicants have access and are able to make submissions, publish manifestos, and track the status of the election.
- In order to implement the Model-View-Controller (MVC) architectural pattern with the help of Java Servlets, JSP and a layered package structure (model, view, controller, service, util) to achieve clean separation of responsibilities.
- To create a responsive user interface with JSP and CSS (with CSS media queries and CSS Flexbox) that would be user friendly in both desktops and mobile devices without any external CSS frameworks.
- To provide one-person-one-vote integrity and anonymous voting to safeguard democratic values and curtail electoral frauds.

1.5 List of Features

People find the following features adopted in NirvachanSetu:

1.5.1 Admin Features:

- Voter Registration Approval: The Admins are able to approve or disapprove the voter registrations which are verified by readmission of submitted legal documents.
- Application Management: Admins will look through and approve or disapprove applicant applications to certain constituencies.
- Election Creation and Management Admins create, configure and maintain House of Representatives elections that have a specific start and end date.
- Constituency Management: Admins establish constituencies and allocate them to the elections.
- Election Monitoring: With this, admins are in a position to monitor the status and progress of ongoing elections in real time.
- Publication of Results: Official election results should be posted by the admins, and that would be at a time when the voting would be complete.
- User Account Management: The admins are able to manage user accounts as well as force re-approval of user profile upon any update.

1.5.2 Voter Features:

Voter Registration: Registered users use valid legal documents and are awaited to be given the administration.

- Voter: Voters will have an encrypted user name and password and access to the site is usually restricted by the control of a session.
- Profile Management: Voters will have the ability to update and see their profile details, which will require re-approval by an administrator.
- Candidate Discovery: Voters can see the candidates in the constituency that they are assigned.
- Search and Filter: Voters are allowed to search and filter the names, party, or constituency of the candidates.
- Candidate Profiles and Manifestos: Voters can view in-depth candidates profiles and read their manifestos.
- Anonymous Vote Casting: This is voting by a person who directly engulfs the election cast.

- One-Vote Enforcement: The system is very strict in applying one vote per voter, election.

1.5.3 Candidate Features:

- Candidate Application: This is done after a user submits necessary documents to contest elections in a given constituency.
- Manifesto Management: Candidates are able to post and update their campaign manifestos.
- Competing Candidate View: Candidates are able to look at the profiles of other candidates in his or her constituency.
- Dual Role Participation: Candidates do not give up their voter status, and may continue to cast a single vote just like any other registered voter.
- Election Status Tracking: Candidates will be able to follow the current situation of the election they are taking part in.

1.5.4 System-Wide Features:

- Role-Based Access Control (RBAC): Imposes different access and permission levels among admins, voters and candidates to all modules in the system.
- Secure Password Encryption: To ensure protection against unauthorized access, hash and encryption of user passwords are done before being written to disk, or sent to a network or the Internet.
- Responsive User Interface: Built using JSP with CSS, with Flexbox and media queries to accommodate desktop and mobile devices.
- Live Voting Countdown Timer: It is a countdown timer which is used to notify the users of the amount of time left to the active election period.
- Scalable Architecture: The system is structured to support beyond HoR election to representing local and provincial election in the future.
- Redirect Management (Filters): Java Filters manage any unauthorized access attempts redirecting any user to the login or error page.

2 Database design and development

Database Design and Development is the process of developing a structured system to store, organize and efficiently manage data. A database is similar to a very efficient filing cabinet in which all information is stored in one place and can be easily accessed anytime. Good database design ensures that data is not unnecessarily duplicated, is accurate over time and that the system is expandable as the volume of data increases. Developers and designers collaborate to gain insight into what data the business requires, the relationships between pieces of data, and how the data will be leveraged on a day-to-day basis.

Typically, the process begins with an understanding of the requirements of the business or application. This means asking questions like: what information needs to be stored, who will be accessing it, and what kind of operations will be performed on it. When the requirements are defined, the designer makes a plan or blueprint of the database before any coding starts. This is an important planning stage because it will be much easier and cheaper to correct any errors in the design than it will be once the database is constructed and data is entered into it.

2.1 Entity Relationship Diagram (ERD) Design with Business Rules

An Entity Relationship Diagram (ERD) is a graphical representation of the relationships between data elements in a database. One of the most significant tools adopted in the database design stage. Rather than having to write a lengthy document explaining the structure of a database, designers create a diagram that can be understood by anyone in relatively short order.

Business rules are specific rules and constraints that indicate the operation of an organization and the way that data should behave. Each organization has its own set of rules and these rules need to be converted into the database as to enable the system to enforce them automatically. Defining clearly the business rules is one of the most crucial steps in database development because these rules will affect the structure of the database as well as the validation of the data within the database.

Main Entities and their attributes

1. constituencies

Attributes: id (PK), name, district, province, total_voters, description, created_at, updated_at

2. users

Attributes: id (PK), full_name, email (UQ), password, citizenship_number (UQ), phone, role, status, profile_image, address, constituency_id (FK), created_at, updated_at

3. elections

Attributes: id (PK), name, description, election_type, start_date, end_date, status, constituency_id (FK nullable), created_at, updated_at

4. candidate_applications

Attributes: id (PK), user_id (FK), election_id (FK), constituency_id (FK), party_name, party_type, manifesto, symbol, status, applied_at, updated_at

5. candidates

Attributes: id (PK), user_id (FK), application_id (FK UQ), election_id (FK), constituency_id (FK), party_name, party_type, manifesto, symbol, total_votes, status, created_at, updated_at

6. votes

Attributes: id (PK), election_id (FK), candidate_id (FK), constituency_id (FK), voter_id (FK), voted_at

Core Business Rules:

- A user belongs to one constituency, but a constituency can have multiple users.
- A user can be able to apply as a candidate in many different elections, but only one time in one election.
- Only approved applications can become candidates.
- A candidate is directly related to election and one constituency.
- A voter can vote only one time in a election.
- Each vote is cast for a single candidate in a specific election.
- Elections can be national, provincial, or local.

2.1.1 Entity Descriptions and Relationships:

The below shows all six entities, their respective primary keys, foreign keys, and the cardinality of every relationship in the database.

Table 1 Entity Descriptions and Relationships

Entity A	Cardinality	Entity B	Description
Constituency	1 : M	User	One constituency has many registered users/voters.
Constituency	1 : M	Election	One constituency may host multiple elections.
Constituency	1 : M	Candidate Application	One constituency is referenced in many applications.
Constituency	1 : M	Candidate	One constituency has many approved candidates.
Constituency	1 : M	Vote	One constituency is linked with many votes cast.
User	1 : M	Candidate Application	One user may apply for candidacy in many elections.
User	1 : M	Candidate	One user may be applicable to become a candidate in many elections.
User	1 : M	Vote	One user as a voter casts one vote per election.
Election	1 : M	Candidate Application	One election can receives many candidate applications.
Election	1 : M	Candidate	One election has multiple approved candidates.
Election	1 : M	Vote	One election records many votes.
Candidate Application	1 : 1	Candidate	One approved application creates exactly one candidate record.
Candidate	1 : M	Vote	One candidate gets many votes.

2.1.2 Normalization Validation up to Third Normal Form (3NF)

Normalization in the database is the process of creating the relational database fully structured to decrease or remove redundancy in data and improve data integrity.

Every table is properly analysed for First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) as given below:

1. Table: Constituencies

Table 2: Constituencies Table

Attribute	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key for each constituency
name	VARCHAR(150)	NOT NULL	Name of the constituency (e.g., Kathmandu-1)
district	VARCHAR(100)	NOT NULL	District in which the constituency belongs
province	VARCHAR(100)	NOT NULL	Province in which the Constituency lies
total_voters	INT	DEFAULT 0	Total number of voters who are registered in the constituency
description	TEXT	NULLABLE	Optional detailed description
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record timestamp of creation
updated_at	TIMESTAMP	AUTO ON UPDATE	Record timestamp of last-modified

Normalization Analysis:

1NF: All attributes of this table has atomic, which means single-valued data. There are no repeating groups and every row is uniquely identified by primary key id.

2NF: There are no partial dependency and each non-key attribute such as name, district, province, total_voters, description is completely functionally dependent on pk.

3NF: There are no transitive dependencies which means no non-key attribute determines another non-key attribute.

Redundancy Check: All the Constituency data is stored only once and all other tables in the database take reference through foreign key of it.

2. Table: Users

Table 3: Users Table

Attribute	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key
full_name	VARCHAR(100)	NOT NULL	Full legal name of the user
email	VARCHAR(150)	NOT NULL, UNIQUE	Unique email address of the user
password	VARCHAR(255)	NOT NULL	Password of the account
citizenship_number	VARCHAR(50)	NOT NULL, UNIQUE	Nepal citizenship certificate number
phone	VARCHAR(20)	NULLABLE	Contact phone number
role	ENUM	NOT NULL, DEFAULT VOTER	Admin or Voter or Candidate
status	ENUM	NOT NULL, DEFAULT PENDING	Pending or Approved or Rejected or Suspended
profile_image	VARCHAR(500)	NULLABLE	Path/URL to profile image.
address	TEXT	NULLABLE	Residential address
constituency_id	INT	FK	Constituency the user belongs to
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Registration timestamp
updated_at	TIMESTAMP	AUTO ON UPDATE	Last-update timestamp

Normalization Analysis:

1NF: Atomic values

2NF: All attributes depend completely on PK

3NF: No transitive attributes

3. Table: Elections

Table 4: Elections Table

Attribute	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique pr key
name	VARCHAR(200)	NOT NULL	Name of the election
description	TEXT	NULLABLE	Detailed description
election_type	ENUM	NOT NULL, DEFAULT HOR	HOR or Provincial or Local
start_date	DATETIME	NOT NULL	Voting start date and time.
end_date	DATETIME	NOT NULL	Voting end date and time.
status	ENUM	NOT NULL, DEFAULT DRAFT	Draft or Registration or Active or Completed or Cancelled
constituency_id	INT	FK (NULLABLE)	Target constituency
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record timestamp of creation
updated_at	TIMESTAMP	AUTO ON UPDATE	Last-update timestamp

Normalization Analysis:

- Completely dependent on PK
- No transitive dependencies

4. Table: candidate_applications

Table 5: candidate_applications Table

Attribute	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key
user_id	INT	FK → users(id), NOT NULL	The user who applies as a candidate
election_id	INT	FK → elections(id), NOT NULL	The election which gets applied for
constituency_id	INT	FK → constituencies(id), NOT NULL	Constituency in which the candidate is contesting in
party_name	VARCHAR(200)	NULLABLE	Political party name
party_type	ENUM	DEFAULT INDEPENDENT	National or Regional or Independent
manifesto	TEXT	NULLABLE	Manifesto of Candidate's election
symbol	VARCHAR(100)	NULLABLE	Election symbol dedicated to the candidate.
status	ENUM	NOT NULL, DEFAULT PENDING	Pending or Approved or Rejected or Withdrawn
applied_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Application submission timestamp
updated_at	TIMESTAMP	AUTO ON UPDATE	Last-update timestamp
UNIQUE	—	unique_application (user_id, election_id)	Prevents a user from applying double to the same election

Normalization Analysis:

- Composite key (user_id + election_id) discards duplication
- No redundant data

5. Table: candidates*Table 6 Candidates Table*

Attribute	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key
user_id	INT	FK NOT NULL	The user as the candidate.
application_id	INT	FK NOT NULL, UNIQUE	The application which is approved
election_id	INT	FK → elections(id), NOT NULL	The election in which the candidate is contesting
constituency_id	INT	FK → constituencies(id), NOT NULL	The constituency represented by the candidate
party_name	VARCHAR(200)	NULLABLE	Political party name
party_type	ENUM	DEFAULT INDEPENDENT	National or Regional or Independent
manifesto	TEXT	NULLABLE	Manifesto of the Candidate
symbol	VARCHAR(100)	NULLABLE	Election symbol
total_votes	INT	DEFAULT 0	The total vote count for this candidate in this election.
status	ENUM	NOT NULL, DEFAULT ACTIVE	Active or Disqualified or Withdrawn
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	AUTO ON UPDATE	Last-update timestamp
UNIQUE	—	unique_candidate_per_election (user_id, election_id)	Ensures one candidate record per user per election

Normalization Analysis:

- Atomic values in 1NF.
- No duplicate data due to constraints applied.

6. Table : Votes*Table 7: Votes Table*

Attribute	Data Type	Constraint	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique primary key
election_id	INT	FK → elections(id), NOT NULL	Election in which the vote was casted
candidate_id	INT	FK → candidates(id), NOT NULL	Candidate who received the vote.
constituency_id	INT	FK → constituencies(id), NOT NULL	Constituency in which the vote was casted
voter_id	INT	FK → users(id), NOT NULL	The voter who casts the vote
voted_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp of when the vote was casted
UNIQUE	—	unique_vote (election_id, voter_id)	Ensures each voter can vote only once per election

Normalization Analysis:

- No duplicate votes due to unique constraint.
- No redundant attributes.

2.2 Database Development

The database of NirvachanSetu named nirvachansetu_db was developed by using MySQL. Here, the table along with their design and sample data which was inserted while development and testing are given below:

2.2.1 Table 1: constituencies

This table in the database is responsible to store all the electoral constituencies all over Nepal. It is the foundation table which the entire system takes reference from.

Table Design:

Table 8: Table design of constituencies

Column	Type	Null	Key	Default	Extra
id	INT	NO	PK	—	AUTO_INCREMENT
name	VARCHAR(150)	NO	—	—	
district	VARCHAR(100)	NO	—	—	
province	VARCHAR(100)	NO	—	—	
total_voters	INT	YES	—	0	
description	TEXT	YES	—	NULL	
created_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP

Sample Records:

9 • `SELECT * FROM nirvachansetu_db.constituencies;`

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

id	name	district	province	total_voters	description	created_at	updated_at
1	Kathmandu-1	Kathmandu	Bagmati Province	125000	Constituency covering central Kathmandu	2026-05-03 17:42:12	2026-05-03 17:42:12
2	Kathmandu-2	Kathmandu	Bagmati Province	118000	Constituency covering eastern Kathmandu	2026-05-03 17:42:12	2026-05-03 17:42:12
3	Kathmandu-3	Kathmandu	Bagmati Province	132000	Constituency covering northern Kathmandu	2026-05-03 17:42:12	2026-05-03 17:42:12
4	Lalitpur-1	Lalitpur	Bagmati Province	95000	Constituency covering central Lalitpur	2026-05-03 17:42:12	2026-05-03 17:42:12
5	Bhaktapur-1	Bhaktapur	Bagmati Province	88000	Constituency covering Bhaktapur district	2026-05-03 17:42:12	2026-05-03 17:42:12
6	Pokhara-1	Kaski	Gandaki Province	110000	Constituency covering Pokhara metropolitan	2026-05-03 17:42:12	2026-05-03 17:42:12
7	Biratnagar-1	Morang	Koshi Province	105000	Constituency covering Biratnagar area	2026-05-03 17:42:12	2026-05-03 17:42:12
8	Chitwan-1	Chitwan	Bagmati Province	97000	Constituency covering Chitwan district	2026-05-03 17:42:12	2026-05-03 17:42:12
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 1: Sample record of constituencies table

2.2.2 Table 2: Users

This table is responsible for storing all users in the system such as admins, voters and candidates. It maintains the authorization, authentication and identity verification data.

Table Design:

Table 9: Table design of users

Column	Type	Null	Key	Default	Extra
id	INT	NO	PK	—	AUTO_INCREMENT
full_name	VARCHAR(100)	NO	—	—	
email	VARCHAR(150)	NO	UNIQUE	—	
password	VARCHAR(255)	NO	—	—	SHA-256 hash
citizenship_number	VARCHAR(50)	NO	UNIQUE	—	
phone	VARCHAR(20)	YES	—	NULL	
role	ENUM	NO	INDEX	VOTER	ADMIN VOTER CANDIDATE
status	ENUM	NO	INDEX	PENDING	PENDING APPROVED REJECTED SUSPENDED
profile_image	VARCHAR(500)	YES	—	NULL	
address	TEXT	YES	—	NULL	
constituency_id	INT	YES	FK/INDEX	NULL	constituencies(id)
created_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP

Sample Records:

9 • `SELECT * FROM nirvachansetu_db.users;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: `FA`

id	full_name	email	password	citizenship_number	phone	role	status	profile_image	ad
1	System Administrator	admin@nirvachansetu.com	590b00d332fb0fd02f0e407b199167a52859008...	ADMIN-001	9800000001	ADMIN	APPROVED		Kat
2	Rajesh Sharma	rajesh@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-001-001	9801000001	VOTER	APPROVED		Kat
3	Sita Kumari	sita@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-001-002	9801000002	VOTER	APPROVED		Kat
4	Hari Bahadur	hari@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-001-003	9801000003	VOTER	APPROVED		Kat
5	Anita Devi	anita@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-001-004	9801000004	VOTER	PENDING		Lalit
6	Ram Prasad	ram@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-001-005	9801000005	VOTER	APPROVED		Kat
7	Dr. Anjali Verma	anjali@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-002-001	9802000001	VOTER	APPROVED		Kat
8	Prakash Hegde	prakash@email.com	0704177628f388f34487a9256c3bcce8c9f7eb60...	CIT-002-002	9802000002	VOTER	APPROVED		Kat

Figure 2: sample records of users table

2.2.3 Table 3: elections

This table stores every elections which are managed by the system. It stores election type, temporal boundaries, and lifecycle status.

Table Design:

Table 10: Table design of elections

Column	Type	Null	Key	Default	Extra
id	INT	NO	PK	—	AUTO_INCREMENT
name	VARCHAR(200)	NO	—	—	
description	TEXT	YES	—	NULL	
election_type	ENUM	NO	INDEX	HOR	HOR PROVINCIAL LOCAL
start_date	DATETIME	NO	—	—	
end_date	DATETIME	NO	—	—	
status	ENUM	NO	INDEX	DRAFT	DRAFT REGISTRATION ACTIVE COMPLETED CANCELLED
constituency_id	INT	YES	FK	NULL	→ constituencies(id)
created_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP

Sample Records:

The screenshot shows a database query result for the query `SELECT * FROM nirvachansetu_db.elections;`. The result is displayed in a table with columns: id, name, description, election_type, start_date, end_date, status, and constituency_id. Two records are shown:

id	name	description	election_type	start_date	end_date	status	constituency_id
1	House of Representatives Election 2024	General election for the House of Representatives	HOR	2024-05-12 08:00:00	2024-05-12 17:00:00	COMPLETED	NULL
2	Provincial Assembly Election 2025	Provincial assembly election for Bagmati Province	PROVINCIAL	2025-06-15 08:00:00	2025-06-15 17:00:00	ACTIVE	NULL

Figure 3: sample record of elections

2.2.4 Table 4: candidate_applications

This table is responsible to store all the requests which are submitted by the users in a specific election. Each application is reviewed by admin and updates their status accordingly.

Table Design:

Table 11: Table design of candidate_applications

Column	Type	Null	Key	Default	Extra
id	INT	NO	PK	—	AUTO_INCREMENT
user_id	INT	NO	FK/IDX	—	users(id)
election_id	INT	NO	FK/IDX	—	elections(id)
constituency_id	INT	NO	FK	—	constituencies(id)
party_name	VARCHAR(200)	YES	—	NULL	
party_type	ENUM	YES	—	INDEPENDENT	NATIONAL REGIONAL INDEPENDENT
manifesto	TEXT	YES	—	NULL	
symbol	VARCHAR(100)	YES	—	NULL	
status	ENUM	NO	INDEX	PENDING	PENDING APPROVED REJECTED WITHDRAWN
applied_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP
UNIQUE KEY	—	—	UNIQUE	—	unique_application (user_id, election_id)

Sample Records:

```
7 • SELECT * FROM nirvachansetu_db.candidate_applications;
```

id	user_id	election_id	constituency_id	party_name	party_type	manifesto	symbol	status	applied_at
1	6	1	1	Nepal Democratic Party	NATIONAL	I commit to improving education, healthcare, an...	Elephant	APPROVED	2026-05-03 17:42:12
2	7	1	1	Progressive Alliance	REGIONAL	As your representative, I will focus on agricultu...	Lotus	APPROVED	2026-05-03 17:42:12
3	8	1	1	NULL	INDEPENDENT	I stand for transparency, accountability, and g...	Lantern	APPROVED	2026-05-03 17:42:12
4	9	1	2	National Unity Forum	NATIONAL	Unity in diversity. I will work towards national in...	Sun	PENDING	2026-05-03 17:42:12
5	6	2	1	Nepal Democratic Party	NATIONAL	Building on my previous experience, I will furthe...	Elephant	APPROVED	2026-05-03 17:42:12
6	7	2	1	Progressive Alliance	REGIONAL	With a proven track record, I will continue fighti...	Lotus	APPROVED	2026-05-03 17:42:12

Figure 4: Sample records of candidate_applications table

2.2.5 Table 5: candidates

This table stores candidates who are officially approved for the election. Records are stored in this table when the corresponding candidate_application is approved by an admin.

Table Design:

Table 12: Table design of candidates table

Column	Type	Null	Key	Default	Extra
id	INT	NO	PK	—	AUTO_INCREMENT
user_id	INT	NO	FK/IDX	—	→ users(id)
application_id	INT	NO	FK/UNI	—	→ candidate_applications(id)
election_id	INT	NO	FK/IDX	—	→ elections(id)
constituency_id	INT	NO	FK	—	→ constituencies(id)
party_name	VARCHAR(200)	YES	—	NULL	
party_type	ENUM	YES	—	INDEPENDENT	NATIONAL REGIONAL INDEPENDENT
manifesto	TEXT	YES	—	NULL	Approval-time snapshot
symbol	VARCHAR(100)	YES	—	NULL	
total_votes	INT	YES	—	0	Incremented on each vote
status	ENUM	NO	—	ACTIVE	ACTIVE DISQUALIFIED WITHDRAWN
created_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	
updated_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP
UNIQUE KEY	—	—	UNIQUE	—	unique_candidate_per_election (user_id, election_id)

Sample Records:

id	user_id	application_id	election_id	constituency_id	party_name	party_type	manifesto	symbol	total_votes	status
1	6	1	1	1	Nepal Democratic Party	NATIONAL	I commit to improving education, healthcare, an...	Elephant	3420	ACTIVE
2	7	2	1	1	Progressive Alliance	REGIONAL	As your representative, I will focus on agricultu...	Lotus	2850	ACTIVE
3	8	3	1	1	INDEPENDENT	INDEPENDENT	I stand for transparency, accountability, and g...	Lantern	1580	ACTIVE
4	6	5	2	1	Nepal Democratic Party	NATIONAL	Building on my previous experience, I will furthe...	Elephant	0	ACTIVE
5	7	6	2	1	Progressive Alliance	REGIONAL	With a proven track record, I will continue fight...	Lotus	0	ACTIVE
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 5: Sample records of candidates table

2.2.6 Table 6: votes

This table is responsible for recording each casted vote by linking to the voter, the candidate, the election and the constituency.

Table Design:

Table 13: Table design of votes table

Column	Type	Null	Key	Default	Extra
id	INT	NO	PK	—	AUTO_INCREMENT
election_id	INT	NO	FK	—	elections(id)
candidate_id	INT	NO	FK	—	candidates(id)
constituency_id	INT	NO	FK	—	constituencies(id)
voter_id	INT	NO	FK	—	users(id)
voted_at	TIMESTAMP	YES	—	CURRENT_TIMESTAMP	Auto-set on INSERT
UNIQUE KEY	—	—	UNIQUE	—	unique_vote (election_id, voter_id)

Sample Records:

id	election_id	candidate_id	constituency_id	voter_id	voted_at
1	1	1	1	2	2026-05-03 17:42:13
2	1	2	1	3	2026-05-03 17:42:13
3	1	1	1	5	2026-05-03 17:42:13
*	NULL	NULL	NULL	NULL	NULL

Figure 6: sample records of votes table

2.3 Relationship Diagram of the overall system

The Relationship Diagram was created from MySQL workbench which confirms all the foreign key relationships in the database visually. It is as given below:

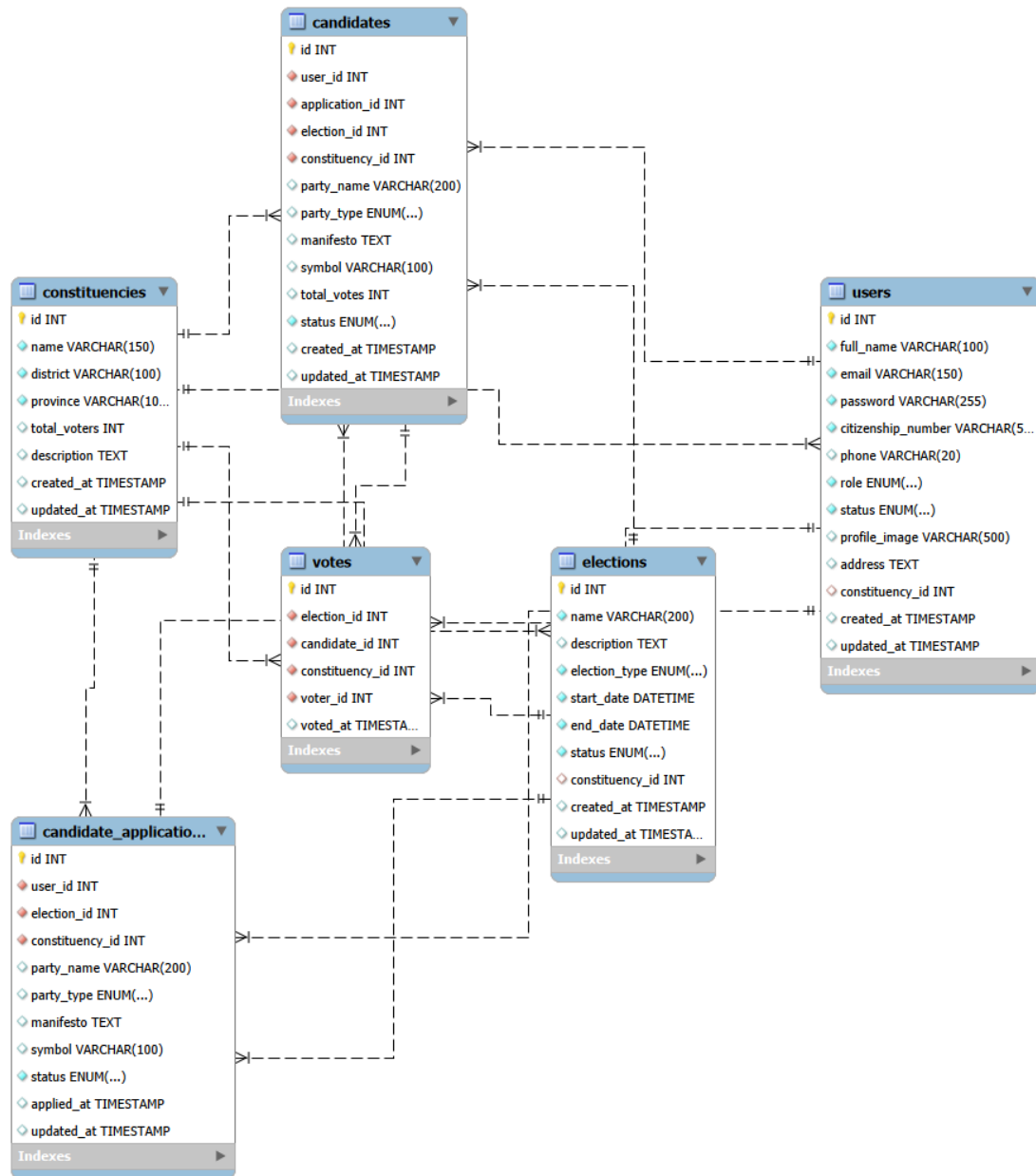


Figure 7: Relationship Diagram of the database

3 Wireframe and Design

Wireframe is a simple visual display of the elements on a web site or a sketch of our web site. Wireframing is the process of designing a web site service at a structure level. Wireframes are applied at the initial stages of the development process to define the fundamental layout of a page and then visual design and content should be incorporated.

3.1 Login

3.1.1 Research Reference

The login interface was drawn, with the help of the analysis of the modern authentication systems that are oriented towards simplicity, security, and user convenience. It was intended to make a clean and professional login page that would be appropriate on a digital government site. Systems like those provided by the Election Commission of Nepal in an official manner were examined to gain an insight into secure access design (Election Commission of Nepal, 2024). Moreover, Google Facebook login interfaces were also examined; to note the way user credentials are processed with a minimum complexity. The principles of usability by Nielsen Norman Group were taken into consideration to make the site easy to understand and navigate. (Nielsen Norman Group, 2024) Layout and styling was inspired with reference of Facebook login page. (Meta Platforms Inc, 2024)

Researched UI Screenshot.

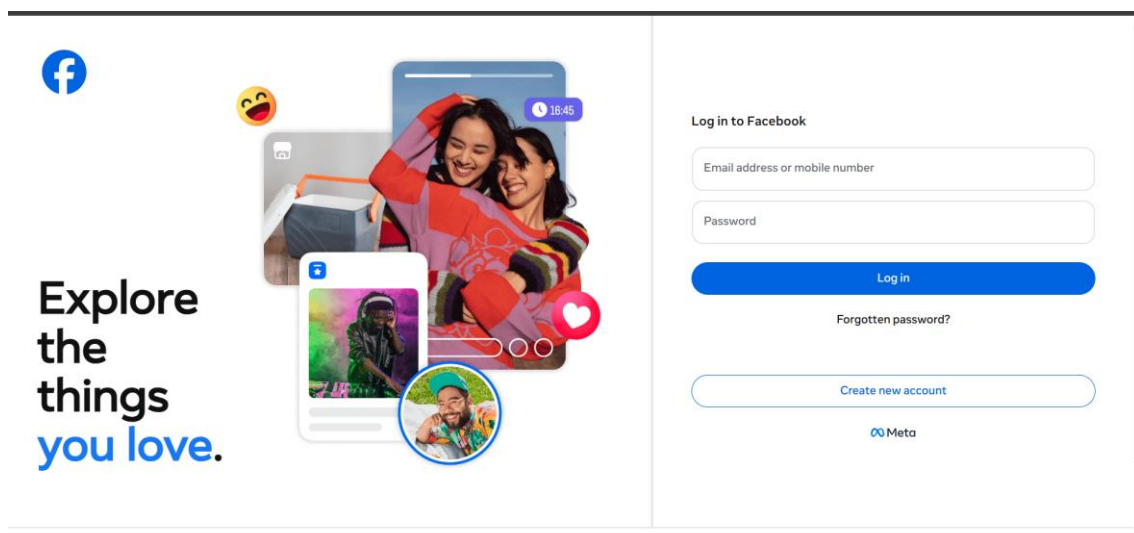


Figure 8 login Research Reference

Notable Things Taken as a point of reference

- Clean two-part design (information panel and log-in form)
- Uncomplicated and targeted fields of login (Email and Password))
- Remember me option to make it convenient.
- Addition of Forgot Password option.
- Definite Login button (CTA))
- Professional and coherent color scheme.
- Minimum design to make the design easy to use
- New user instructions (Register Now option)

3.1.2 Wireframe

NirvachanSetu

Empowering the Digital Democracy.

Access your secure voting portal, track candidates, and participate in the future of governance.

Secure Authentication
Your identity is protected by multi-layer encryption and EC-standard security protocols.

Welcome Back
Please enter your credentials to access your account.

Voter ID / Email
Enter Voter ID or Email

Password [Forgot Password?](#)
.....

☐ Remember this device

Login →

or

New to the platform? [Register Now](#)

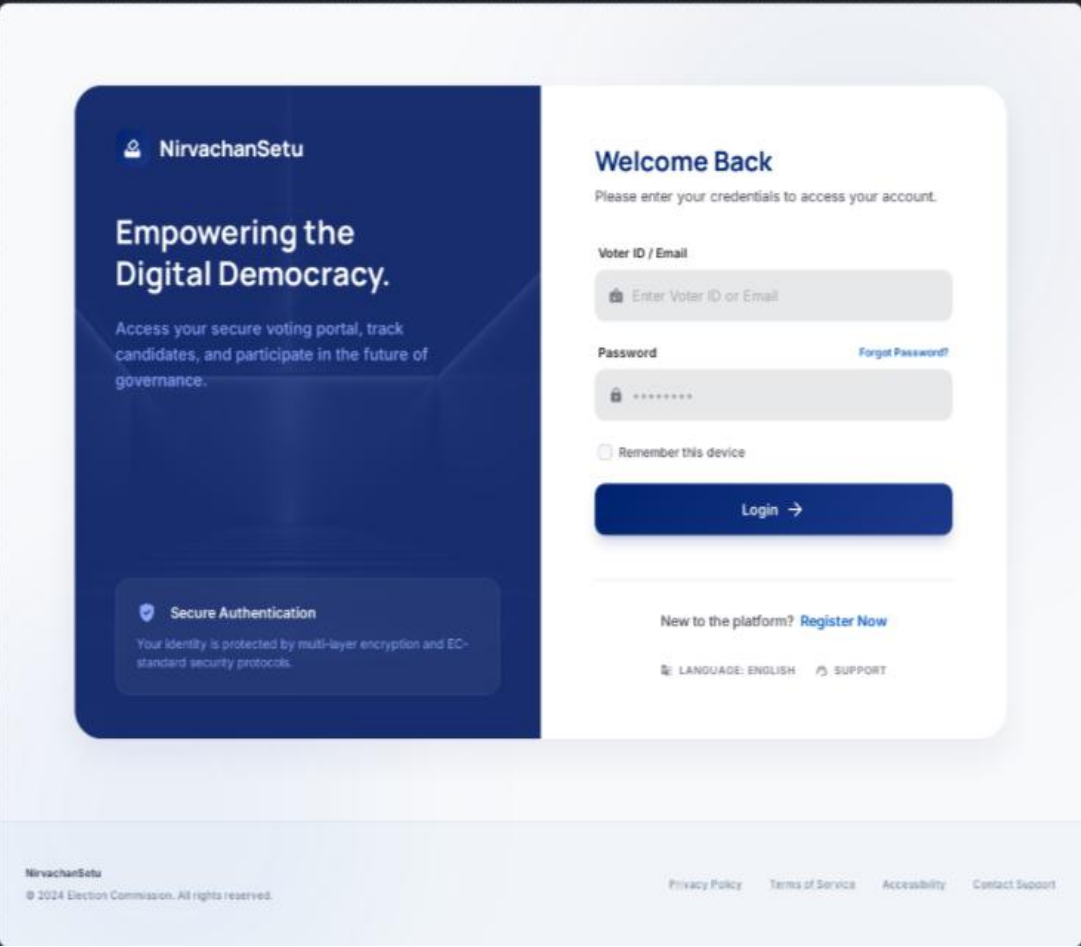
LANGUAGE: ENGLISH | SUPPORT

NirvachanSetu
© 2024 Election Commission. All rights reserved.

[Privacy Policy](#) [Terms of Service](#) [Accessibility](#) [Contact Support](#)

Figure 9 login Wireframe

3.1.3 Design Development



The image shows a login page for 'NirvachanSetu'. The page is divided into two main sections: a dark blue left panel and a white right panel. The left panel contains the logo 'NirvachanSetu' with a person icon, the headline 'Empowering the Digital Democracy.', a sub-headline 'Access your secure voting portal, track candidates, and participate in the future of governance.', and a 'Secure Authentication' section stating 'Your identity is protected by multi-layer encryption and EC-standard security protocols.' The right panel is titled 'Welcome Back' and asks the user to 'Please enter your credentials to access your account.' It features input fields for 'Voter ID / Email' (with a placeholder 'Enter Voter ID or Email') and 'Password' (with a placeholder '*****' and a 'Forgot Password?' link). There is a 'Remember this device' checkbox and a 'Login →' button. Below the login button, there is a link 'New to the platform? Register Now' and footer links for 'LANGUAGE: ENGLISH' and 'SUPPORT'. The footer of the page includes the copyright notice '© 2024 Election Commission. All rights reserved.' and links for 'Privacy Policy', 'Terms of Service', 'Accessibility', and 'Contact Support'.

NirvachanSetu

Empowering the Digital Democracy.

Access your secure voting portal, track candidates, and participate in the future of governance.

Secure Authentication

Your identity is protected by multi-layer encryption and EC-standard security protocols.

Welcome Back

Please enter your credentials to access your account.

Voter ID / Email

Enter Voter ID or Email

Password [Forgot Password?](#)

Remember this device

Login →

New to the platform? [Register Now](#)

LANGUAGE: ENGLISH SUPPORT

NirvachanSetu
© 2024 Election Commission. All rights reserved.

[Privacy Policy](#) [Terms of Service](#) [Accessibility](#) [Contact Support](#)

Figure 10 login Design Development

3.2 Registration

3.2.1 Research Reference

To design the registration page of the NirvachanSetu system, the contemporary and popular registration interfaces were researched, especially the Instagram Registration Page. Instagram was selected as a reference, as it adheres to a minimal user-friendly and secure design and effectively gathers necessary user data (Meta Platforms Inc, 2024).

Besides visual layout, special consideration was given to identity verification mechanisms to make sure that only legitimate citizens will be able to register in the system. NirvachanSetu is an election-based system, and due to the necessity of verifying citizenship, an additional document upload feature was introduced to the system, which is more than the standard practice of registering social media accounts.

Researched UI Screen shot.

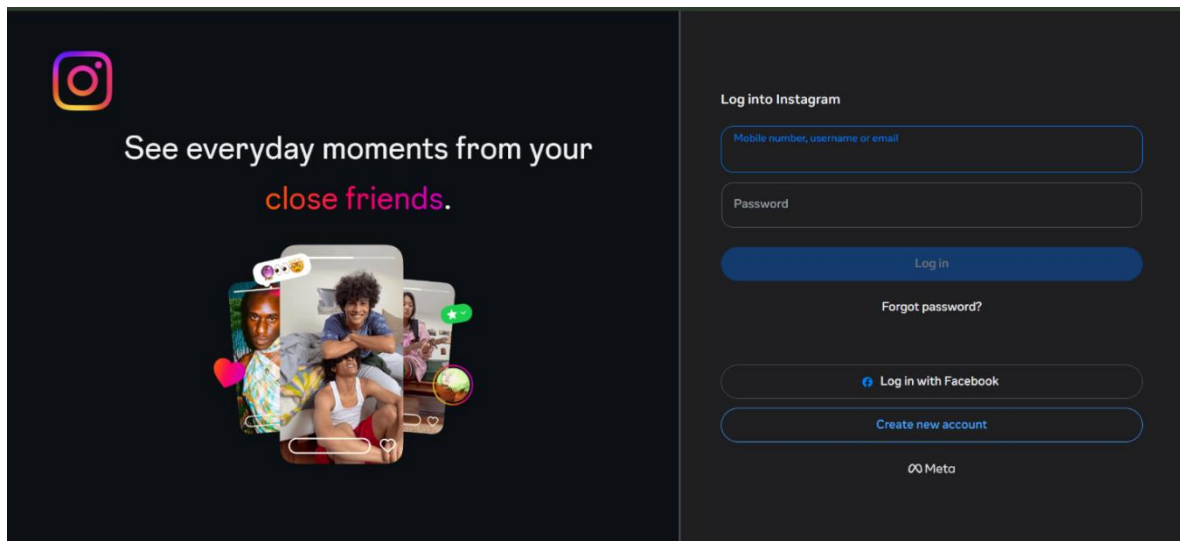


Figure 11 register Research Reference

The Design was influenced by key points which have been discussed below.

Our system borrowed and improved the following aspects of the studied UI:

- Minimalist and clean design to enhance user focus.
- Well labelled input fields.
- Data collection to be done step-by-step to minimize user confusion.
- Handling of inputs of secure passwords.
- Clearly defined call-to-action buttons (e.g., Sign Up / Complete Registration)

- The focus on the user trust by a simple and professional design.
- Form structure: Responsive and accessible form structure.

Other Improvement in our System.

In contrast to Instagram, NirvachanSetu involves checking citizenship. In this regard, a document upload form was included to make sure that the registered user is an authenticated citizen of the nation. This enhances credibility of the systems, helps eliminate bogus registration and helps in carrying out fair election practices.

The wireframe was created, to visualize the layout and structure of the registration page, prior to implementation. It lays emphasis on the content positioning and not colours or style.

3.2.2 Wire Frame

The wireframe includes:

- User information (name, email, password)
- Citizenship number input
- Constituency selection dropdown
- Document upload section
- Registration submission button

NirvachanSetu Support

Join the Digital Bastion

Create your secure identity to participate in the democratic process. Your data is protected by sovereign-grade encryption.

VERIFIED

Official Election Portal

SECURE

End-to-end encrypted data

Create Account

Please provide your official details to register as a voter.

FULL NAME

As per citizenship ID

EMAIL ADDRESS

name@example.com

PASSWORD

CITIZENSHIP NUMBER

D-000-000-000

CONSTITUENCY

Select your registered constituency

LEGAL IDENTIFICATION PROOF

Click to upload or drag and drop

PDF, PNG or JPG (Max. 10MB)

Complete Registration →

Already have an account? [Sign In](#)

Figure 12 register wireframe

3.2.3 Design developed

The screenshot displays the 'NirvachanSetu' 'Create Account' interface. On the left, a dark blue sidebar titled 'Join the Digital Bastion' encourages users to create a secure identity for the democratic process, noting that data is protected by sovereign-grade encryption. It also features 'VERIFIED' (Official Election Portal) and 'SECURE' (End-to-end encrypted data) badges. The main form area, titled 'Create Account', requests official details for voter registration. It includes input fields for 'FULL NAME' (with a placeholder 'As per citizenship ID'), 'EMAIL ADDRESS' (placeholder 'name@example.com'), 'PASSWORD' (masked with dots), and 'CITIZENSHIP NUMBER' (placeholder 'ID-000-000-000'). A dropdown menu for 'CONSTITUENCY' prompts the user to 'Select your registered constituency'. Below these is a 'LEGAL IDENTIFICATION PROOF' section with a dashed border and an upload icon, accompanied by the text 'Click to upload or drag and drop (PDF, PNG or JPG (max. 10MB))'. A prominent blue 'Complete Registration' button is at the bottom, with a link to 'Sign in' for existing users. The page header includes the 'NirvachanSetu' logo and a 'Support' link.

Figure 13 Design developed

3.3 User management

3.3.1 Research Reference

To design the User Management page of the NirvachanSetu system, research has been conducted on the modern interfaces of an admin dashboard, which handles a large amount of user data, the registration processes, and status tracking through approvals. Emphasis was put on platforms that maintain citizen or voter records with multi-status pipelines (Pending, Approved, Rejected) as these are very similar to those needed by an election management system.

Reference Website

The first resource that was used to develop this design is the AdminLTE Dashboard Template, an open-source web-based administration panel that is widely adopted and has been used to directly influence the layout and structure of the User Management page.

Reference Site

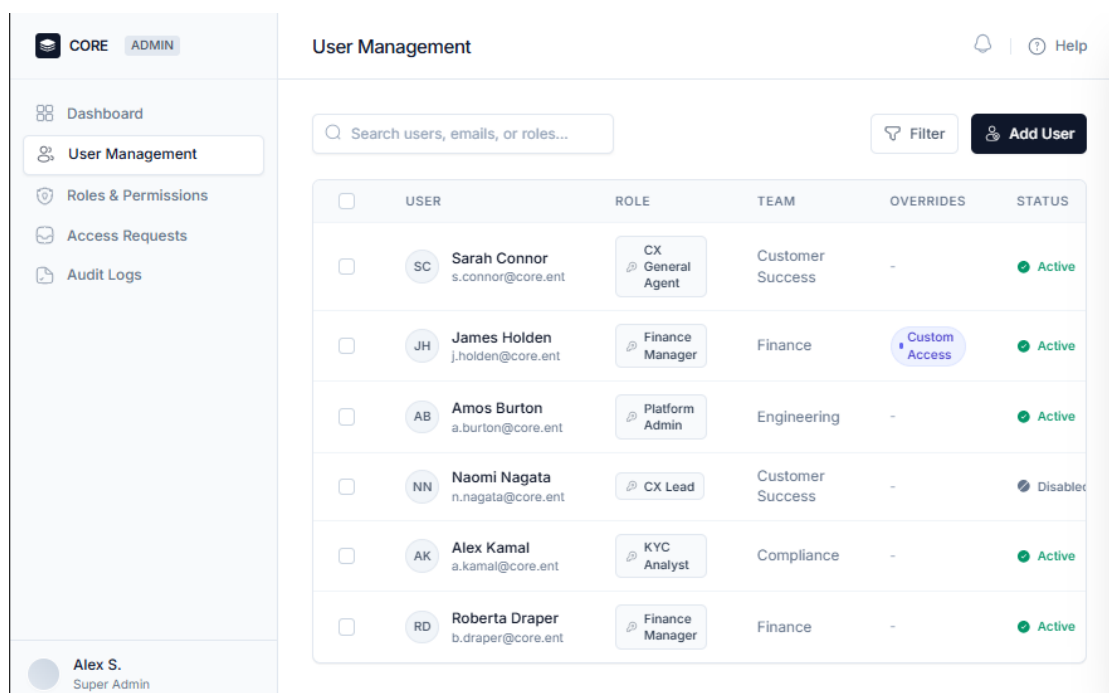


Figure 14 User management Research Reference

Other professional dashboard UI examples comprise:

A modern React and Tailwind based admin dashboard with a responsive sidebar layout and structured data tables with inline action controls, suitable to voter registration management. (shadcn, 2024)

An enterprise-level UI components library with paginated data table, search bar, color-coded system of status tags, and more, and is suitable to the task of managing and reviewing large voter datasets effectively. (Ant Design Team, 2024)

Important Factors, which affected the design.

- Based on these reference dashboards, the following UI features affected the design of the User Management page:
- Sidebar Navigation Structure - This assists the administrators to navigate between the modules of the system like the Dashboard, Users, Candidates, Elections, Constituencies, and Results without losing track.
- Summary KPI Cards - Four metric cards of Total Voters, pending review, approved today and verification rate enable the administrators to have a glance at the overall registration situation.
- Paginated Table of Data with Action Controls - The voter registration table with the columns Voter Name, Registration Date, Status, and Actions (Approve/Reject) follows the pattern of the data table that is commonly used in both AdminLTE and Ant Design Pro to manage the individual records with high efficiency.
- Status tag elements Status badges (such as Approved (green) and Pending (yellow/orange)) were borrowed as status tag elements widely used in modern libraries of administration UIs and made visuals clearer.
- Search Bar and Filter Controls — The presence of a top-aligned search bar and a Filter button is the best practice in usability when navigating and filtering large datasets.

3.3.2 Wire frame

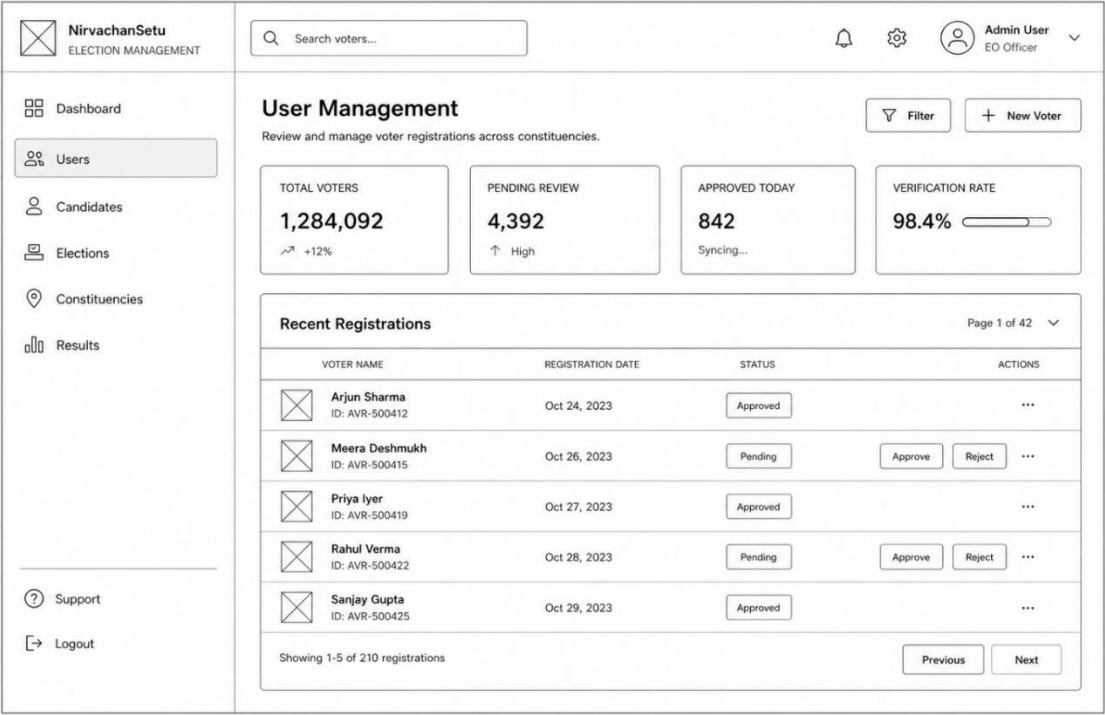


Figure 15user management Wire frame

3.3.3 Developed Design

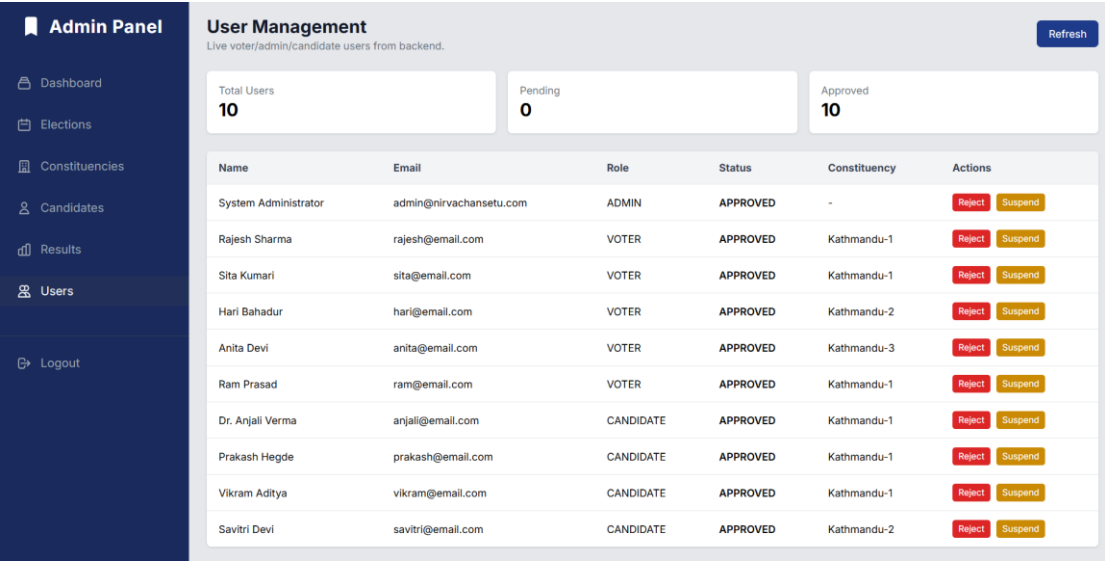


Figure 16 user management Developed Design

3.4 Candidate Management

3.4.1 Research Reference

Research Conducted

To inform the design of the Candidate Management Dashboard that you created (as shown in the screenshot) some research was conducted on recent admin dashboard UI templates, and candidate-management interfaces. These templates demonstrate data-driven, clean layouts that can be used to arrange user statistics, filters, action buttons, tables, and overview cards - all of which are reflected in the design of the Candidate Management page.

Reference Web Site / UI template.

The Aura Admin Dashboard Template is one of the handy sources of creating the admin dashboard and management panel. This is a modern dashboard interface offering cards of analytics, sidebar navigation, and organized tables, which can be customized to candidate workflows such as filtering, tracking statuses and managing actions. (Envato, 2024)

Professional KPI cards and tables with status indicator: handy in designing summary metrics and record listing such as: Pending, Approved and Rejected. (Ant Design Team, 2024)

Reference site

☐	USER	ROLE	TEAM	OVERRIDES	STATUS
☐	SC Sarah Connor s.connor@core.ent	CX General Agent	Customer Success	-	Active
☐	JH James Holden j.holden@core.ent	Finance Manager	Finance	Custom Access	Active
☐	AB Amos Burton a.burton@core.ent	Platform Admin	Engineering	-	Active
☐	NN Naomi Nagata n.nagata@core.ent	CX Lead	Customer Success	-	Disabled
☐	AK Alex Kamal a.kamal@core.ent	KYC Analyst	Compliance	-	Active
☐	RD Roberta Draper b.draper@core.ent	Finance Manager	Finance	-	Active

Figure 17 Candidate Management Research Reference

Major Features that impacted the design.

Based on these reference dashboards, the following UI elements came into play during the design of your candidate management page:

- Sidebar Navigation Structure - Assists users to alternate between modules such as Dashboard, Users, and Candidates easily.
- Summary KPI Cards - These cards are used to display the totals (e.g. Total Applications, Approved, Rejected).
- Data Table Layouts With Action Controls Tables with action icons or buttons (View, Approve, Reject) enable individual record management to be efficient.
- Filter / Search Options- Contemporary dashboards have filtering features to narrow down on large datasets. This will improve usability of candidate review tasks.
- Clean & Responsive Card UI - Statistical information has uniform card styles that enhance the readability and visual hierarchy in the cards.

3.4.2 Wire frame

NirvachanSetu
ELECTION MANAGEMENT

Search applications...

Admin Panel System Overseer

Candidate Management
Review and manage official application filings for the 2024 General Elections.

Filters + New Nomination

TOTAL APPLICATIONS
1,284
12% from last week

PENDING REVIEW
42
Requires immediate action

APPROVED
1,192
Verified & Finalized

REJECTED
50
Documentation issues

Recent Applications
Sort by: Latest Submission

CANDIDATE DETAILS	CONSTITUENCY	APPLIED DATE	STATUS	ACTIONS
Dr. Aruna Sharma Independent • 15 years exp.	North Delhi Central ZONE-04	Oct 12, 2023	Pending Review	Approve
Vikram Malhotra People's Front Party	South Mumbai ZONE-01	Oct 10, 2023	Approved	View Details
Sanya Verma Social Justice Alliance	Bangalore East ZONE-09	Oct 09, 2023	Rejected	View Reason
Rajesh Gokhale National Heritage Party	Pune West ZONE-02	Oct 08, 2023	Pending Review	Approve

Showing 1-10 of 1,284 candidates

© 2024 ELECTION COMMISSION MANAGEMENT PORTAL • NIRVACHANSETU

Figure 18 candidate Wire frame

3.4.3 Developed Design

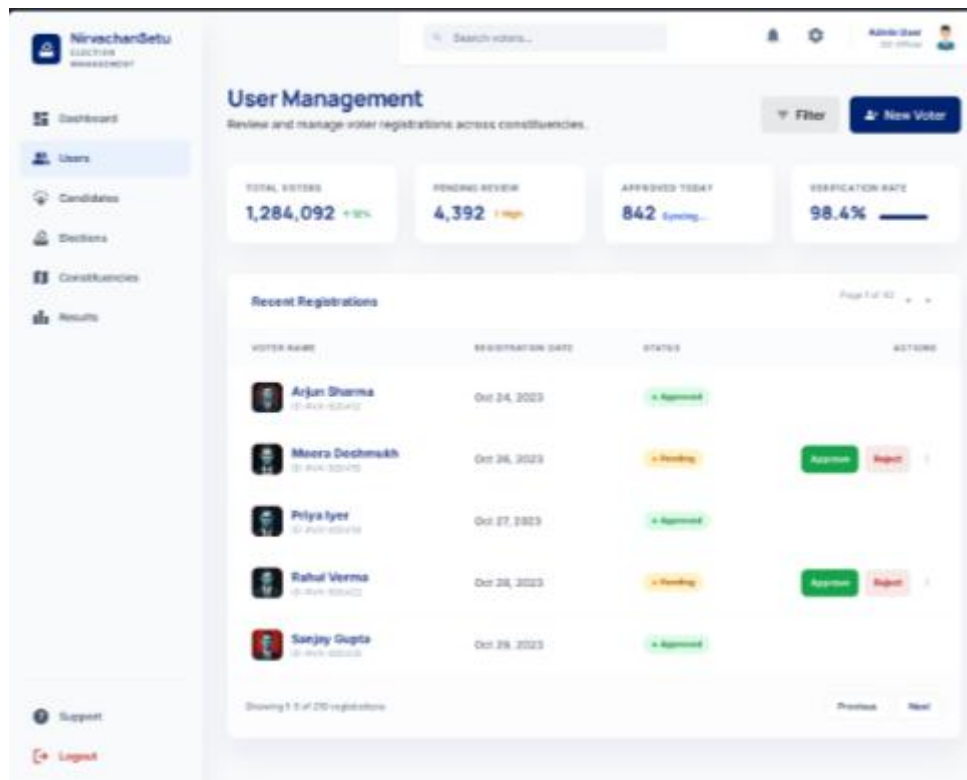
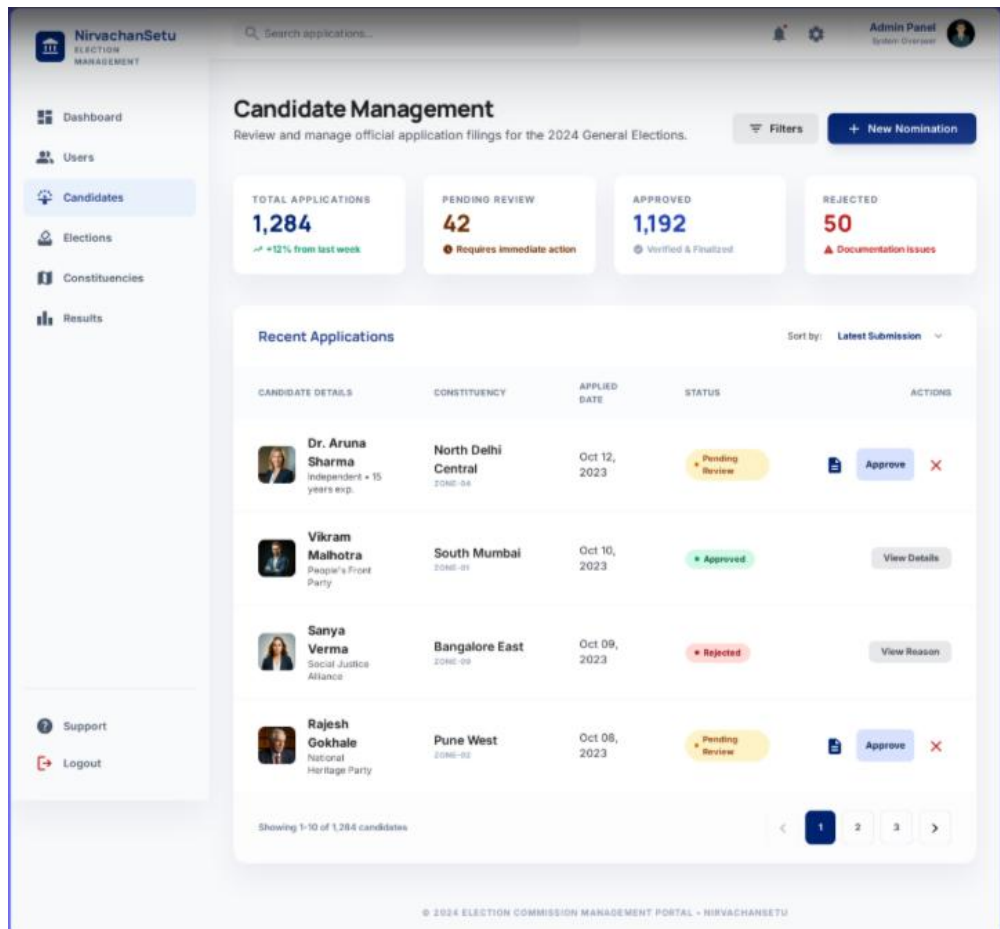


Figure 19 candidate Developed Design

*Figure 20 candidate Developed Design*

3.5 . Elections Gallery

3.5.1 Research Reference

In order to design the Elections Gallery page of the NirvachanSetu system, the modern task and schedule management interfaces have been researched, presenting a structured list of items with status tracking, filtering, priority levels and pagination. The aim was to discover a clean and data-centric pattern of UI that could be customized to show and manage various electoral cycles and their related phases, dates, and statuses.

Reference Website / UI Template.

The main source to do so is the Shadcn UI - Tasks Example, a modern and open-source React and Tailwind-based interface that illustrates a highly structured table layout to manage items with type labels, status indicators, priority tags, and per-row action menus. Its very simple but practical design style had a direct impact on the Elections Gallery design. (shadcn, 2024)

Other supporting sources are:

- Shadcn UI Dashboard — This is a provided reference on the top-level summary statistics panel (Active Elections, Scheduled, Completed YTD) and the overall sidebar navigation structure that is used throughout the NirvachanSetu system. (shadcn, 2024)
- Ant Design Pro List Table - An enterprise UI reference of paginated data tables with structured metadata columns like title, date range, status, and action controls. (Ant Design Team, 2024)

Reference Page

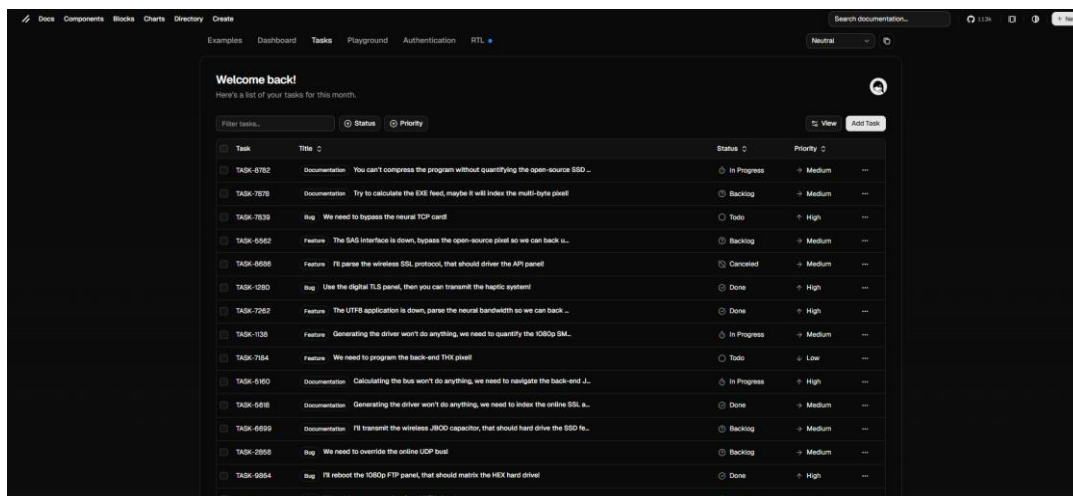


Figure 21 Elections Gallery Research Reference

Based on the Shadcn UI Tasks page and the supporting references the following UI features influenced the Elections Gallery page:

- **Structured Data Table Layout** Structured table The list of tasks in the Shadcn Tasks table includes an ID, type label, title, status, and priority column. This is the inspiration of the Elections Schedule table in NirvachanSetu which also presents each election entry by Title, ID, Start Date, End Date, Status, and Actions.
- **Status Indicator Chips** The Tasks interface has clear status labels like: In Progress, Backlog, Done and Canceled per row. This trend was followed in the Elections Gallery in the form of color-coded status chips of the lifecycle stage of each election, making the status of each election instantly visible.
- **Type/Category Labels** This is done by assigning category tags to each item on the Tasks page like: Bug, Feature, Documentation etc. This affected the type icons used to differentiate between the entries of General Assembly Elections, State Legislature, Municipal Corporation and By-Elections.
- **Per-Row Action Menu** The action menu of a three-dot context-sensitive operation appears in each row of the Shadcn Tasks table. This was ported into the Elections Gallery as a per-row action control that allows administrators to view or edit or otherwise manage individual election records.
- **Filter and View Controls** - The Tasks interface has filter and view toolbar-level options above the table. This had a direct impact on the Filter and Export controls positioned over the table of Election Schedules, in the NirvachanSetu design.
- **Pagination Controls** :The row-level pagination system (Page 1 of 4, rows per page select, navigation arrow) featured on the Shadcn Tasks page was borrowed to support the display of up to 24 election entries on more than one page.

Other Improvement in our System.

The Elections Gallery of NirvachanSetu was built with the purpose of managing an electoral cycle. It includes an Ongoing Phase Banner that highlights the currently running election with live voter and polling booth statistics, a Quick Stats panel with at-a-glance electoral health statistics, and a Feature Highlights section featuring Smart Scheduling, Integrity Checks, and Resource Archive capabilities, far exceeding those provided by a typical task management UI.

3.5.2 Wire frame

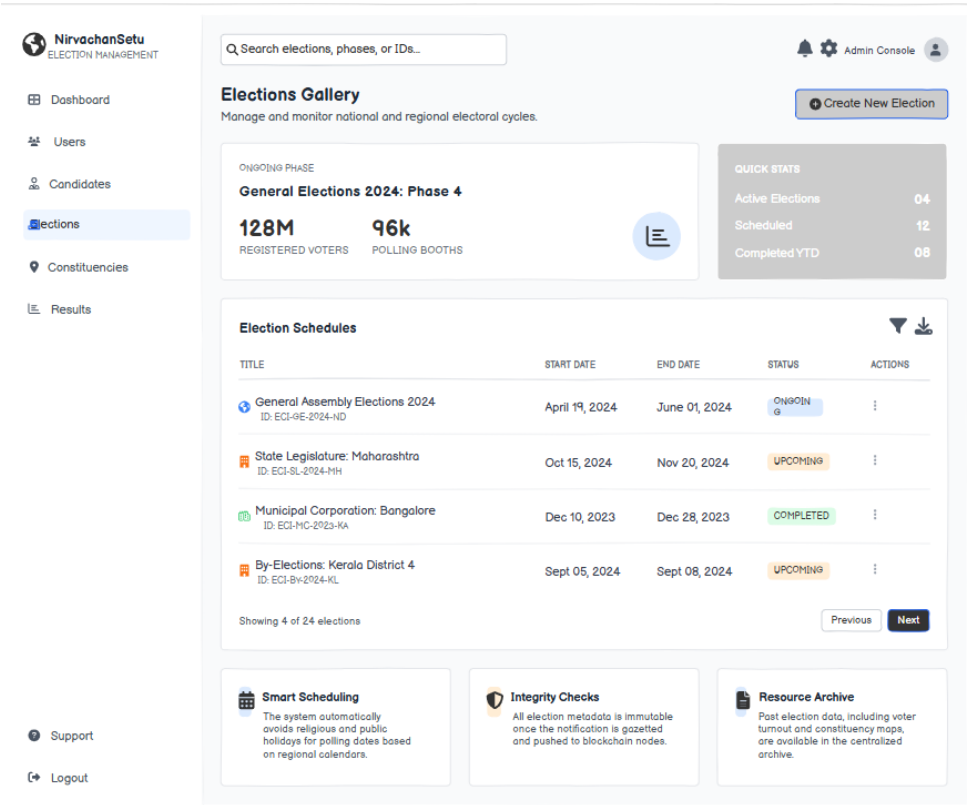


Figure 22 Wire frame

3.5.3 Developed Design

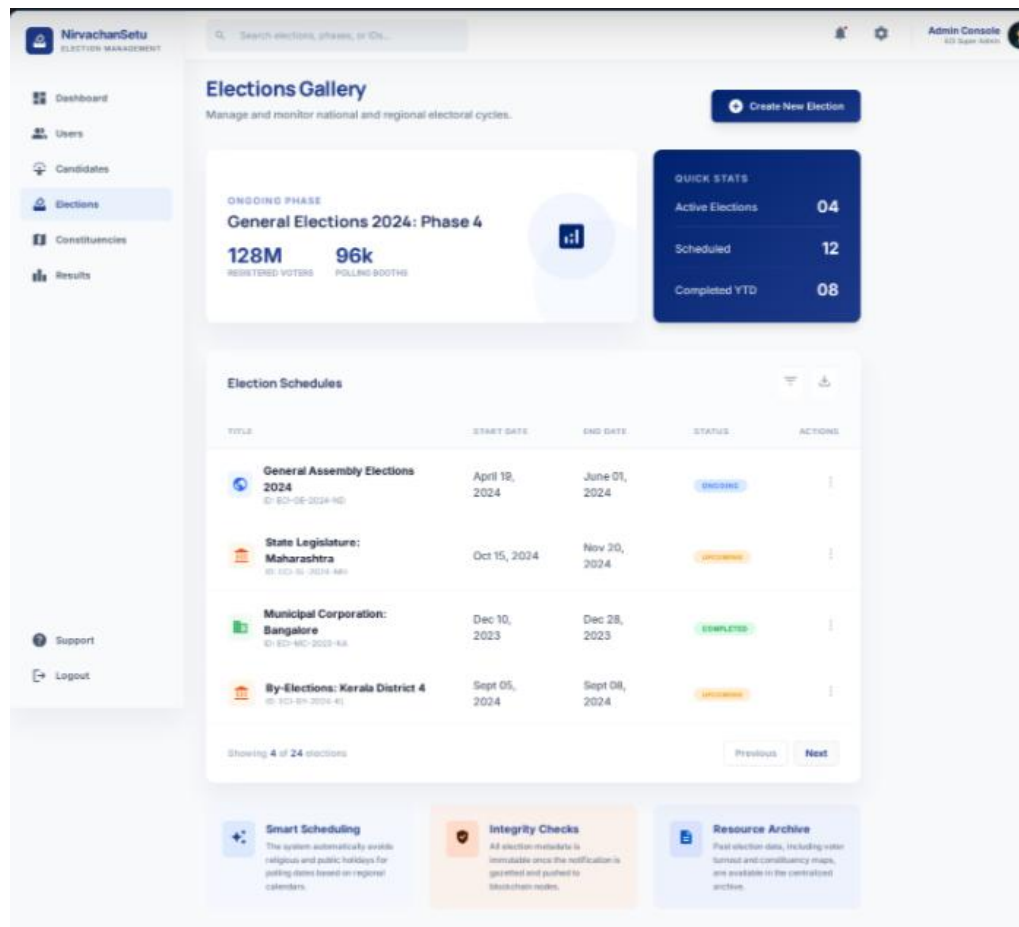


Figure 23 Developed Design

3.6 Results Analytics

3.6.1 Research Reference

In order to design the Results Analytics page of the NirvachanSetu system, research was done on modern data visualization libraries and chart-based dashboard interfaces that display real-time metrics, comparative statistics and ranked data in a clean and readable format. Because this page was tasked with presenting live vote counts, candidate standings, turnout rates, and geographic participation data, the emphasis was put on the need to find chart components that are visually intelligible and functional in their nature.

The main source here was the Shadcn UI - Charts Collection (Area Charts) a collection of ready-to-use chart components created on top of Recharts, an open-source React charting library. The collection features a range of area, bar, line, pie, radar, and radial chart layouts - all designed in the Shadcn design system. These elements had a direct impact on the charting method and visual language employed in the Results Analytics page. (shadcn, 2024)

The other supportive sources are:

- Shadcn UI - Bar Charts- This offers direct reference to the bar chart of "Total Vote Counts per Election" which is a comparison of the voter turnout across a series of multiple election cycles (National Assembly, State Council, Local Gov, Senate, Provincial) in a grouped bar format.
- Recharts Documentation — The charting library underlying Shadcn UI, the volume/percentage toggle behavior and the multi-category axis configuration used in the vote count visualization. (Recharts Group, 2024)

Reference page

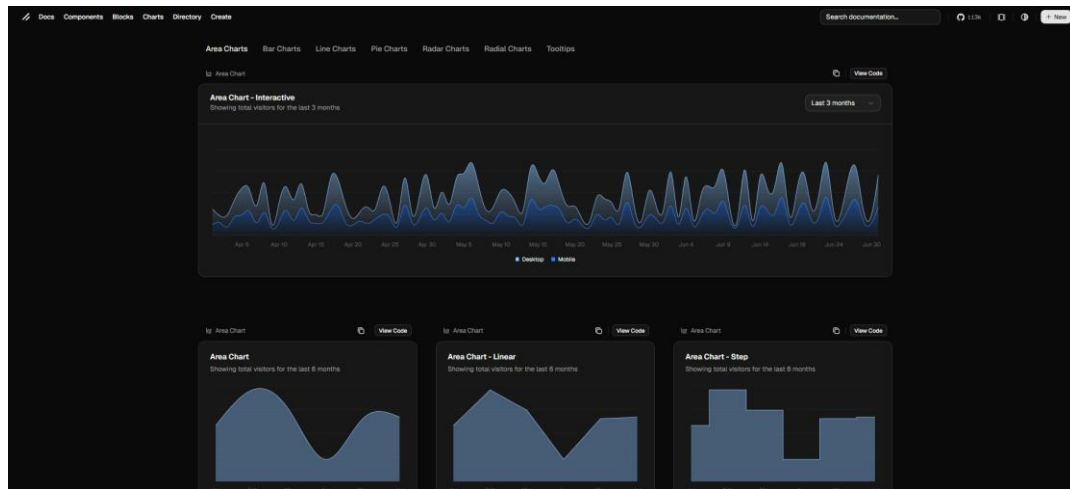


Figure 24 Results Analytic Research Reference

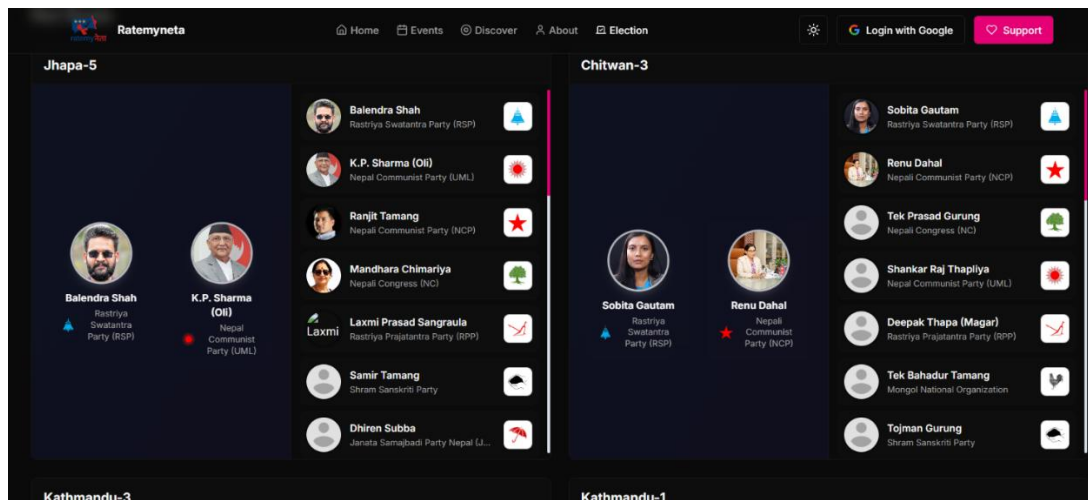


Figure 25 Results Analytics Research Reference

Based on the collection of Shadcn UI Charts and supporting sources, the next UI characteristics influenced the Results Analytics page:

- **Area and Bar Chart Components** - The Shadcn Charts collection is shown with clean and minimal chart layouts labeling axes, tooltips and legend indicators. This had a direct influence on the chart of the "Turnout Data on five types of elections on a similar bar chart format.
- **Volume / Percentage Toggle** - A number of chart variants in the Shadcn collection can be configured to switch data views (e.g., stacked vs. percentage view). This was the same pattern that was followed on the Results Analytics page where an administrator can toggle between absolute numbers and proportional comparisons.
- **Regular Chart Wrapping as a Card** with Title, Subtitle and Footer metadata - Each chart in the Shadcn collection is wrapped as a clean card with a title,

subtitle and footer metadata. This pattern of wrapping the cards was used on all the analytical panels in the Results Analytics page, such as the vote count chart, candidate ranking panel, and geographic heatmap section.

3.6.2 Wire frame

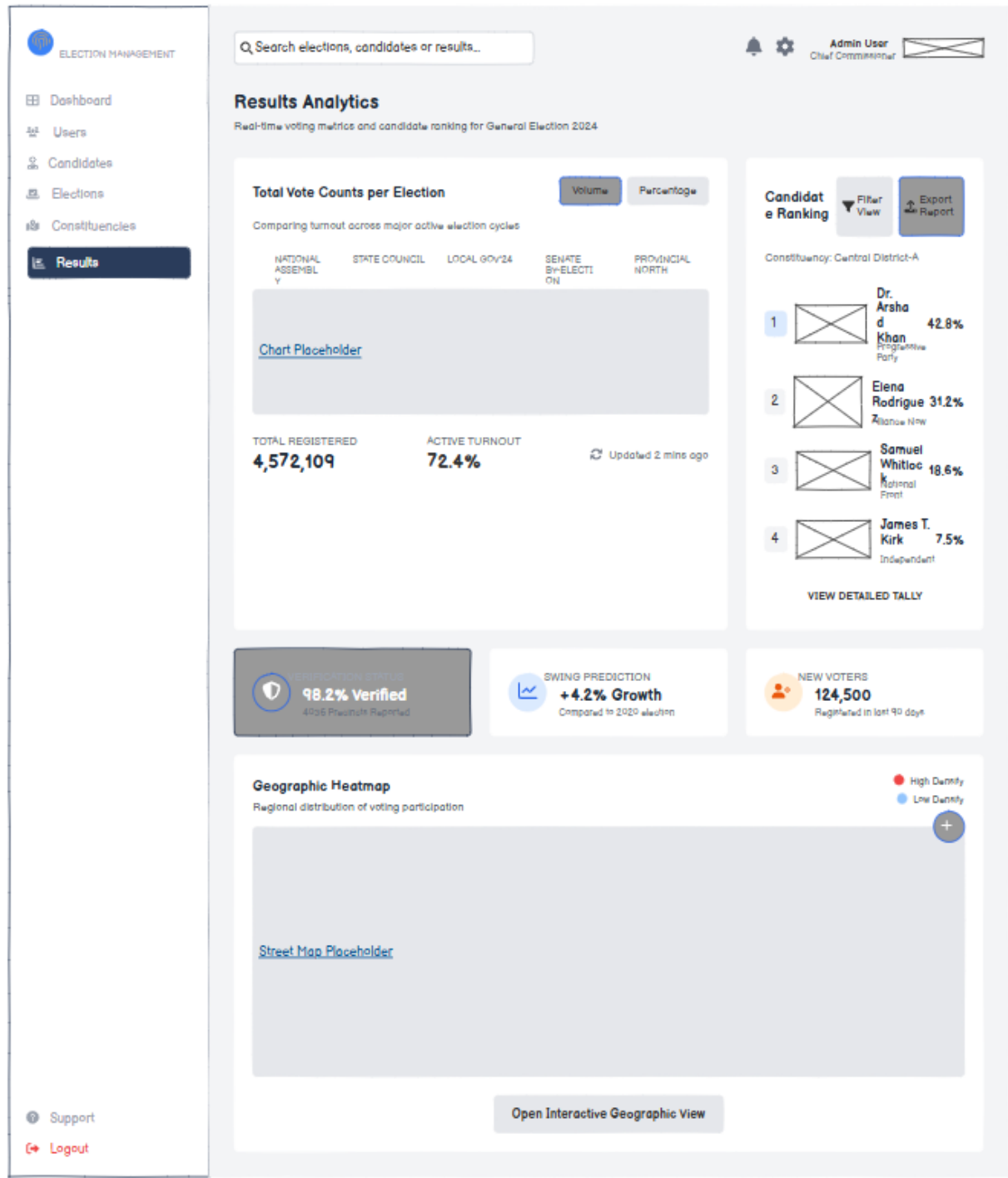


Figure 26 Wire frame

3.6.3 Developed Design

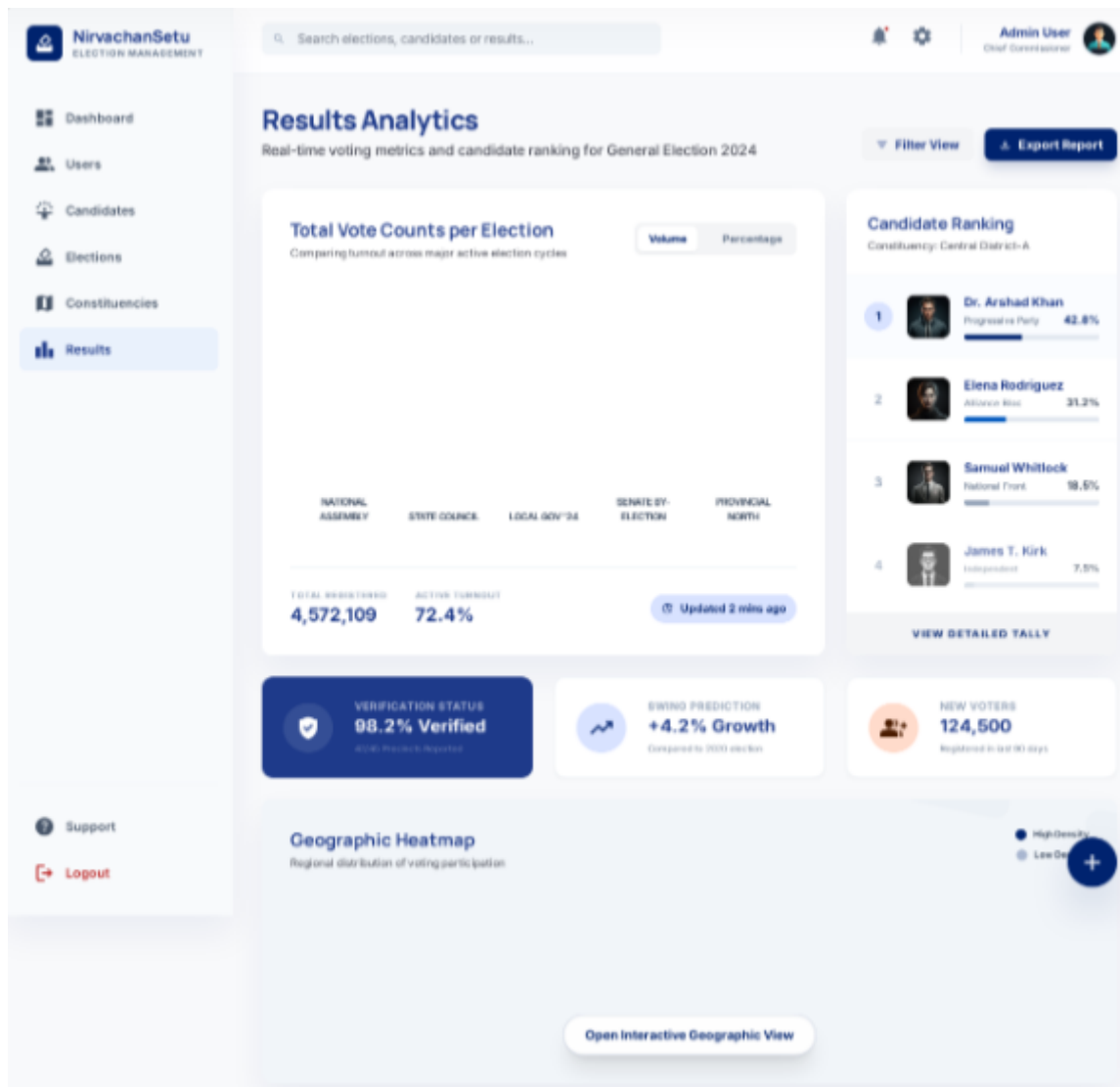


Figure 27 Developed Design

3.7 Voter Dashboard

The major interface whereby the registered voters interact with the NirvachanSetu system is the Voter Dashboard. It is structured in such a way that all the information and actions, which are relevant to voters, are centralized and put into one, sensible perspective. In order to inform this design, research was done on contemporary personal dashboard controls that display user specific data, task queue and status updates in an understandable and comprehensible format.

References Websites / UI Templates.

As the main source of the Voter Dashboard, the Shadcn UI Dashboard Example, a React-based, Tailwind-based administrative interface, proved to be helpful in showing how to combine summary metrics cards, activity feeds, and side bars into a single cohesive layout. The card hierarchy and lack of visual language directly affected the design of the Voter Dashboard. (shadcn, 2024)

Other supporting sources are:

Dribbble Voter Portal UI Concepts: A variety of community-submitted UI explorations of civic portals and election dashboards that demonstrate how they display the registration status of a voter, the election schedule and active ballot information in a user-friendly card format. (Dribbble Holdings Ltd, 2024)

Visual treatment of the status chips and action buttons in the dashboard was informed by the Material Design Guidelines (Google) which describes the card-based component system, elevation shadows and color-coded status indicators. (Google LLC, 2024)

Gov.uk Design System: This public-sector design system was a guide to the accessibility standards, plain-language labelling, and layout conventions that should be used in government-facing voter interfaces. (Government Digital Service, 2024)

Researched UI Screenshot

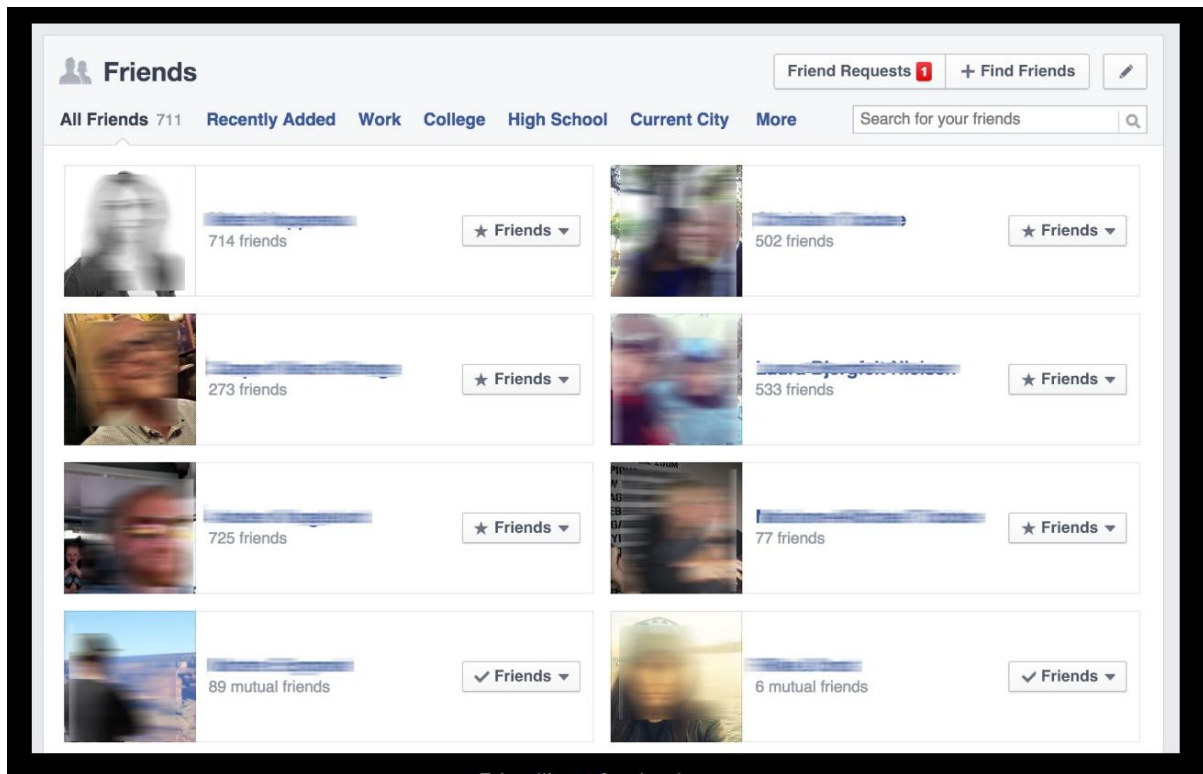


Figure 28 Voter Dashboard

Significant Features That are adopted as points of reference.

- Individualized welcome header that shows the name of the voter and his/her registration constituency and creates instant context to the user.
- Status card indicating state of voter registration approval (Pending, Approved or Rejected) with color-coded badges to allow at-a-glance understanding.
- Election schedule panel which is a list of future and active elections relevant to the constituency of the voter.
- Quick-action buttons to navigate to the ballot casting, checking results, and managing the profile.
- Notification area surfacing key notifications like changes in election date or document verification requests.
- Adaptable and approachable design that meets the requirements of the public-sector in terms of usability and is accessible to a variety of devices.

3.7.2 Wire Frame

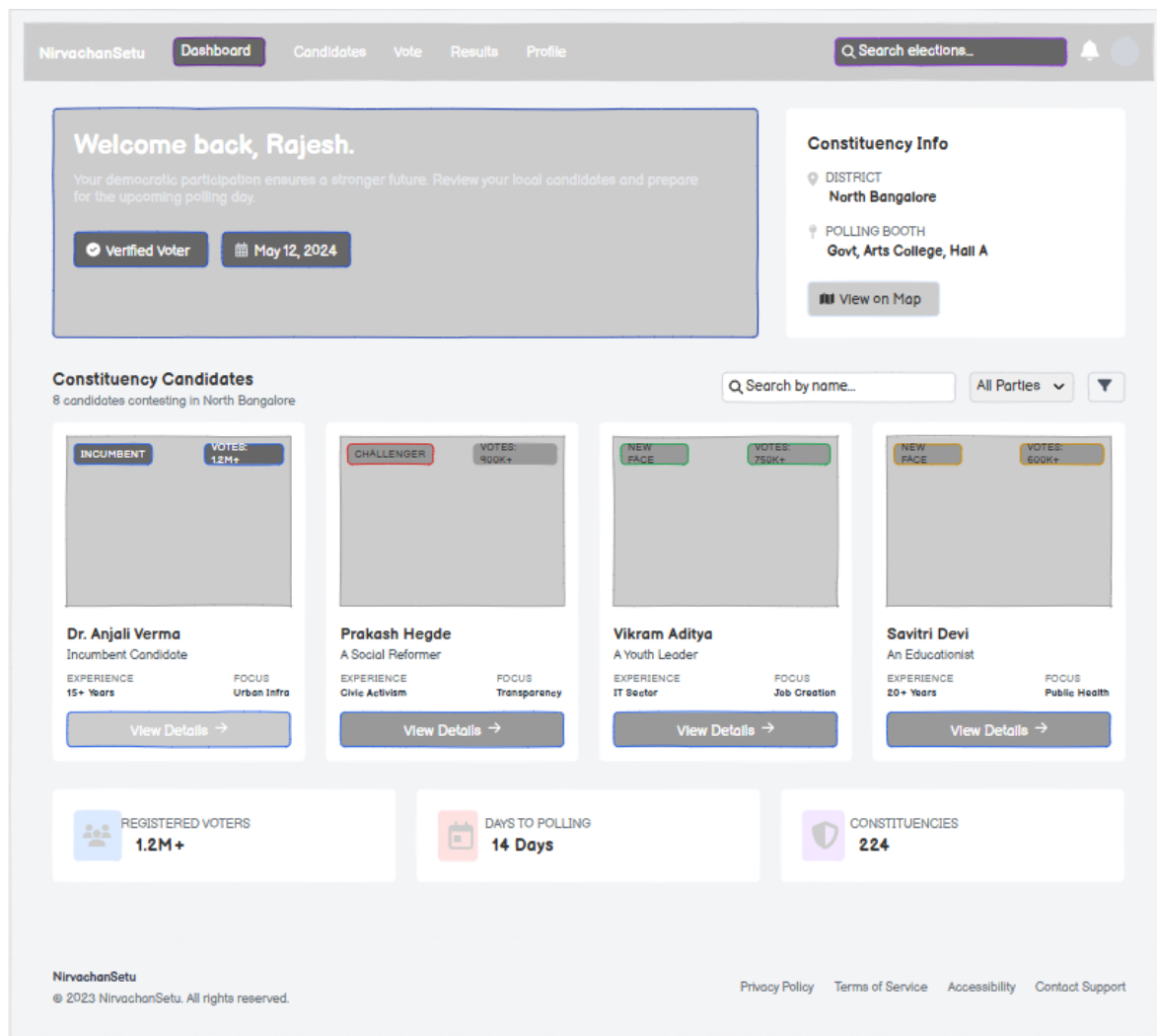


Figure 29 Wire Frame

3.7.1 Developed Design

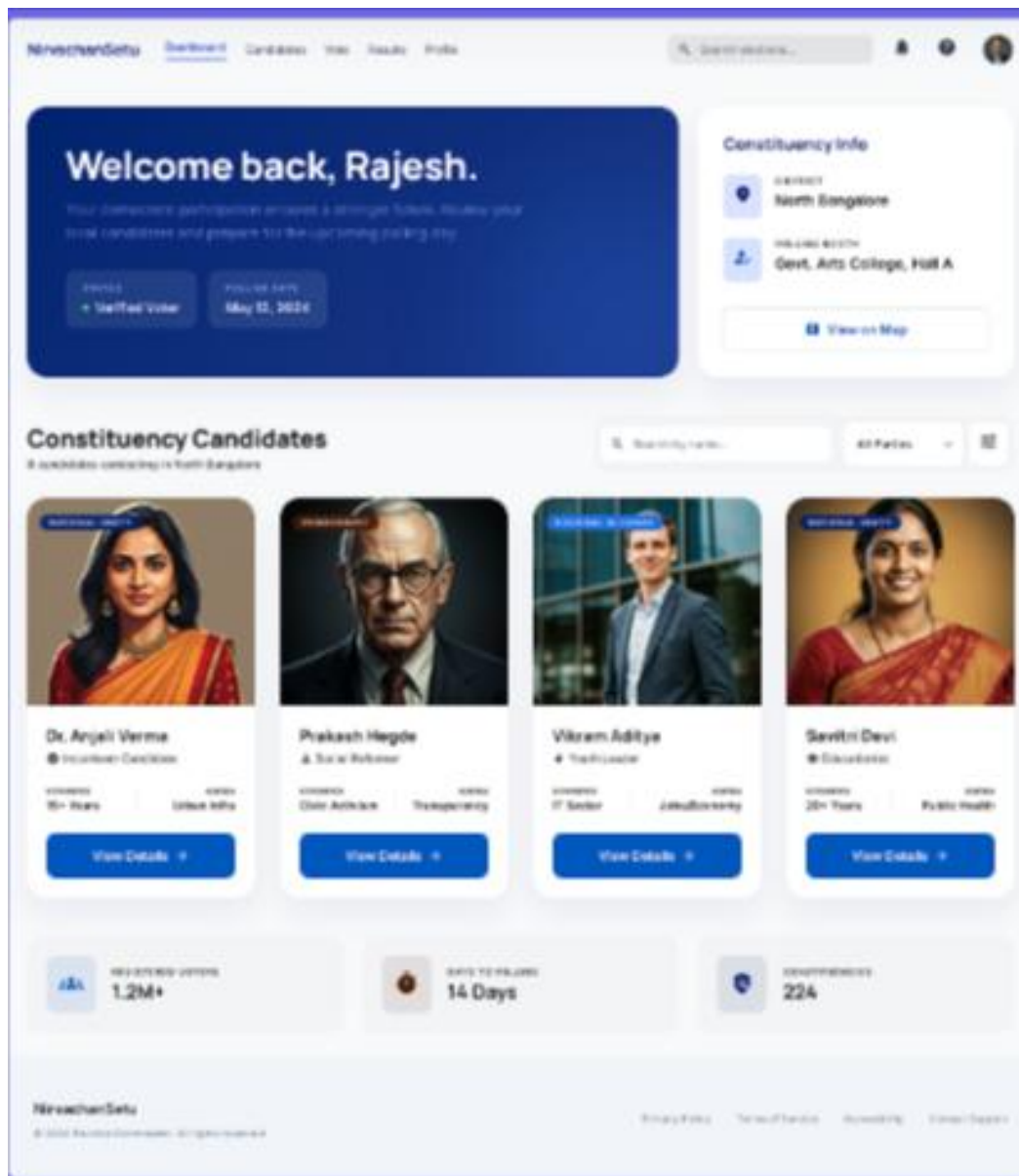


Figure 30 Developed Design

3.8 Electoral Candidate

3.8.1 Research Reference

The Electoral Candidate page is a public-facing listing interface, which displays all the registered and approved candidates standing in a given election. Through this page, voters have the freedom to access the profiles of the candidates, their affiliations and constituencies, and make informed decisions before voting. A study on the modern directory and listing interfaces was carried out where structured profiles are presented, which can be filtered and searched.

References Websites / UI Templates

The main source used in this page was the Shadcn UI - Data Table Component which exemplifies a clean, filterable and paginated table layout that is capable of being used to present structured records. The presence of a column-based information hierarchy, inline search bar and status filtering had a direct effect on the layout of the candidate listing. (shadcn, 2024)

Other supporting sources are:

- LinkedIn People Search Results Page :This interface served as a reference to display individual profile cards including name, role and affiliation in a scannable grid or list format with succinct call-to-action links. (LinkedIn Corporation, 2024)
- List and Card Page Provided an enterprise-grade UI pattern to list items having thumbnail images, metadata, and action buttons, suitable to a candidate directory context. (Ant Design Team, 2024)
- Election Commission of India Candidate Affidavit Portal A real-world civic reference used to display the information about the candidates and their views on certain issues in a structured tabular manner. (Election Commission of India, 2024)

Researched UI Screenshot

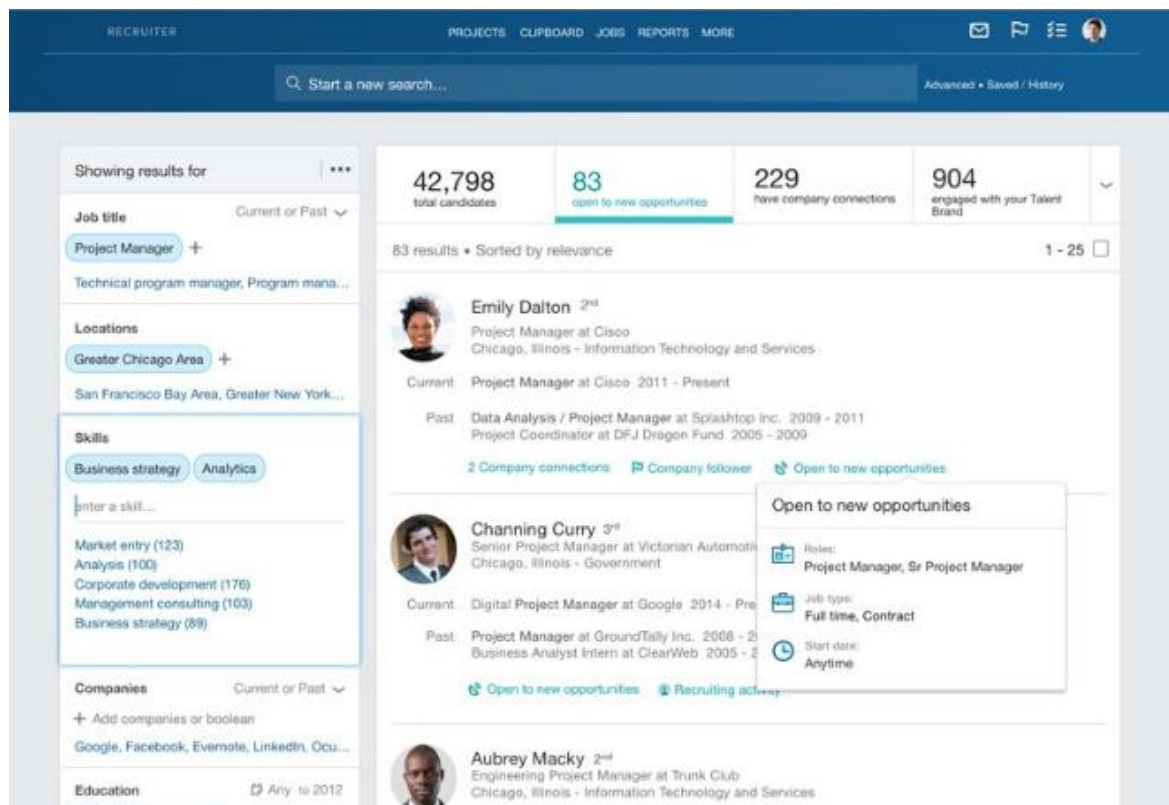


Figure 31 Electoral Candidate Research Reference

Marked Features that are used as points of reference.

- Candidate profile cards, with their name, party affiliation, constituency, and a short bio note, allowing the voter to easily find the relevant candidates.
- Constituency based filtering to reduce the list of candidates based on the area that the voter is registered in.
- Name or Party search option to filter down to specific candidates, making it easier to use in large election competitions.
- Status badge of approval to be able to quickly differentiate between verified and active members versus under review members.
- View Profile button per candidate which links to a detailed profile and manifesto page.
- Paginated design that can support large number of candidates without compromising on page performance.

3.8.2 Wire frame

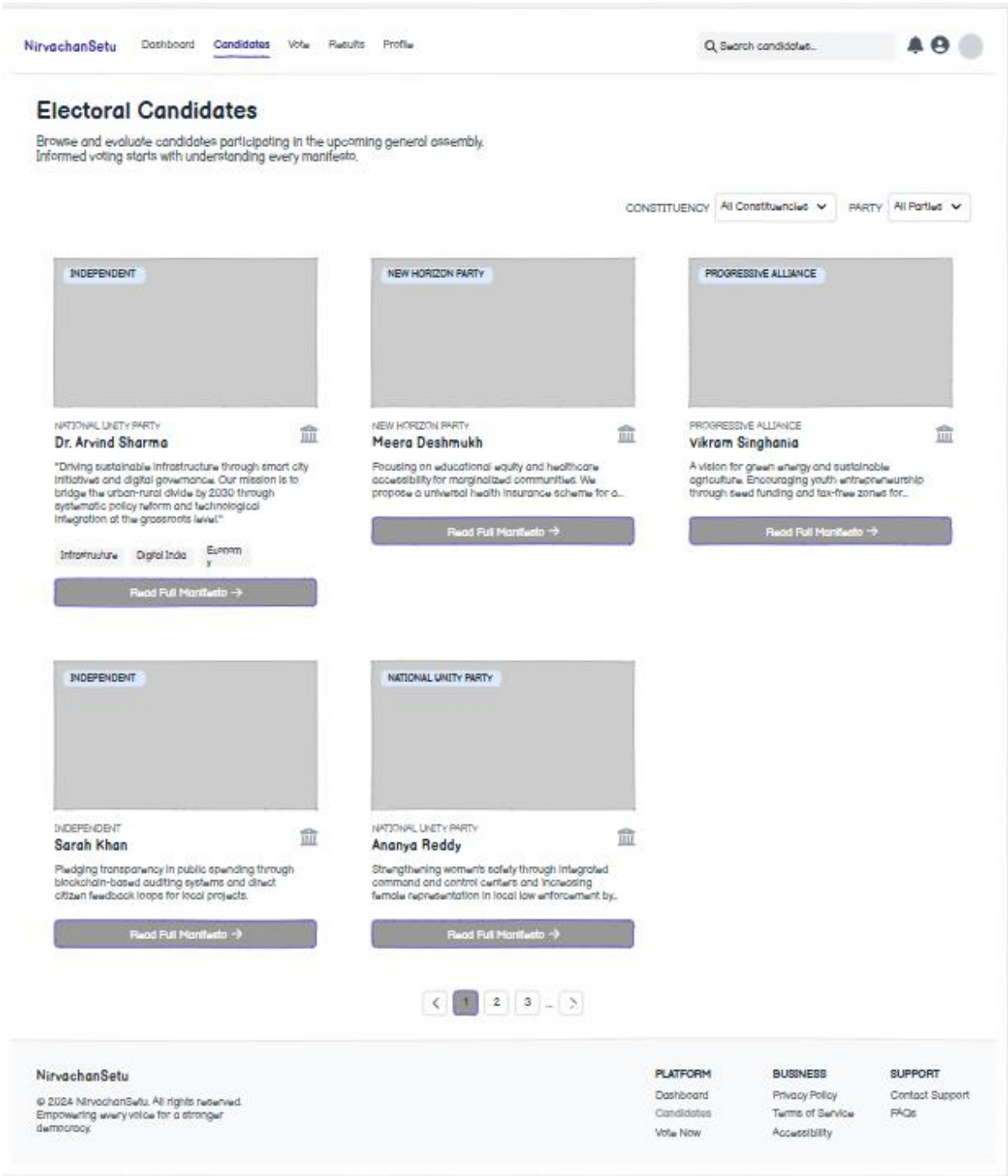


Figure 32 Wire frame

3.8.3 Developed Design

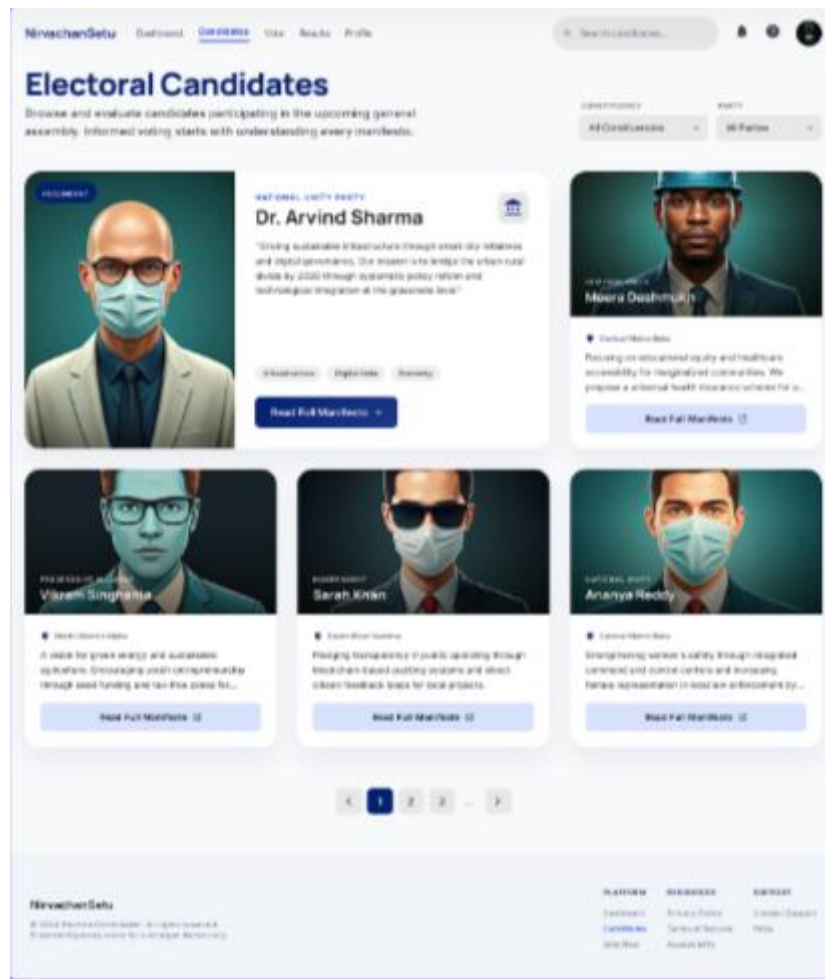


Figure 33 Developed Design

3.9 Cast Vote

3.9.1 Research Reference

The Cast Vote page is the most important working interface of the NirvachanSetu system. It allows qualified voters to cast their ballot safely and precisely in a current election. Research on ballot UI designs was focused on designs that focuses on clarity, error prevention, accessibility, and confirmation safeguards.

References Websites / UI Templates.

The main source was the MIT TrueVote Research Project that tested the usability of an electronic ballot interface and highlights the importance of presenting selections sequentially, in stages, and giving visual confirmation of selections and providing explicit submission safeguards to reduce voter error. Its results had a direct influence on the design strategy in the Cast Vote interface. (Massachusetts Institute of Technology, 2024)

Other sources that support the work are:

- Digital Ballot Interface - This civic technology reference served as guidance on how to present the options of the candidate in a clear and single column interface with radio button selections and explicit confirmation modals before final submission. (Overseas Vote Foundation, 2024)
- The visit to gov.uk Design System Form Components The form pattern guidelines, especially those that concern radio buttons and step-by-step journeys were consulted to make sure that the vote casting flow is clear and instructible. (Government Digital Service, 2024)
- References: Sometimes, usability research on confirmation dialogues, undo mechanisms, and warning messages informed the addition of a pre-submission review screen and warning about irreversibility.

Researched UI Screenshot

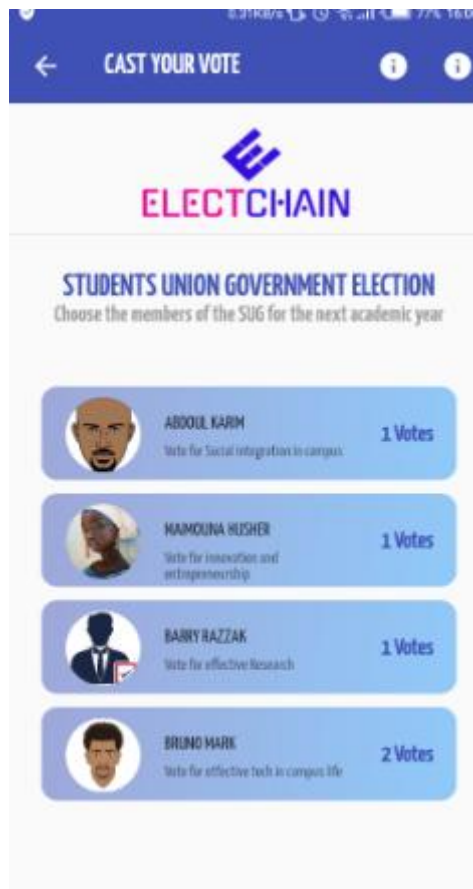


Figure 34 Cast Vote Research Reference

Distinctive Features that are captured as the points of reference.

- Sequential ballot flow that guides the voter through the selection of the candidate to a final submission by reviewing the ballot, thereby limiting the chances of making an inadvertent or uninformed vote.
- Selection of each candidate as a radio button with name, party and constituency displayed to ensure that a well informed choice is made.
- Pre-submission review screen that shows the selection that the voter has made to be verified before the vote is cast.
- Explicit confirmation modal, which has a warning about irreversibility, which informs that a vote that has been submitted cannot be changed.
- The submit button goes into the disabled state until a choice has been made, this will prevent incomplete submissions.

- Screen of confirmation of success with a specially designed submission reference number of the.

3.9.2 Wire frame

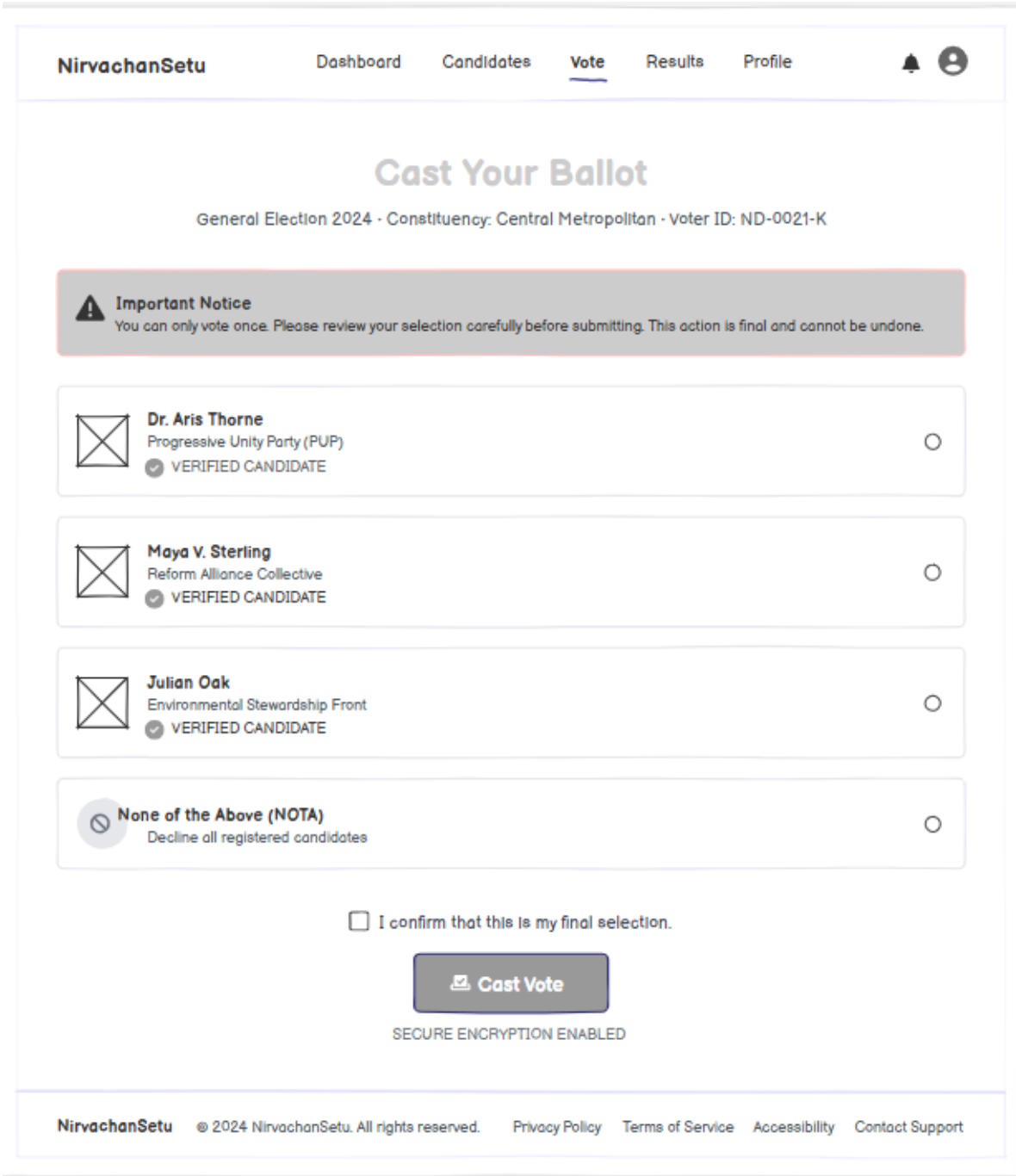


Figure 35 Wire frame

3.9.3 Developed Design

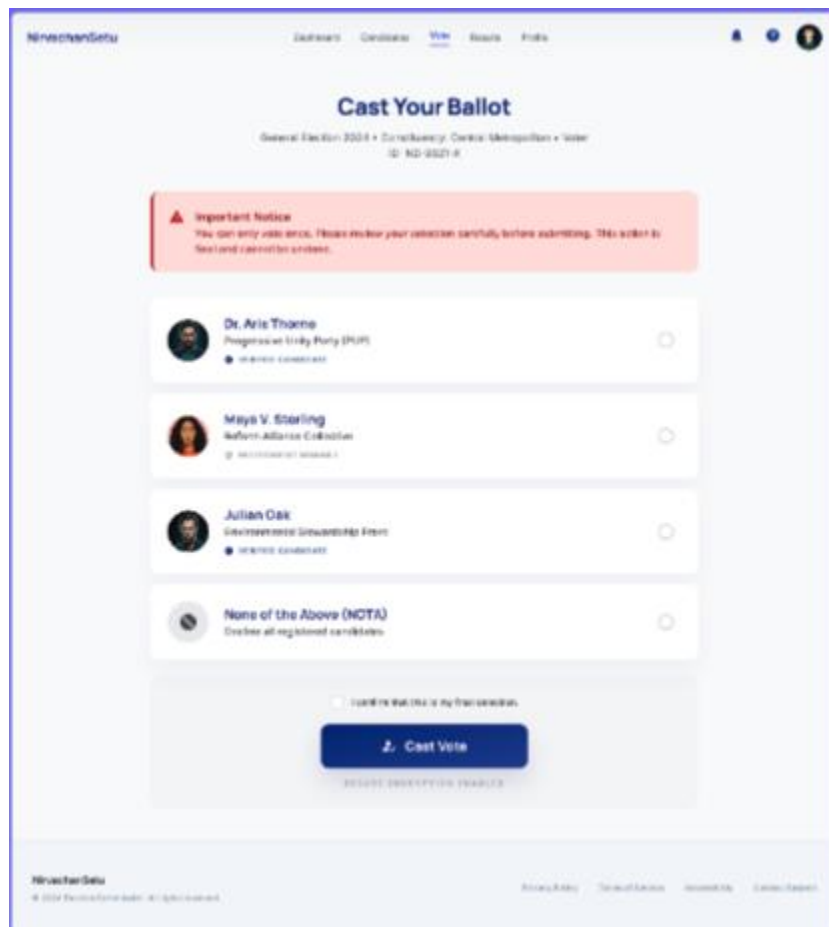


Figure 36 Developed Design

3.10 Result

Result page is the page where the voters get to see the post-election results in a readable and transparent manner. It also shows the number of votes cast by the candidates, the victorious party or individual and the district-by-district results after an election is over. The study was carried out on interfaces of public-facing election results that are densely populated with data and are visually clear.

Reference Websites / UI Templates.

The main source was the BBC UK General Election Results Page which is generally accepted as the model of clear and accessible public reporting of elections. Its use of candidate rankings, percentage vote, bar-chart comparisons and drill-down navigation of the constituency directly influenced the layout of the NirvachanSetu Result page. (British Broadcasting Corporation, 2024)

Other supporting sources are:

- Source: 2012 Election Results in Nepal by the Election Commission of Nepal Official Results Portal: A civic source providing tabular results of the 2012 election in Nepal by the Election Commission of Nepal. (Election Commission of Nepal, 2024)
- Shaden UI -Bar Charts and Data Tables -Provided chart component reference to visualise vote share comparison across candidates using bar and pie charts. (shaden, 2024)
- The Guardian - Election Results Visualisation- Provided information on colour-coded party result cards, percentage-based progress bars, and seat tally counters used in reporting multiple-constituency elections. (Guardian News and Media Ltd, 2024)

Researched UI Screenshot

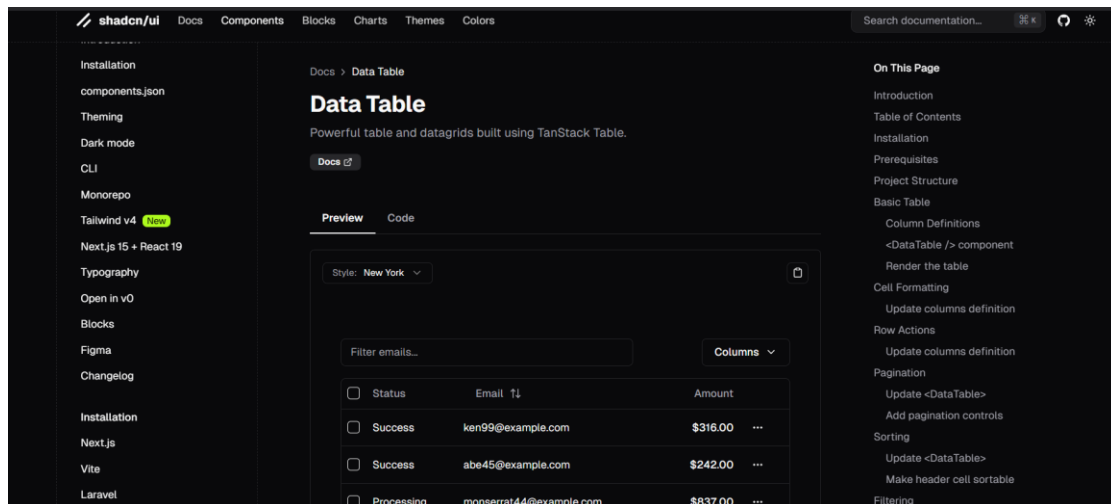


Figure 37 Result

Significant Elements Stolen as a reference.

- Candidate result cards, which give an overview of the performance of each candidate in a concise but clear manner.
- Bar chart of the percentage of the votes of all the candidates in a single election to provide quick visual understanding of the election results.
- Declared winner highlight use a highly coloured banner or icon to focus attention on the winning candidate.
- Constituency filter in order to enable voters to see the results in their locality within a wider election.
- Statistics on voter turnout have been provided in conjunction with the candidate results so as to give the participation rate.
- Timestamp of the time when the results were formally announced in support of transparency and auditability.

3.10.1 Wire frame

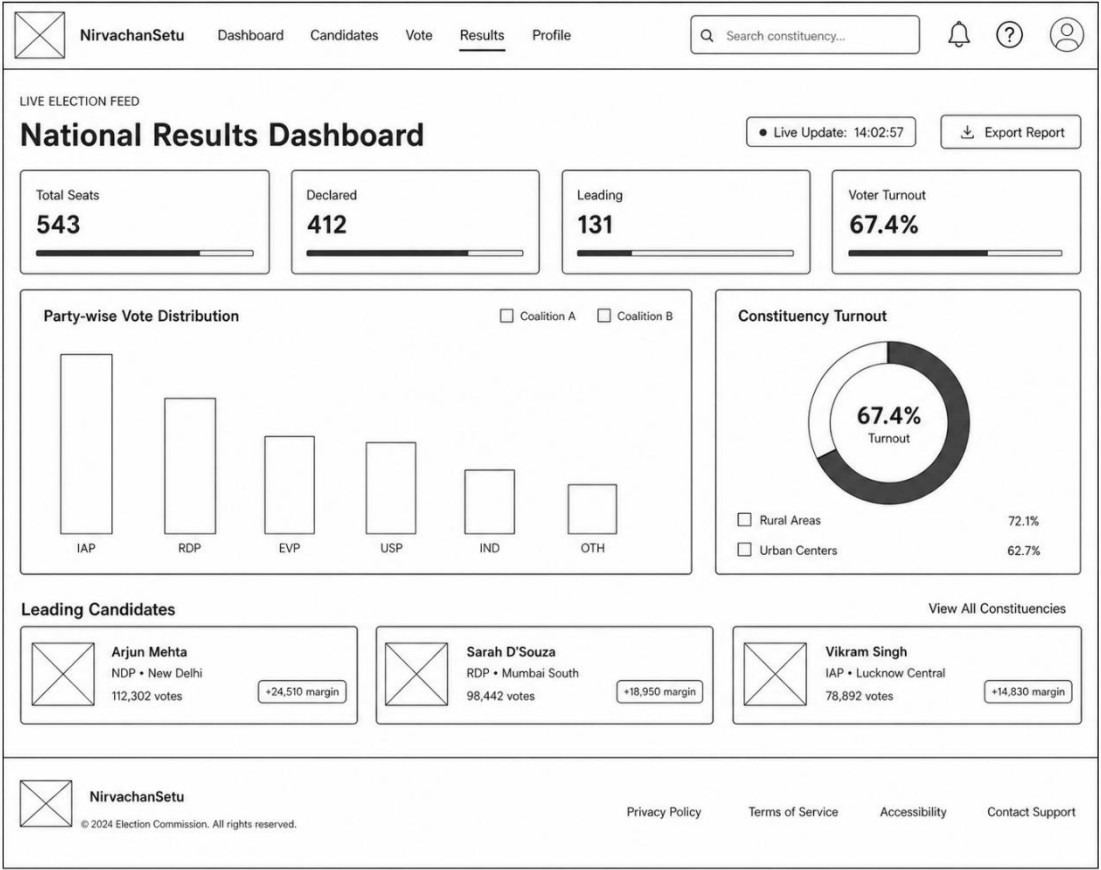


Figure 38 Wire frame

3.10.2 Developed Design

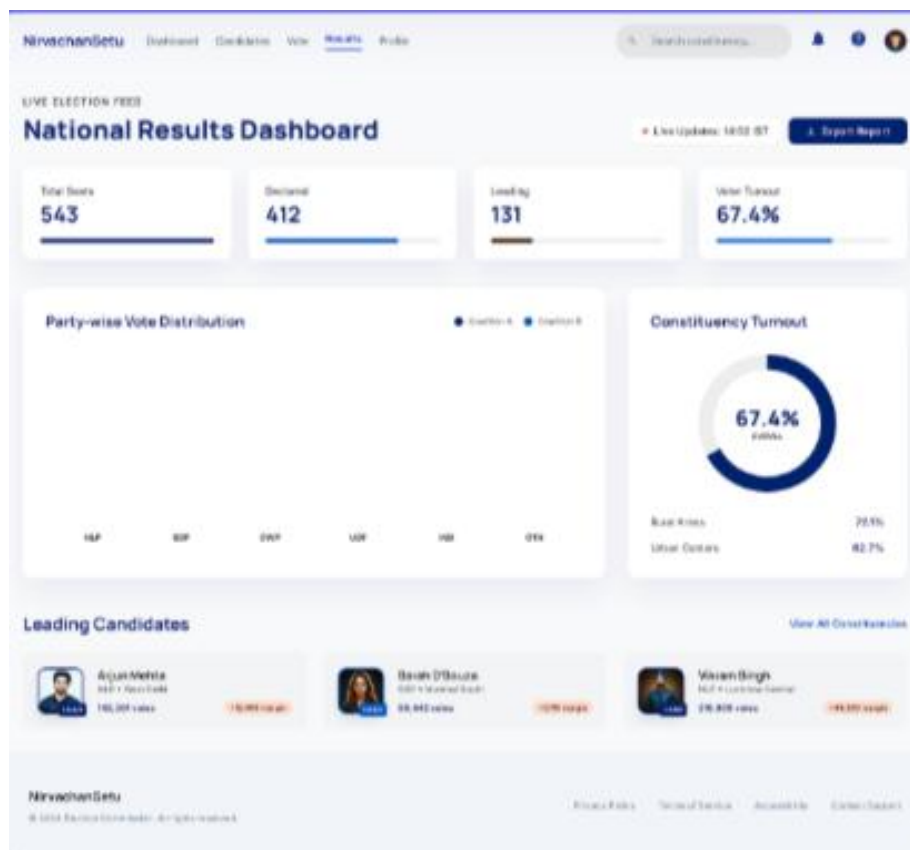


Figure 39 Developed Design

3.11 Profile

The Profile page allows registered voters to access and maintain their personal account information in the NirvachanSetu system. It shows various details including name, constituency registered in, citizenship verification status, and uploaded documents. Studies were targeted at user account pages of modern web apps that merge the display of a profile with document management and settings controls.

References Websites / UI Templates.

The main source was the GitHub User Settings and Profile Page which offers a formal personal information layout with well-defined sections of account data, security settings and linked documents. Its blend of read-only profile display with editable form fields directly impacted Profile page layout. (GitHub Inc, 2024)

Other supporting sources are:

- Shadcn UI (Settings Page) Sample Provided a sectioned card-based settings layout with labelled fields, edit controls and save actions which guided the structural organization of the voter profile interface. (shadcn, 2024)
- Dribbble — User Profile UI Concepts — Provides visual reference to neat profile card layouts with avatars, name, registration metadata, and status badges that would be appropriate in a government digital service. (Dribbble Holdings Ltd, 2024)

Researched UI Screenshot

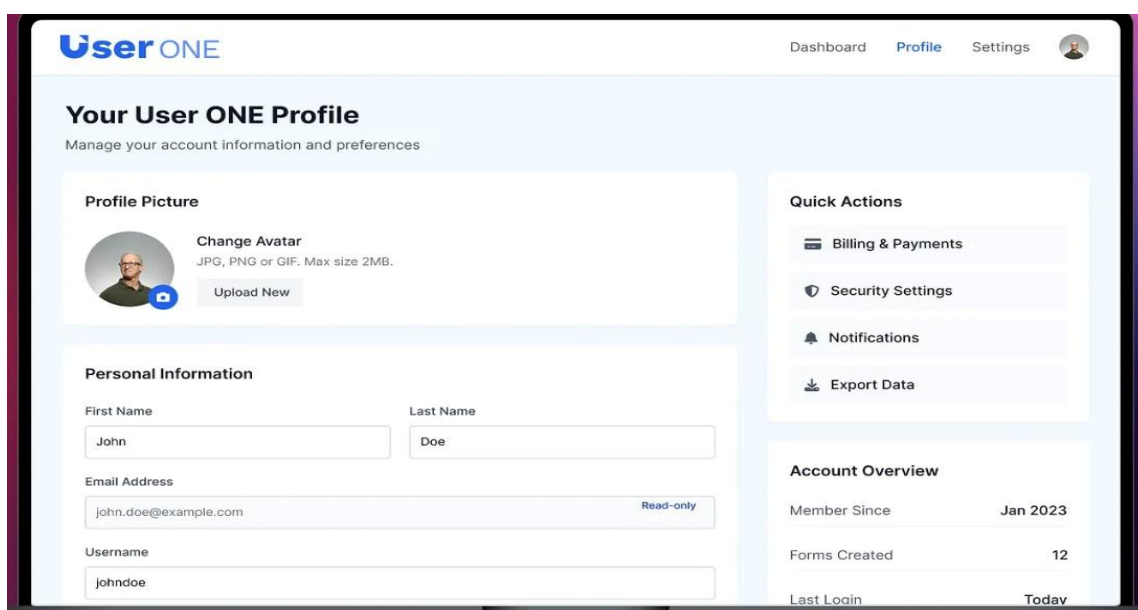


Figure 40 Profile

The prominent features that are captured as points of reference.

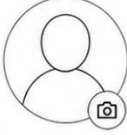
- Profile header card showing voter name, profile photo and registered constituency to be immediately identified.
- Verification status badge that states whether the citizenship documents of the voter are approved, pending review or rejected.
- Personal details section with read-only fields: date of birth, citizenship number and registered address.
- Document upload history: This will show a list of verification documents uploaded previously with their date of upload and status of their review.
- Account settings section offers the choice to change the contact details and to change the password.
- Out and account management actions: actions that are obviously placed to uphold the expected navigation conventions.

3.11.1 Wire frame

**NirvachanSetu**

DashboardCandidatesVoteResultsProfile





Arjun Srinivasan
VOTER ID: A061234567

STATUS

● Active

REGISTERED ON

12 Oct 2021

Polling Information

 **Constituency**
Bangalore South (IN)

 **Polling Station**
St. Mary's School, 5th Main, Block 2

View Polling Map →

Personal Particulars

Edit Profile

Details as recorded in the National Electoral Roll

FULL NAME

Arjun Srinivasan

DATE OF BIRTH

14 May 1992

GENDER

Male

MOBILE NUMBER

+91 98XXXX X5042

REGISTERED RESIDENTIAL ADDRESS

Flat 402, Green Meadows Apts., 6th Main, HSR Layout Sector 2, Bangalore - 560102

FATHER'S NAME

Srinivasan Ramasubram (Father)

EPIC CARD TYPE

Digital EPIC (e-EPIC)

 **Administrative Notice**
To ensure the integrity of the electoral process, any modification to your profile details except mobile number will require approval by the Electoral Registration Officer. Please ensure you have supporting documents ready for upload.

 **Recent Activity**

 **Identity Re-verification**
Completed via Aadhaar OTP

01 May 2024
Success

 **Voter Slip Downloaded**
Constituency: Bangalore South

04 Oct 2024
11:30 AM

**NirvachanSetu**
© 2024 Election Commission. All rights reserved.

Privacy PolicyTerms of ServiceAccessibilityContact Support

Figure 41 Wire frame

3.11.2 Developed Design

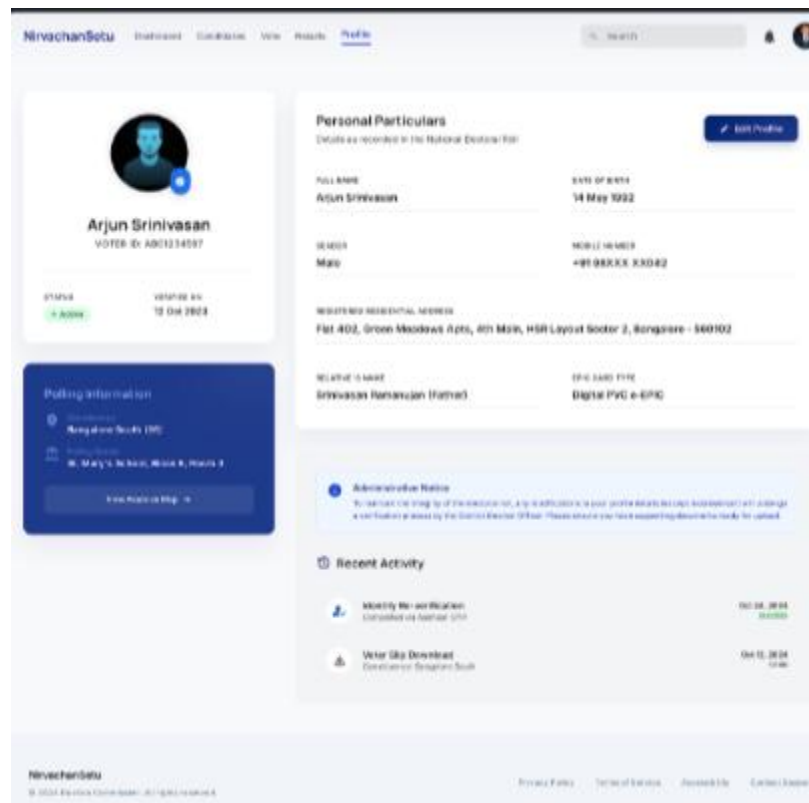


Figure 42 Developed Design

3.12 Candidate dashboard

Candidate Dashboard is the main interface of registered candidates in the NirvachanSetu system. It gives candidates a single view of their application status, real-time number of votes during active elections, campaign performance metrics and access to their profile and manifesto management tools. Personal analytics and campaign-management dashboards that combine performance measures in a hierarchical format were researched.

Reference Websites / UI Templates.

The basic source was the Shadcn UI Dashboard Example, which illustrates the utilization of summary KPI cards, activity timelines, and sidebar navigation to display a personalized operational overview. Its modular card design and data visualization method directly contributed to the Candidate Dashboard design. (shadcn, 2024)

Other supporting sources are:

- HubSpot CRM Dashboard — A popular sales and campaign analytics interface that was used as a reference in real-time metric cards, progress indicators, and ranked leaderboard views which could be applied to candidate vote-count tracking. (HubSpot Inc, 2024)
- Ant Design Pro Analytics Page - Includes charting reference to display vote distribution over time using area and bar chart elements, which can be used to provide at-a-glance monitoring of campaign performance. (Ant Design Team, 2024)

Researched UI Screenshot

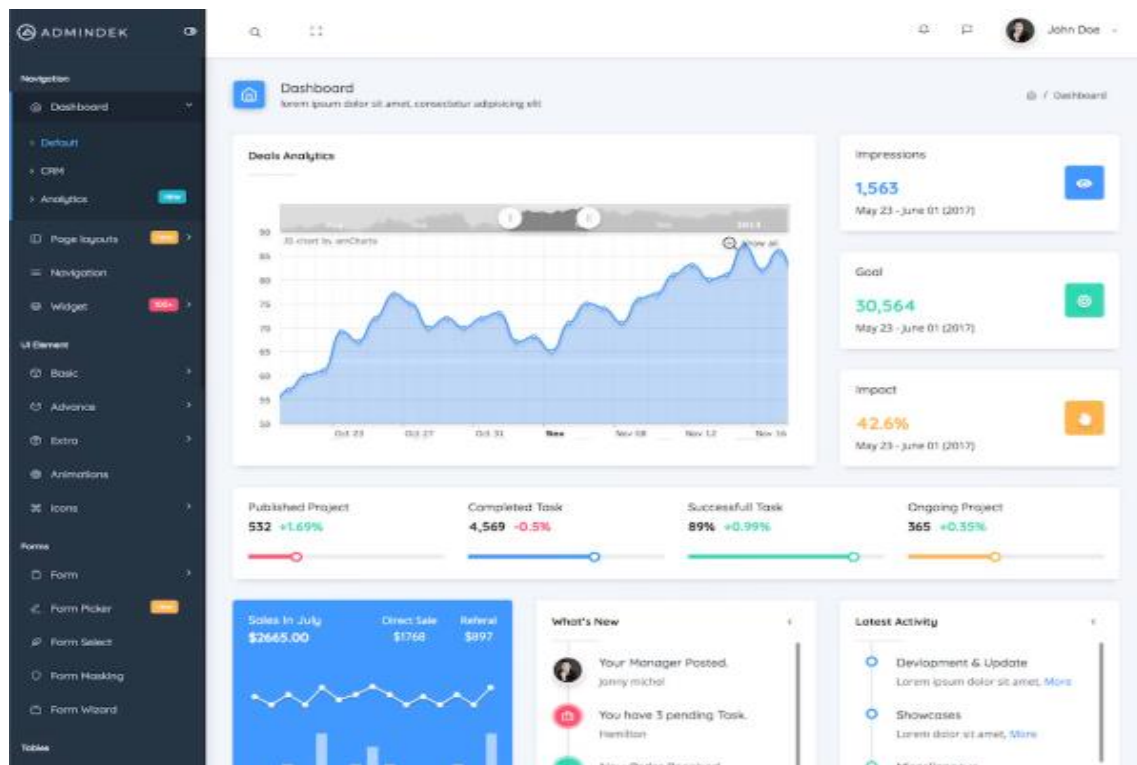


Figure 43 Candidate dashboard

Notable Features Taken as Points of Reference

- Summary KPI cards that show the total votes received, current position among candidates, constituency, application status to give an overview of the campaign immediately.
- Live vote counter tracker that is updated during the ongoing election to allow candidates to see their positions.
- Application status indicator: This clearly indicates application status of either awaiting application, approved or rejected by the election administrator.
- Navigation links to sub-pages specific to the candidate such as Application, Profile and Manifesto, and Competitors.
- A chart of vote distributions over the election period that make use of either an area or bar chart format.
- System announcement notification panel: This would provide system notifications that are important to the candidate, e.g. the results of document reviews and adjustments to the election schedule.

3.12.1 Wireframe

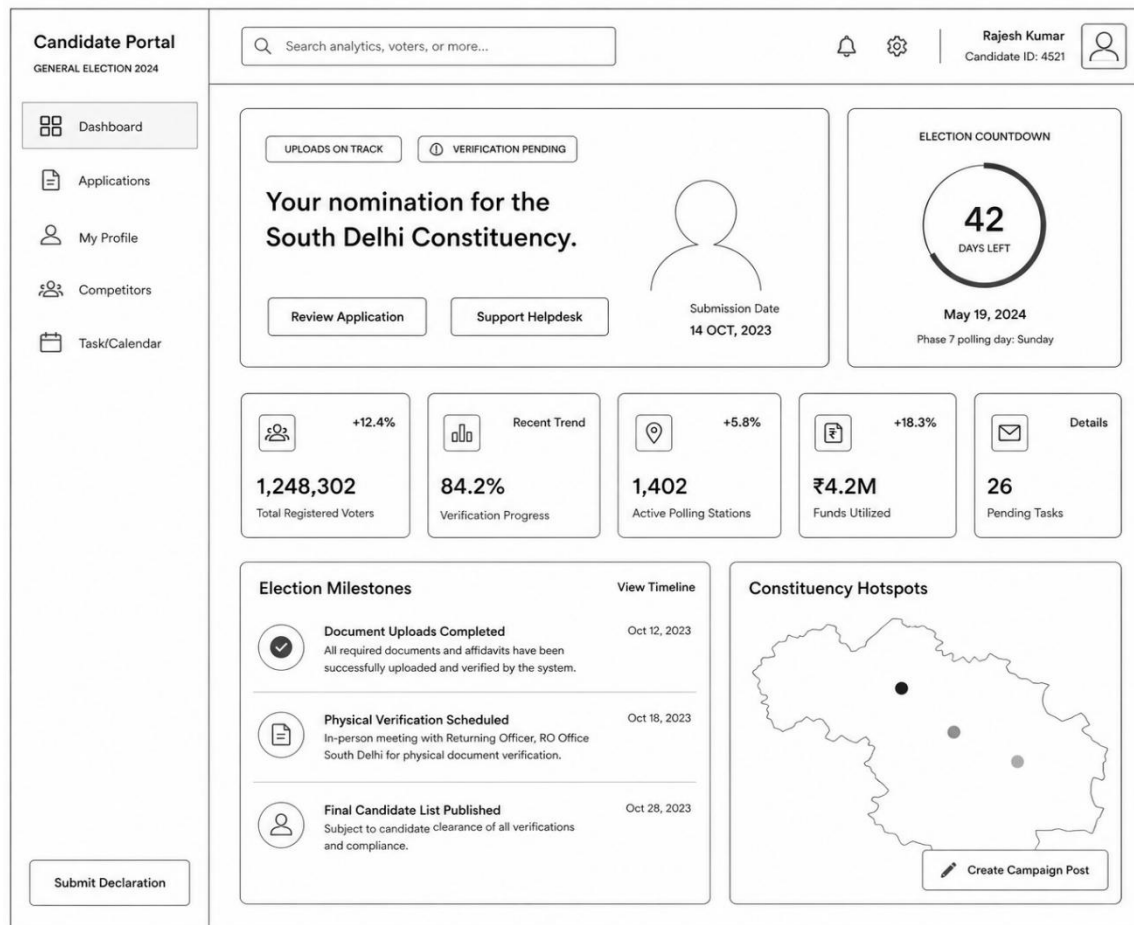


Figure 44 Wireframe

3.12.2 Developed Design

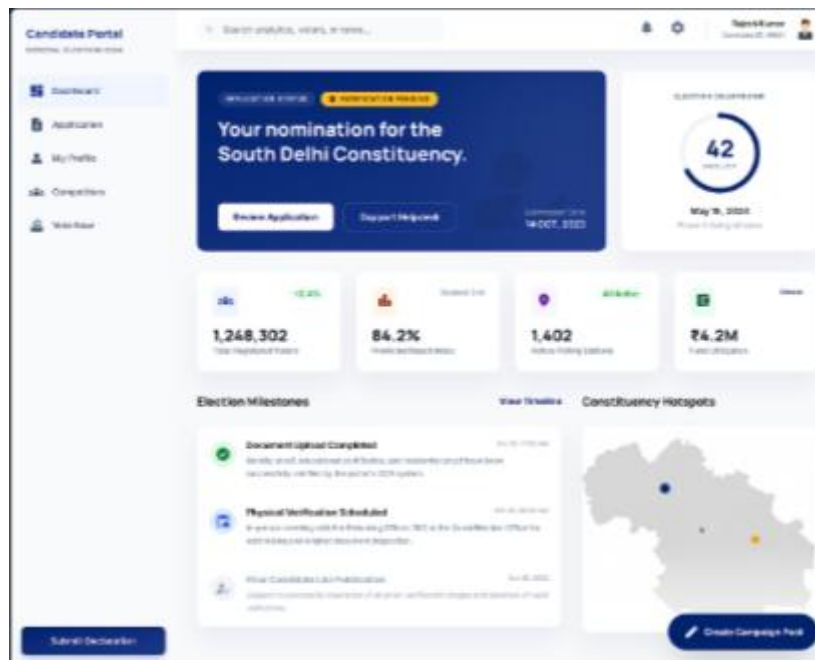


Figure 45 Developed Design

3.13 Application

Application page is used to officially submit a candidacy application in the NirvachanSetu system. It is a structured form by which an eligible citizen may announce their desire to challenge a particular election by submitting personal information, choosing which constituency to contest and uploading the supporting documents necessary. Multi-step form interfaces and application submission workflows in civic and enterprise settings were researched.

Reference Websites / UI Templates.

The main source was the Gov.uk Apply for a Licence and Multi-Step Form Pattern, which sets a standard on how the applicant is guided through the data entry steps in a sequential manner with the field marked and labelled in a logical manner, inline validation and a summary review screen in front of a final submission. (Government Digital Service, 2024)

Other supporting material would be:

- Typeform Multi-Step Application Interface Provided reference is a progressive presentation of a form with several stages, which allows reducing cognitive load by showing one part of the form at a time, which is suitable in a structured candidacy application. (Typeform SL, 2024)
- Shadcn UI - Form Components - Provided reference to labelled input fields, file upload controls, dropdown selectors, and validation messages all of which are core components of the Application form. (shadcn, 2024)
- Election Commission of Nepal - Online Candidate Nomination Guidelines - A domain specific civic reference that identifies the mandatory fields and document types required to have a valid candidacy nomination, informing the content structure of the Application form. (Election Commission of India, 2024)

Researched UI Screenshot

Bug Report
Help us improve by reporting bugs you encounter.

Bug Title

Login button not working on mobile

Description

I'm having an issue with the login button on mobile.

0/100 characters

Include steps to reproduce, expected behavior, and what actually happened.

Reset **Submit**

Figure 46 Application

Remarkable Characteristics that are captured as points of reference.

- Step-by-step form layout with separation of personal information, election and constituency selection, and document upload into separate sections to minimize the complexity of the form and increase completion rates.
- Clearly identified input fields containing candidate name, date of birth, citizenship number, contact details and party affiliation.

- Selection of constituency and election selection using dropdown controls that are filled with available options in the system database.
- Document upload section where citizenship certificates, nomination papers and other documents as required by law are uploaded.
- On-the-fly field validation that gives immediate feedback about the missing entries or incorrectly formatted entries before the form is submitted.
- Application summary review screen that shows all the information that has been entered on it to be finally verified before the application can be submitted.
- Confirmation of submission message where the candidate gets a unique application reference number that he or she is identified by.

3.13.1 Wire frame

The wireframe shows a web application titled "NirvachanSetu" with a "Cancel Application" link. The form is divided into several sections:

- Candidate Nomination:** A sidebar section with instructions: "Please provide accurate information for your official candidacy. Your nomination will be reviewed by the Election Commission." It includes a profile picture placeholder and the name "Vikram Singh Rathore".
- Required Files:** A list of required documents: Government ID Proof (PDF), Domicile Certificate, Affidavit by Candidate, and Recent Passport Photo.
- Identity Details:** A main section with fields for "FULL NAME" (placeholder: "Enter as per Aadhaar/ID"), "PARTY AFFILIATION" (placeholder: "Independent or Party Name"), and "CONSTITUENCY ELECTION" (placeholder: "Choose your primary constituency").
- Document Verification:** Two upload areas: "ID Proof" (placeholder: "Upload Max File (5 MB)") and "Nomination Papers" (placeholder: "Upload Max File (10 MB)"). Each has a "Choose File" button.
- Declaration:** A checkbox with the text: "I hereby declare that the information provided is true to the best of my knowledge and I understand that any discrepancy may lead to immediate disqualification."
- Submit Button:** A large button labeled "Submit Nomination" with a right arrow.
- Footer:** "SECURE TRANSMISSION ENABLED • AES PROTOCOL V2.0" and "© 2024 NirvachanSetu • Election Commission Digital Infrastructure".

Figure 47 Wire frame

3.13.2 Developed design

The screenshot displays the NirvachanSetu application interface for candidate nomination. The header includes the logo and name 'NirvachanSetu' and a link to 'Secret Application'. The main content area is divided into two columns. The left column contains a 'Candidate Nomination' section with instructions and a 'Required Files' box listing 'Candidate and Nomination Form (PDF)', 'Nomination Form (PDF)', and 'Candidate Photo (JPG)'. The right column contains an 'Identity Details' section with fields for 'Full Name' (with a dropdown for 'Select your Affiliation'), 'Party Affiliation' (with a dropdown for 'Select your Party Name'), and 'Secondary Address' (with a dropdown for 'Choose your primary constituency'). Below this is a 'Document Verification' section with two boxes for 'ID Photo' and 'Nomination Paper', each with a 'Document File' label. At the bottom, there is a checkbox for 'I hereby declare that the information provided is true to the best of my knowledge and I understand that any discrepancy may lead to nomination being rejected' and a 'Submit Nomination' button. The footer contains the text '© 2024 NirvachanSetu - Election Commission Digital Infrastructure'.

Figure 48 Developed design

3.14 Profile and manifesto

The Profile and Manifesto page offers each candidate a unique public-facing profile that voters can visit to learn more about the background and policy views of the candidate. It is a mix of biographical details and a manifesto part where the candidate indicates his or her main promises and goals. The profile pages of political candidates and interfaces to public-facing interfaces of professional biography platforms have been studied.

Reference Websites / UI Templates.

The main source was the official candidate profile pages of the Election Commission of India and the UK Electoral Commission, both of which provide structured biographical summary and officially stated candidate statements or manifestos, which sets a civic precedent of transparent candidate communication. (Election Commission of India, 2024)

Other supporting sources are:

- LinkedIn Public Profile Page -It was used as a reference to create the structure of a professional biography along with a header section, summary statement, key attributes, and a structured body of content, which is displayed in a clean single-column format. (LinkedIn Corporation, 2024)
- Dribbble Political candidate profile UI concepts Community-submitted UI concepts demonstrating how to present the profile photo of a candidate, their name, party, constituency, and manifesto in a visually appealing and yet structured design.
- Medium Article Page - This is the reference to the typographic treatment and reading structure of the section of the manifesto text, provided in medium article format. (Medium Corp, 2024)

Researched UI Screenshot

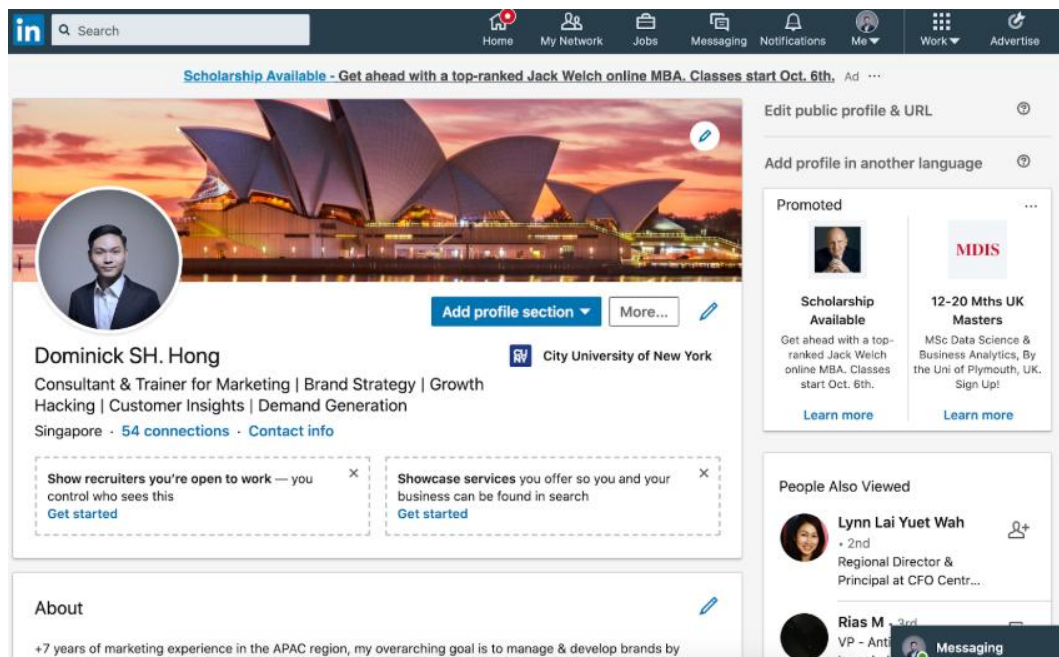


Figure 49 Profile and manifesto

Notable Features Taken as Points of Reference

- Profile header section with details on candidate photograph, full name, party affiliation, constituency and application status badge.
- Biographical summary which gives a brief and personal and professional background statement written by the candidate.
- Manifesto section organized with headings and paragraphs to convey the major policy promises made by the candidate in the relevant areas, including infrastructure, education and governance.
- Fields of candidate-editable content where the profile and manifesto text can be edited via the candidate management interface.
- Public visibility indicator that notifies the candidate on the level at which the profile of the candidate is currently visible to registered voters.

3.14.1 Wire frame

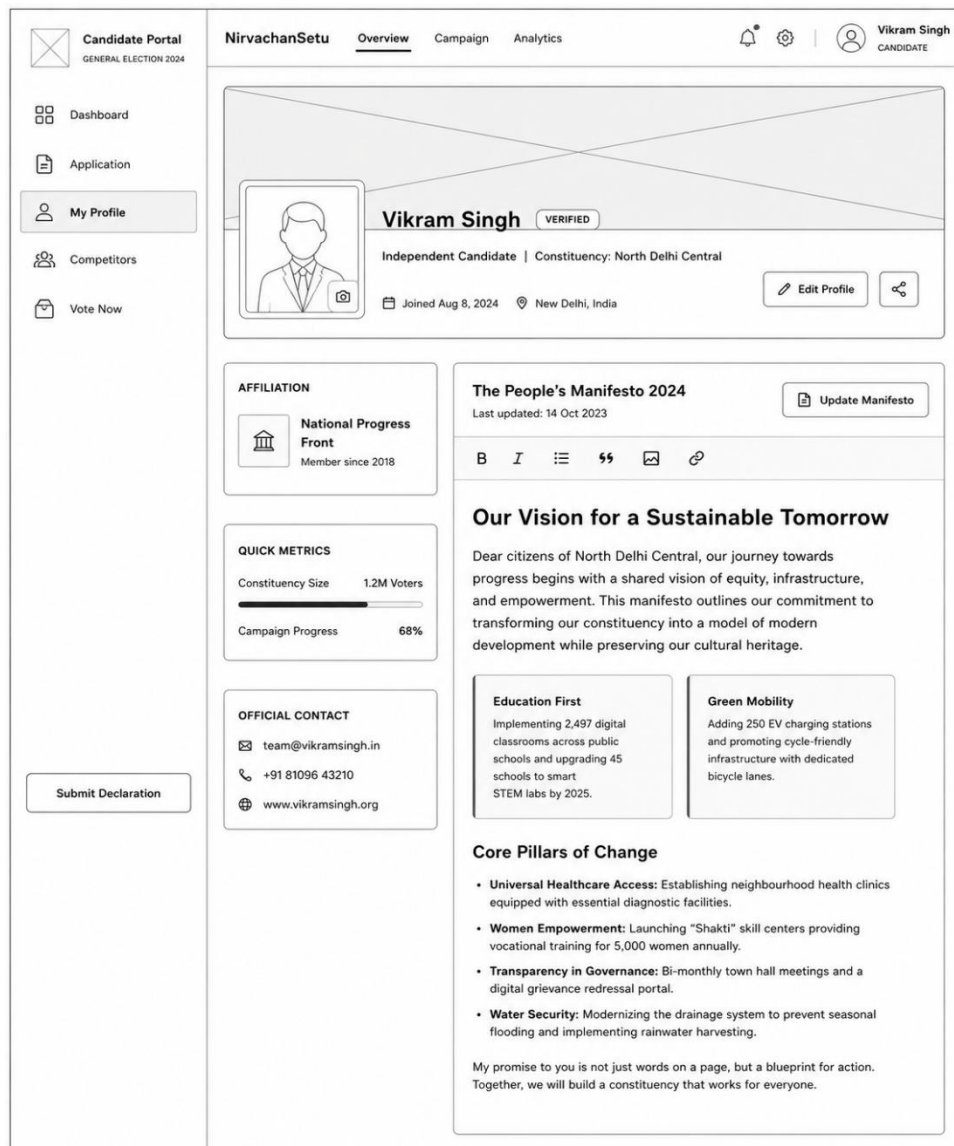


Figure 50 Wire frame

3.14.2 Developed design

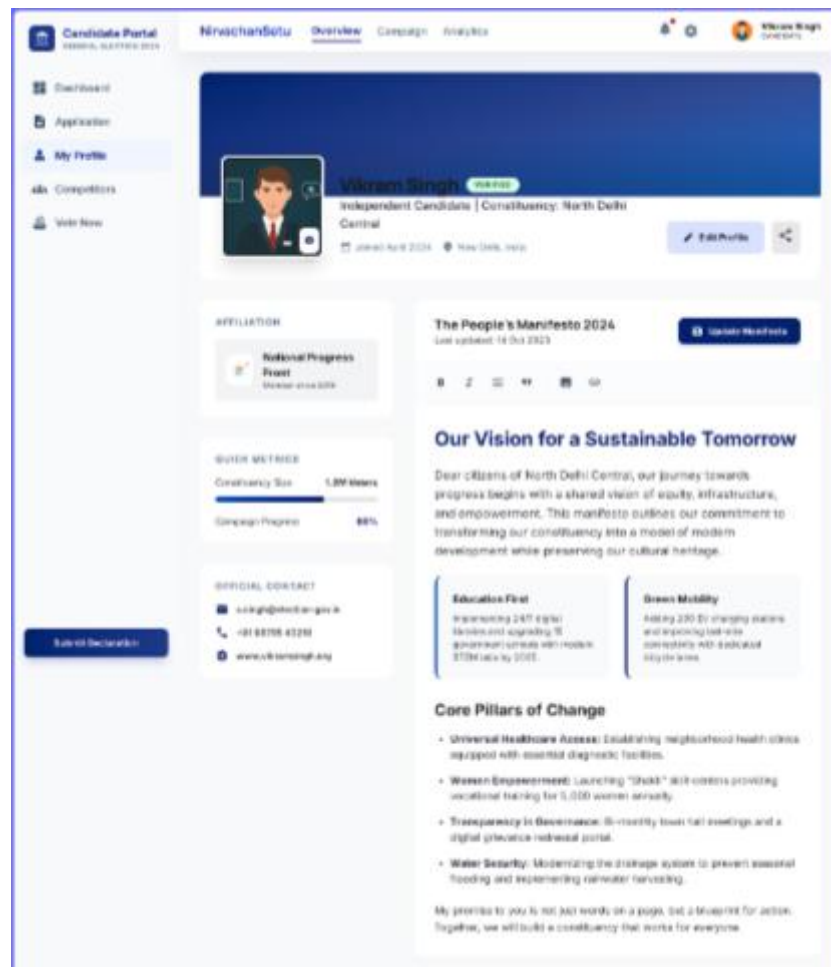


Figure 51 Developed design

3.15 Competitors

The Competitors page gives the candidate a glimpse of all the other candidates that are competing for the same election in his or her constituency. This interface enables candidates to examine the competitive environment, the profile of their opponents and track relative positions in an ongoing election. The leaderboard, comparison and competitor analysis interfaces were studied in the context of civic technology as well as enterprise applications.

Reference Websites / UI Templates.

The main source was the Shadcn UI - Leaderboard and Rankings Component, which illustrates a clean ranked list layout with names, scores, and relative position with the help of row-based cards and position indicators. This trend specifically had an effect on the ranked competitor listing format. (shadcn, 2024)

The other supporting sources are:

- BBC Election Results - Constituency Breakdown Page - This publicly facing civic reference site provides the public with the necessary service and it is important to note that the Competitors page requires equal the amount of functionality. (British Broadcasting Corporation, 2024)
- Kaggle Competition Leaderboard -Provided reference on how the ranked entries with position numbers, participant names, scores, and change-in-rank indicators should be visually treated in a compact and scannable table format. (Google LLC, 2024)
- Comparison Table Provided an enterprise level reference to display multiple entities side by side with comparative metrics, which could be used to display competing candidates with their respective vote tallies. (Ant Design Team, 2024)

Researched UI Screenshot

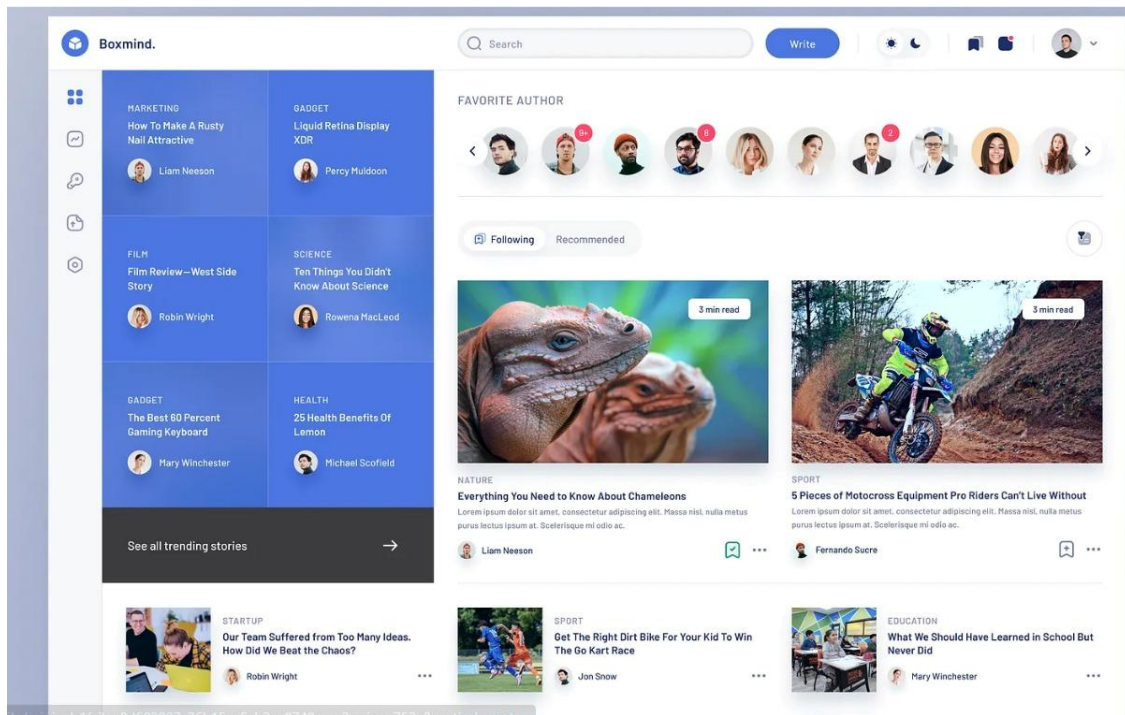


Figure 52 Competitors

Notable Features - Taken to serve as points of reference.

- List of ranked competitors showing all candidates in the same constituency in a list of current vote counts with position numbers clearly indicated.
- Candidate rows with entries of name, party, number of votes and percentage of votes per competitor.
- Row with a highlight indicating the current logged-in candidate his/her own position in the ranked list to immediately assess himself/herself against the list.
- Real-time updates of the number of votes during the ongoing election period to allow candidates to check their positions as the votes are counted.
- Profile view link each competitor pointing to their open Profile and Manifesto page, in support of transparency.
- Constituency label which ensures that all listed competitors are in the same electoral area and thus they do not cause confusion when there are multiple electoral areas represented.

3.15.1 Wire frame

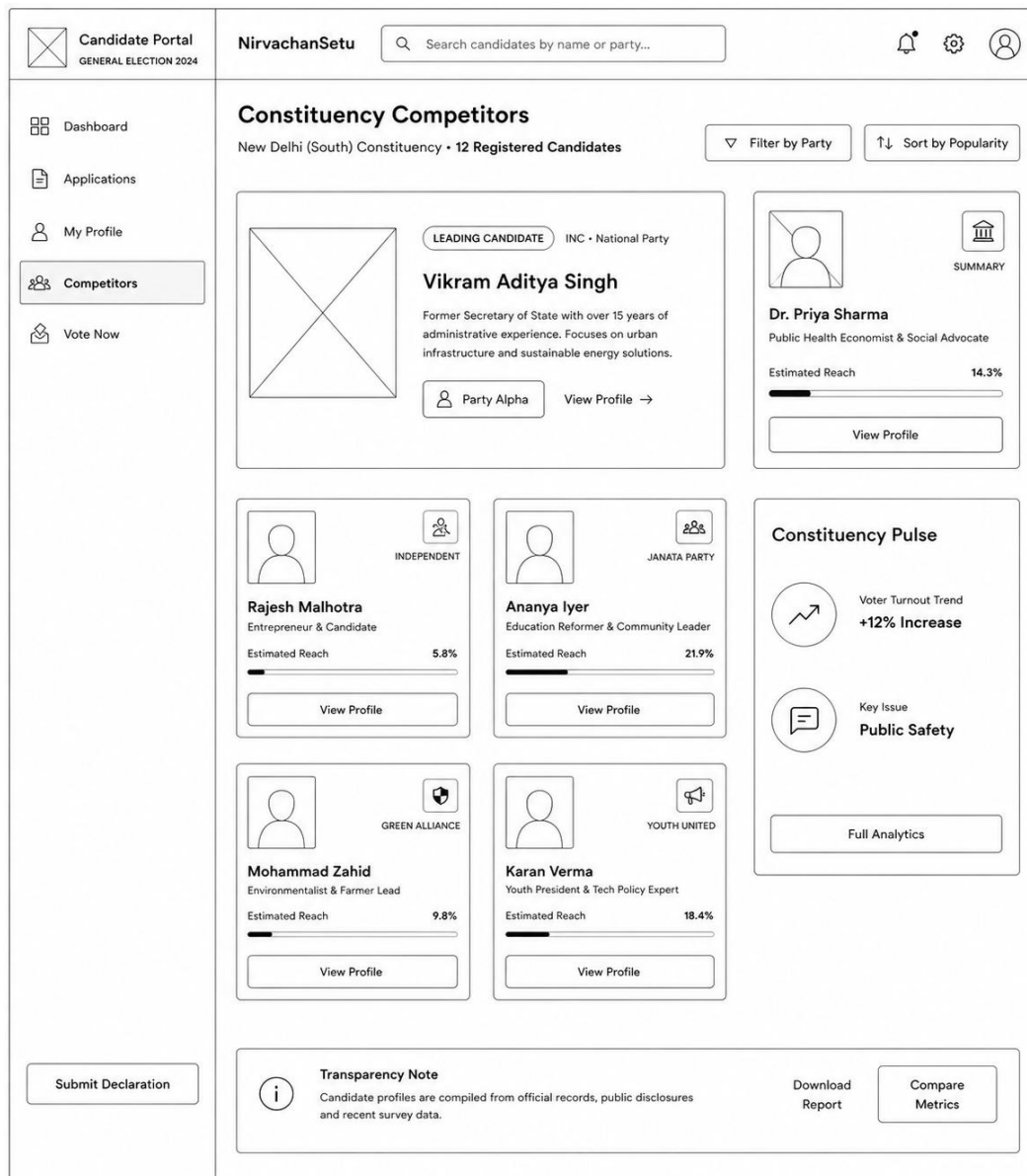


Figure 53 Wire frame

3.15.2 Developed design

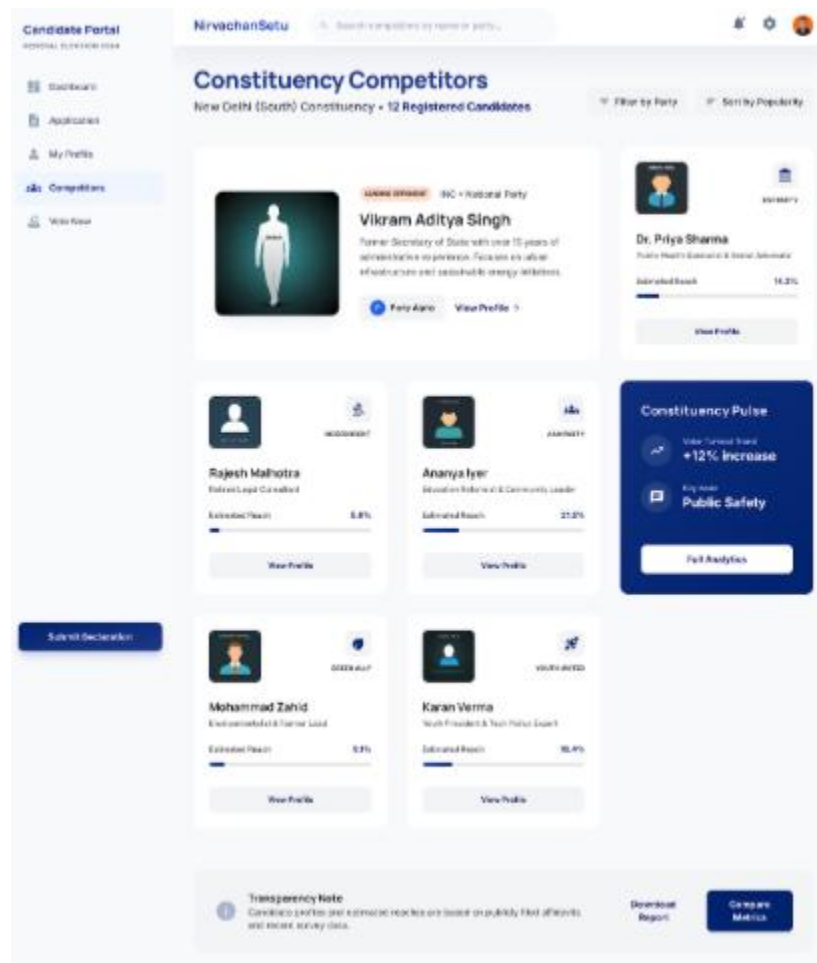


Figure 54 Developed design

4 Java Classes and Methods

4.1 Class Diagram

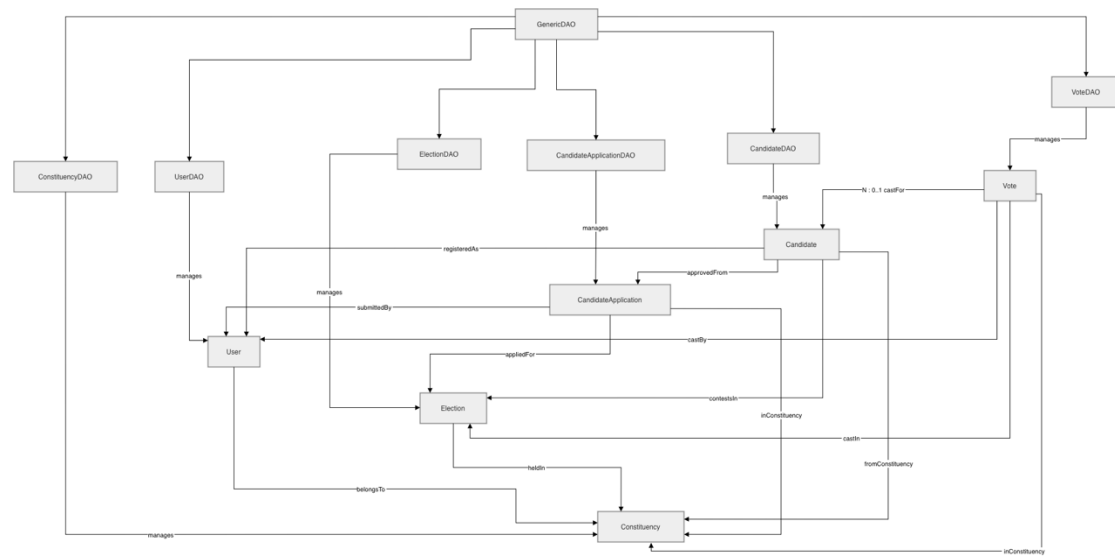


Figure 55: Class Diagram

5 References

There are no sources in the current document.

5.1 Method explanation

5.1.1 DAOs

a. CandidateApplicationDAO

CandidateApplicationDAO
+findByUserAndElection(userId, electionId) : CandidateApplication +findByElection(electionId) : List +findByStatus(status) : List +findByUser(userId) : List

Figure 56 CandidateApplicationDAO

i. findByUserAndElection

```
public CandidateApplication findByUserAndElection(Integer userId,
Integer electionId)
```

Finds a candidate application by user and election.

Parameters:

userId - the user ID

electionId - the election ID

Returns:

the CandidateApplication, or null if not found

```
public CandidateApplication findByUserAndElection(Integer userId, Integer electionId) {
    try {
        TypedQuery<CandidateApplication> query = em.createQuery(
            "SELECT ca FROM CandidateApplication ca WHERE ca.user.id = :userId AND ca.election.id = :electionId",
            CandidateApplication.class);
        query.setParameter("userId", userId);
        query.setParameter("electionId", electionId);
        List<CandidateApplication> results = query.getResultList();
        return results.isEmpty() ? null : results.get(0);
    } catch (Exception e) {
        throw new RuntimeException("Error finding application for user " + userId + " and election " + electionId, e);
    }
}
```

Figure 57 findByUserAndElection Code

ii. findByElection

```
public List<CandidateApplication> findByElection(Integer electionId)
```

Finds all candidate applications for a specific election.

Parameters:

electionId - the election ID

Returns:

a list of candidate applications for the election

```
public List<CandidateApplication> findByElection(Integer electionId) {  
    try {  
        TypedQuery<CandidateApplication> query = em.createQuery(  
            "SELECT ca FROM CandidateApplication ca WHERE ca.election.id = :electionId ORDER BY ca.appliedAt DESC",  
            CandidateApplication.class);  
        query.setParameter("electionId", electionId);  
        return query.getResultList();  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding applications by election: " + electionId, e);  
    }  
}
```

Figure 58 findByElection Code

iii. findByStatus

```
public List<CandidateApplication> findByStatus(String status)
```

Finds all candidate applications with a specific status.

Parameters:

status - the application status

Returns:

a list of candidate applications with the given status

```
public List<CandidateApplication> findByStatus(String status) {  
    try {  
        TypedQuery<CandidateApplication> query = em.createQuery(  
            "SELECT ca FROM CandidateApplication ca WHERE ca.status = :status ORDER BY ca.appliedAt DESC",  
            CandidateApplication.class);  
        query.setParameter("status", CandidateApplication.ApplicationStatus.valueOf(status));  
        return query.getResultList();  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding applications by status: " + status, e);  
    }  
}
```

Figure 59 *findByStatus* Code

iv. findByUser

```
public List<CandidateApplication> findByUser(Integer userId)
```

Finds all candidate applications submitted by a specific user.

Parameters:

userId - the user ID

Returns:

a list of candidate applications by the user

```
public List<CandidateApplication> findByUser(Integer userId) {  
    try {  
        TypedQuery<CandidateApplication> query = em.createQuery(  
            queryString: "SELECT ca FROM CandidateApplication ca WHERE ca.user.id = :userId ORDER BY ca.appliedAt DESC",  
            CandidateApplication.class);  
        query.setParameter( name: "userId", userId);  
        return query.getResultList();  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding applications by user: " + userId, e);  
    }  
}
```

Figure 60 findByUser Code

b. CandidateDAO

CandidateDAO
+findByUser(userId) : Candidate
+findByElection(electionId) : List
+findByConstituency(constituencyId) : List
+findByElectionAndConstituency(eld, cld) : List
+findActiveByElection(electionId) : List
+countByElection(electionId) : long

*Figure 61 CandidateDAO***i. findByUser**

public Candidate findByUser(Integer userId)

Finds a candidate by user ID.

Parameters:

userId - the user ID

Returns:

the Candidate, or null if not found

```
public Candidate findByUser(Integer userId) { 1 usage 2 Sugam-Khatiwada
    try {
        TypedQuery<Candidate> query = em.createQuery(
            queryString: "SELECT c FROM Candidate c WHERE c.user.id = :userId", Candidate.class);
        query.setParameter( name: "userId", userId);
        List<Candidate> results = query.getResultList();
        return results.isEmpty() ? null : results.get(0);
    } catch (Exception e) {
        throw new RuntimeException("Error finding candidate by user: " + userId, e);
    }
}
```

Figure 62 findByUser Code

ii. findByElection

public List<Candidate> findByElection(Integer electionId)

Finds all candidates for a specific election.

Parameters:

electionId - the election ID

Returns:

a list of candidates in the election

```
public List<Candidate> findByElection(Integer electionId) { 4 usages 2 Sugam-Khatiwada
    try {
        TypedQuery<Candidate> query = em.createQuery(
            queryString: "SELECT c FROM Candidate c WHERE c.election.id = :electionId ORDER BY c.totalVotes DESC",
            Candidate.class);
        query.setParameter( name: "electionId", electionId);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding candidates by election: " + electionId, e);
    }
}
```

Figure 63 findByElection code

iii. findByConstituency

public List<Candidate> findByConstituency(Integer constituencyId)

Finds all candidates in a specific constituency.

Parameters:

constituencyId - the constituency ID

Returns:

a list of candidates in the constituency

```
public List<Candidate> findByConstituency(Integer constituencyId) { 1 usage  Sugam-Khatiwada
    try {
        TypedQuery<Candidate> query = em.createQuery(
            queryString: "SELECT c FROM Candidate c WHERE c.constituency.id = :constituencyId ORDER BY c.totalVotes DESC",
            Candidate.class);
        query.setParameter( name: "constituencyId", constituencyId);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding candidates by constituency: " + constituencyId, e);
    }
}
```

Figure 64 indByConstituency code

iv. findByElectionAndConstituency

```
public List<Candidate> findByElectionAndConstituency(Integer electionId,  
Integer constituencyId)
```

Finds all candidates for a specific election in a specific constituency.

Parameters:

electionId - the election ID

constituencyId - the constituency ID

Returns:

a list of candidates in the election and constituency

```
public List<Candidate> findByElectionAndConstituency(Integer electionId, Integer constituencyId) { 2 usages  ▲ Sugam-Khatiwada  
    try {  
        TypedQuery<Candidate> query = em.createQuery(  
            @String: "SELECT c FROM Candidate c WHERE c.election.id = :electionId AND c.constituency.id = :constituencyId ORDER BY c.totalVotes DESC",  
            Candidate.class);  
        query.setParameter(name: "electionId", electionId);  
        query.setParameter(name: "constituencyId", constituencyId);  
        return query.getResultList();  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding candidates by election " + electionId  
            + " and constituency " + constituencyId, e);  
    }  
}
```

Figure 65 *findByElectionAndConstituency* code

v. findActiveByElection

public List<Candidate> findActiveByElection(Integer electionId)

Finds all active candidates for a specific election (used for role revert).

Parameters:

electionId - the election ID

Returns:

a list of active candidates in the election

```
public List<Candidate> findActiveByElection(Integer electionId) { 1 usage  Sugam-Khatiwada
    try {
        TypedQuery<Candidate> query = em.createQuery(
            queryString: "SELECT c FROM Candidate c WHERE c.election.id = :electionId AND c.status = :status",
            Candidate.class);
        query.setParameter( name: "electionId", electionId);
        query.setParameter( name: "status", com.nirvachansetu.model.Candidate.CandidateStatus.ACTIVE);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding active candidates by election: " + electionId, e);
    }
}
```

Figure 66 findActiveByElection code

vi. countByElection

public long countByElection(Integer electionId)

Counts the number of candidates in a specific election.

Parameters:

electionId - the election ID

Returns:

the count of candidates

```
public long countByElection(Integer electionId) { 1 usage  Sugam-Khatriwada
    try {
        TypedQuery<Long> query = em.createQuery(
            queryString: "SELECT COUNT(c) FROM Candidate c WHERE c.election.id = :electionId",
            Long.class);
        query.setParameter( name: "electionId", electionId);
        return query.getSingleResult();
    } catch (Exception e) {
        throw new RuntimeException("Error counting candidates by election: " + electionId, e);
    }
}
```

Figure 67 countByElection code

c. ConstituencyDAO*Figure 68 ConstituencyDAO***i. findByDistrict**

`public List<Constituency> findByDistrict(String district)`

Finds all constituencies in a specific district.

Parameters:

district - the district name

Returns:

a list of constituencies in the district

```
public List<Constituency> findByDistrict(String district) { no usages  Sugam-Khatiwada
    try {
        TypedQuery<Constituency> query = em.createQuery(
            queryString: "SELECT c FROM Constituency c WHERE c.district = :district ORDER BY c.name",
            Constituency.class);
        query.setParameter( name: "district", district);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding constituencies by district: " + district, e);
    }
}
```

Figure 69 findByDistrict code

ii. findByProvince

public List<Constituency> findByProvince(String province)

Finds all constituencies in a specific province.

Parameters:

province - the province name

Returns:

a list of constituencies in the province

```
public List<Constituency> findByProvince(String province) { no usages  Sugam-Khatiwada
    try {
        TypedQuery<Constituency> query = em.createQuery(
            queryString: "SELECT c FROM Constituency c WHERE c.province = :province ORDER BY c.name",
            Constituency.class);
        query.setParameter( name: "province", province);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding constituencies by province: " + province, e);
    }
}
```

Figure 70 findByProvince code

d. ElectionDAO

ElectionDAO
<pre>+findByStatus(status) : List +findByType(type) : List +findActiveElectionByConstituency(id) : Election</pre>

*Figure 71 ElectionDAO code***i. findByStatus**

`public List<Election> findByStatus(String status)`

Finds all elections with a specific status.

Parameters:

status - the election status

Returns:

a list of elections with the given status

```
public List<Election> findByStatus(String status) { 2 usages 2 Sugam-Khatiwada  
    try {  
        TypedQuery<Election> query = em.createQuery(  
            queryString: "SELECT e FROM Election e WHERE e.status = :status ORDER BY e.startDate DESC",  
            Election.class);  
        query.setParameter( name: "status", Election.ElectionStatus.valueOf(status));  
        return query.getResultList();  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding elections by status: " + status, e);  
    }  
}
```

Figure 72 findByStatus code

ii. findByType

```
public List<Election> findByType(String type)
```

Finds all elections of a specific type.

Parameters:

type - the election type

Returns:

a list of elections of the given type

```
public List<Election> findByType(String type) { 1 usage 2 Sugam-Khatiwada
    try {
        TypedQuery<Election> query = em.createQuery(
            qString: "SELECT e FROM Election e WHERE e.electionType = :type ORDER BY e.startDate DESC",
            Election.class);
        query.setParameter( name: "type", Election.ElectionType.valueOf(type));
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding elections by type: " + type, e);
    }
}
```

Figure 73 findByType code

iii. findActiveElectionByConstituency

public Election findActiveElectionByConstituency(Integer constituencyId)

Finds the active election for a specific constituency.

Parameters:

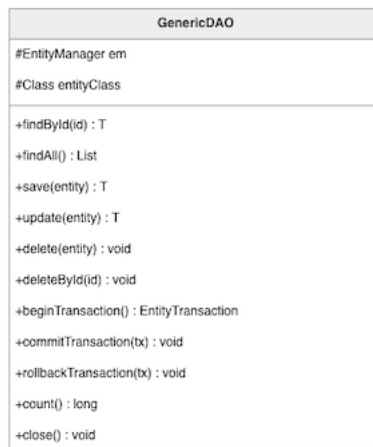
constituencyId - the constituency ID

Returns:

the active election, or null if not found

```
public Election findActiveElectionByConstituency(Integer constituencyId) {  
    try {  
        TypedQuery<Election> query = em.createQuery(  
            "SELECT e FROM Election e WHERE e.status = :status AND e.constituency.id = :constituencyId",  
            Election.class);  
        query.setParameter( "name: \"status\"", Election.ElectionStatus.ACTIVE);  
        query.setParameter( "name: \"constituencyId\"", constituencyId);  
        List<Election> results = query.getResultList();  
        return results.isEmpty() ? null : results.get(0);  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding active election by constituency: " + constituencyId, e);  
    }  
}
```

Figure 74 findActiveElectionByConstituency code

e. **GenericDAO<T,ID>***Figure 75 GenericDAO<T,ID>*i. **findById**

```
public T findById(ID id)
```

Finds an entity by its primary key.

Parameters:

id - the primary key

Returns:

the entity, or null if not found

```

public T findById(ID id) {  @ Sugam-Khatriwada
    try {
        return em.find(entityClass, id);
    } catch (Exception e) {
        throw new RuntimeException("Error finding entity by ID: " + id, e);
    }
}
  
```

Figure 76 findById code

ii. findAll`public List<T> findAll()`

Returns all entities of this type.

Returns:

a list of all entities

```
public List<T> findAll() {  Sugam-Khatiwada
    try {
        String jpql = "SELECT e FROM " + entityClass.getSimpleName() + " e";
        TypedQuery<T> query = em.createQuery(jpql, entityClass);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding all " + entityClass.getSimpleName() + " entities", e);
    }
}
```

Figure 77 findAll code

iii. save

public T save(T entity)

Saves a new entity (persist) or updates an existing one (merge).

Parameters:

entity - the entity to save

Returns:

the saved entity

```
public T save(T entity) { // Sugam-Khatriwada
    EntityTransaction tx = null;
    try {
        tx = beginTransaction();
        if (em.contains(entity)) {
            entity = em.merge(entity);
        } else {
            em.persist(entity);
        }
        commitTransaction(tx);
        return entity;
    } catch (Exception e) {
        if (tx != null && tx.isActive()) {
            rollbackTransaction(tx);
        }
        throw new RuntimeException("Error saving " + entityClass.getSimpleName(), e);
    }
}
```

Figure 78 save code

iv. update

public T update(T entity)

Updates an existing entity using merge.

Parameters:

entity - the entity to update

Returns:

the updated entity

```
public T update(T entity) {  @ Sugam-Khatiwada
    EntityTransaction tx = null;
    try {
        tx = beginTransaction();
        entity = em.merge(entity);
        commitTransaction(tx);
        return entity;
    } catch (Exception e) {
        if (tx != null && tx.isActive()) {
            rollbackTransaction(tx);
        }
        throw new RuntimeException("Error updating " + entityClass.getSimpleName(), e);
    }
}
```

Figure 79 update code

v. delete

public void delete(T entity)

Deletes an entity.

Parameters:

entity - the entity to delete

```
public void delete(T entity) { no usages 2 Sugam-Khatiwada
    EntityTransaction tx = null;
    try {
        tx = beginTransaction();
        // Merge first to ensure the entity is managed
        T managedEntity = em.merge(entity);
        em.remove(managedEntity);
        commitTransaction(tx);
    } catch (Exception e) {
        if (tx != null && tx.isActive()) {
            rollbackTransaction(tx);
        }
        throw new RuntimeException("Error deleting " + entityClass.getSimpleName(), e);
    }
}
```

Figure 80 delete code

vi. deleteById

public void deleteById(ID id)

Deletes an entity by its primary key.

Parameters:

id - the primary key of the entity to delete

```
public void deleteById(ID id) { 1 usage  Sugam-Khatiwada
    EntityTransaction tx = null;
    try {
        tx = beginTransaction();
        T entity = em.find(entityClass, id);
        if (entity != null) {
            em.remove(entity);
        }
        commitTransaction(tx);
    } catch (Exception e) {
        if (tx != null && tx.isActive()) {
            rollbackTransaction(tx);
        }
        throw new RuntimeException("Error deleting " + entityClass.getSimpleName() + " by ID: " + id, e);
    }
}
```

Figure 81 deleteById code

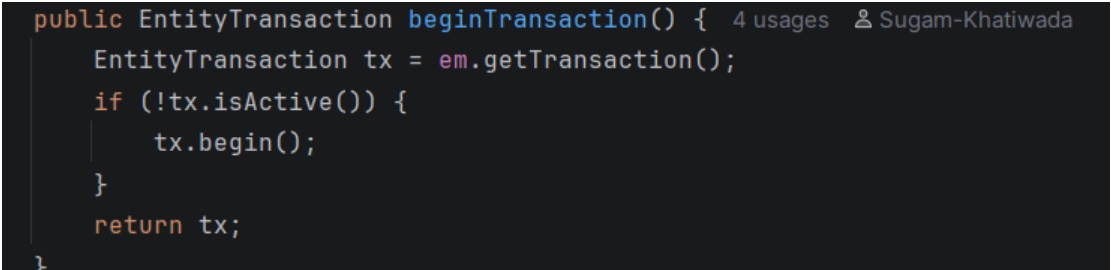
vii. beginTransaction

public jakarta.persistence.EntityTransaction beginTransaction()

Begins a new transaction.

Returns:

the EntityTransaction

A screenshot of a code editor showing the implementation of the beginTransaction method. The code is in Java and is part of the jakarta.persistence.EntityTransaction interface. It shows the method signature, a comment indicating 4 usages and the author Sugam-Khatriwada, and the implementation logic: getting the current transaction, checking if it's active, and starting a new one if necessary.

```
public EntityTransaction beginTransaction() { 4 usages  Sugam-Khatriwada
    EntityTransaction tx = em.getTransaction();
    if (!tx.isActive()) {
        tx.begin();
    }
    return tx;
}
```

Figure 82 beginTransaction code

viii. commitTransaction

public void commitTransaction(jakarta.persistence.EntityTransaction tx)
Commits the current transaction.

Parameters:

tx - the EntityTransaction to commit

```
public void commitTransaction(EntityTransaction tx) { 4 usages  Sugam-Khatiwada
    if (tx != null && tx.isActive()) {
        tx.commit();
    }
}
```

Figure 83 commitTransaction code

ix. rollbackTransaction

public void rollbackTransaction(jakarta.persistence.EntityTransaction tx)
Rolls back the current transaction.

Parameters:

tx - the EntityTransaction to roll back

```
public void rollbackTransaction(EntityTransaction tx) { 4 usages  Sugam-Khatiwada
    if (tx != null && tx.isActive()) {
        tx.rollback();
    }
}
```

Figure 84 rollbackTransaction code

x. count

public long count()

Returns the count of all entities.

Returns:

the total count

```
public long count() {  Sugam-Khatiwada
    try {
        String jpql = "SELECT COUNT(e) FROM " + entityClass.getSimpleName() + " e";
        TypedQuery<Long> query = em.createQuery(jpql, Long.class);
        return query.getSingleResult();
    } catch (Exception e) {
        throw new RuntimeException("Error counting " + entityClass.getSimpleName() + " entities", e);
    }
}
```

Figure 85 count code

xi. close

public void close()

Clears the EntityManager persistence context. NOTE: We do NOT close the EM here because the same DAO/EM instance is reused across multiple service method calls. Closing the EM would cause "EntityManager is closed" errors on subsequent calls. The EM will be garbage collected when the DAO instance is discarded.

```
public void close() {  @ Sugam-Khatiwada
    if (em != null && em.isOpen()) {
        em.clear();
    }
}
```

Figure 86 close code

f. UserDao

UserDAO
+findByEmail(email) : User
+findByCitizenshipNumber(cn) : User
+findByRole(role) : List
+findByStatus(status) : List
+findByConstituency(constituencyId) : List
+countByRole(role) : long
+countByStatus(status) : long
+findAllWithPagination(offset, limit) : List

*Figure 87 UserDao code***i. findByEmail**

`public User findByEmail(String email)`

Finds a user by email address.

Parameters:

email - the email to search for

Returns:

the User, or null if not found

```
public User findByEmail(String email) { 3 usages 2 Sugam-Khatriwada
    try {
        TypedQuery<User> query = em.createQuery(
            queryString: "SELECT u FROM User u WHERE u.email = :email", User.class);
        query.setParameter( name: "email", email);
        List<User> results = query.getResultList();
        return results.isEmpty() ? null : results.get(0);
    } catch (Exception e) {
        throw new RuntimeException("Error finding user by email: " + email, e);
    }
}
```

Figure 88 findByEmail code

ii. findByCitizenshipNumber

public User findByCitizenshipNumber(String citizenshipNumber)

Finds a user by citizenship number.

Parameters:

citizenshipNumber - the citizenship number to search for

Returns:

the User, or null if not found

```
public User findByCitizenshipNumber(String citizenshipNumber) { 1 usage  Sugam-Khatiwada
    try {
        TypedQuery<User> query = em.createQuery(
            queryString: "SELECT u FROM User u WHERE u.citizenshipNumber = :citizenshipNumber", User.class);
        query.setParameter( name: "citizenshipNumber", citizenshipNumber);
        List<User> results = query.getResultList();
        return results.isEmpty() ? null : results.get(0);
    } catch (Exception e) {
        throw new RuntimeException("Error finding user by citizenship number: " + citizenshipNumber, e);
    }
}
```

Figure 89 findByCitizenshipNumber code

iii. findByRole

public List<User> findByRole(String role)

Finds all users with a specific role.

Parameters:

role - the role to filter by

Returns:

a list of users with the given role

```
public List<User> findByRole(String role) { 1 usage  Sugam-Khatriwada
    try {
        TypedQuery<User> query = em.createQuery(
            qlString: "SELECT u FROM User u WHERE u.role = :role ORDER BY u.createdAt DESC", User.class);
        query.setParameter( name: "role", User.Role.valueOf(role));
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding users by role: " + role, e);
    }
}
```

Figure 90 findByRole code

iv. findByStatus

```
public List<User> findByStatus(String status)
```

Finds all users with a specific status.

Parameters:

status - the status to filter by

Returns:

a list of users with the given status

```
public List<User> findByStatus(String status) { no usages  Sugam-Khatiwada
    try {
        TypedQuery<User> query = em.createQuery(
            queryString: "SELECT u FROM User u WHERE u.status = :status ORDER BY u.createdAt DESC", User.class);
        query.setParameter( name: "status", User.UserStatus.valueOf(status));
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding users by status: " + status, e);
    }
}
```

Figure 91 *findByStatus* code

v. findByConstituency

public List<User> findByConstituency(Integer constituencyId)

Finds all users belonging to a specific constituency.

Parameters:

constituencyId - the constituency ID

Returns:

a list of users in the constituency

```
public List<User> findByConstituency(Integer constituencyId) { no usages  Sugam-Khatiwada
    try {
        TypedQuery<User> query = em.createQuery(
            queryString: "SELECT u FROM User u WHERE u.constituency.id = :constituencyId ORDER BY u.fullName",
            User.class);
        query.setParameter( name: "constituencyId", constituencyId);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding users by constituency: " + constituencyId, e);
    }
}
```

Figure 92 findByConstituency code

vi. countByRole

public long countByRole(String role)

Counts users with a specific role.

Parameters:

role - the role to count

Returns:

the count of users with the given role

```
public long countByRole(String role) { no usages  Sugam-Khatiwada
    try {
        TypedQuery<Long> query = em.createQuery(
            queryString: "SELECT COUNT(u) FROM User u WHERE u.role = :role", Long.class);
        query.setParameter( name: "role", User.Role.valueOf(role));
        return query.getSingleResult();
    } catch (Exception e) {
        throw new RuntimeException("Error counting users by role: " + role, e);
    }
}
```

Figure 93 countByRole code

vii. countByStatus

public long countByStatus(String status)

Counts users with a specific status.

Parameters:

status - the status to count

Returns:

the count of users with the given status

```
public long countByStatus(String status) { 1 usage 2 Sugam-Khatriwada
    try {
        TypedQuery<Long> query = em.createQuery(
            queryString: "SELECT COUNT(u) FROM User u WHERE u.status = :status", Long.class);
        query.setParameter( name: "status", User.UserStatus.valueOf(status));
        return query.getSingleResult();
    } catch (Exception e) {
        throw new RuntimeException("Error counting users by status: " + status, e);
    }
}
```

Figure 94 countByStatus code

viii. findAllWithPagination

```
public List<User> findAllWithPagination(int offset,  
int limit)
```

Finds users with pagination.

Parameters:

offset - the starting index

limit - the maximum number of results

Returns:

a list of users

```
public List<User> findAllWithPagination(int offset, int limit) { 1 usage 2 Sugam-Khatiwada  
    try {  
        TypedQuery<User> query = em.createQuery(  
            queryString: "SELECT u FROM User u ORDER BY u.createdAt DESC", User.class);  
        query.setFirstResult(offset);  
        query.setMaxResults(limit);  
        return query.getResultList();  
    } catch (Exception e) {  
        throw new RuntimeException("Error finding users with pagination", e);  
    }  
}
```

Figure 95 findAllWithPagination code

g. VoteDAO

VoteDAO
+findByElectionAndVoter(electionId, voterId) : Vote +countByElection(electionId) : long +countByCandidate(candidateId) : long +hasVoted(electionId, voterId) : boolean +findVotesByElection(electionId) : List +findVotesByCandidate(candidateId) : List

*Figure 96 VoteDAO***i. findByElectionAndVoter**

public Vote findByElectionAndVoter(Integer electionId,
Integer voterId)

Finds a vote by election and voter.

Parameters:

electionId - the election ID

voterId - the voter (user) ID

Returns:

the Vote, or null if not found

```
public Vote findByElectionAndVoter(Integer electionId, Integer voterId) { no usages  & Sugam-Khatiwada
    try {
        TypedQuery<Vote> query = em.createQuery(
            queryString: "SELECT v FROM Vote v WHERE v.election.id = :electionId AND v.voter.id = :voterId",
            Vote.class);
        query.setParameter( name: "electionId", electionId);
        query.setParameter( name: "voterId", voterId);
        List<Vote> results = query.getResultList();
        return results.isEmpty() ? null : results.get(0);
    } catch (Exception e) {
        throw new RuntimeException("Error finding vote for election " + electionId
            + " and voter " + voterId, e);
    }
}
```

Figure 97 findByElectionAndVoter code

ii. countByElection

public long countByElection(Integer electionId)

Counts the total votes in a specific election.

Parameters:

electionId - the election ID

Returns:

the count of votes

```
public long countByElection(Integer electionId) { 2 usages  Sugam-Khatiwada
    try {
        TypedQuery<Long> query = em.createQuery(
            queryString: "SELECT COUNT(v) FROM Vote v WHERE v.election.id = :electionId",
            Long.class);
        query.setParameter(name: "electionId", electionId);
        return query.getSingleResult();
    } catch (Exception e) {
        throw new RuntimeException("Error counting votes by election: " + electionId, e);
    }
}
```

Figure 98 countByElection code

iii. countByCandidate

public long countByCandidate(Integer candidateId)

Counts the votes for a specific candidate.

Parameters:

candidateId - the candidate ID

Returns:

the count of votes for the candidate

```
public long countByCandidate(Integer candidateId) { 1 usage  Sugam-Khatiwada
    try {
        TypedQuery<Long> query = em.createQuery(
            queryString: "SELECT COUNT(v) FROM Vote v WHERE v.candidate.id = :candidateId",
            Long.class);
        query.setParameter( name: "candidateId", candidateId);
        return query.getSingleResult();
    } catch (Exception e) {
        throw new RuntimeException("Error counting votes by candidate: " + candidateId, e);
    }
}
```

Figure 99 countByCandidate code

iv. hasVoted

public boolean hasVoted(Integer electionId,
Integer voterId)

Checks if a voter has already voted in a specific election.

Parameters:

electionId - the election ID

voterId - the voter (user) ID

Returns:

true if the voter has already voted, false otherwise

```
public boolean hasVoted(Integer electionId, Integer voterId) { 2 usages  Sugam-Khatiwada
    try {
        TypedQuery<Long> query = em.createQuery(
            queryString: "SELECT COUNT(v) FROM Vote v WHERE v.election.id = :electionId AND v.voter.id = :voterId",
            Long.class);
        query.setParameter( name: "electionId", electionId);
        query.setParameter( name: "voterId", voterId);
        return query.getSingleResult() > 0;
    } catch (Exception e) {
        throw new RuntimeException("Error checking if voter " + voterId
            + " has voted in election " + electionId, e);
    }
}
```

Figure 100 hasVoted code

v. findVotesByElection

```
public List<Vote> findVotesByElection(Integer electionId)
```

Finds all votes for a specific election.

Parameters:

electionId - the election ID

Returns:

a list of votes for the election

```
public List<Vote> findVotesByElection(Integer electionId) { no usages 2 Sugam-Khatriwada
    try {
        TypedQuery<Vote> query = em.createQuery(
            queryString: "SELECT v FROM Vote v WHERE v.election.id = :electionId ORDER BY v.votedAt DESC",
            Vote.class);
        query.setParameter( name: "electionId", electionId);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding votes by election: " + electionId, e);
    }
}
```

Figure 101 findVotesByElection code

vi. findVotesByCandidate

public List<Vote> findVotesByCandidate(Integer candidateId)

Finds all votes for a specific candidate in an election.

Parameters:

candidateId - the candidate ID

Returns:

a list of votes for the candidate

```
public List<Vote> findVotesByCandidate(Integer candidateId) { no usages 2 Sugam-Khatriwada
    try {
        TypedQuery<Vote> query = em.createQuery(
            queryString: "SELECT v FROM Vote v WHERE v.candidate.id = :candidateId ORDER BY v.votedAt DESC",
            Vote.class);
        query.setParameter( name: "candidateId", candidateId);
        return query.getResultList();
    } catch (Exception e) {
        throw new RuntimeException("Error finding votes by candidate: " + candidateId, e);
    }
}
```

Figure 102 findVotesByCandidate code

5.1.2 Utility Classes

a. AppConstants

AppConstants
+SESSION_USER String
+SESSION_USER_ID String
+SESSION_ROLE String
+SESSION_EMAIL String
+ROLE_ADMIN String
+ROLE_VOTER String
+ROLE_CANDIDATE String
+STATUS_PENDING String
+STATUS_APPROVED String
+ELECTION_STATUS_ACTIVE String
+ELECTION_STATUS_COMPLETED String
+MSG_VOTE_CAST String
+MSG_ALREADY_VOTED String
+URL_LOGIN String
+URL_ADMIN_DASHBOARD String
+PUBLIC_PATHS String[]

Figure 103 AppConstants

b. PasswordUtil

PasswordUtil
-String SALT
+hashPassword(plainText) : String
+verifyPassword(plainText, hashedPassword) : boolean
-bytesToHex(bytes) : String

*Figure 104 PasswordUtil***i. hashPassword**

public static String hashPassword(String plainText)

Hashes a plain text password using SHA-256 with salt.

Parameters:

plainText - the plain text password

Returns:

the hashed password as a hexadecimal string

```
public static String hashPassword(String plainText) { 2 usages  👤 Sugam-Khatiwada
    try {
        MessageDigest digest = MessageDigest.getInstance( algorithm: "SHA-256");
        String saltedPassword = plainText + SALT;
        byte[] hashBytes = digest.digest(saltedPassword.getBytes(StandardCharsets.UTF_8));
        return bytesToHex(hashBytes);
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException("SHA-256 algorithm not found", e);
    }
}
```

Figure 105 hashPassword code

ii. verifyPassword

public static boolean verifyPassword(String plainText,
String hashedPassword)

Verifies a plain text password against a hashed password.

Parameters:

plainText - the plain text password to verify

hashedPassword - the stored hashed password

Returns:

true if the password matches, false otherwise

```
public static boolean verifyPassword(String plainText, String hashedPassword) { 1 usage  Sugam-Khatiwada
    String newHash = hashPassword(plainText);
    return newHash.equalsIgnoreCase(hashedPassword);
}
```

Figure 106 verifyPassword code

5.1.3 Controller Class: AdminCandidateServlet

AdminCandidateServlet
-CandidateService candidateService
-CandidateApplicationDAO candidateApplicationDAO
+doGet(req, res) : void
+doPost(req, res) : void

Figure 107 Controller Class: AdminCandidateServlet code

a. Methods

i. doGet

protected void doGet(jakarta.servlet.http.HttpServletRequest request,
jakarta.servlet.http.HttpServletResponse response)
throws jakarta.servlet.ServletException,
IOException

Overrides:

doGet in class jakarta.servlet.http.HttpServlet

Throws:

jakarta.servlet.ServletException

IOException

```
@Override  @ Sugam-Khatriwada
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    try {
        List<CandidateApplication> applications = candidateApplicationDAO.findAll();
        request.setAttribute( name: "applications", applications);
        request.getRequestDispatcher( path: "/admin/candidates.jsp").forward(request, response);
    } catch (Exception e) {
        request.setAttribute( name: "error", o: "Error loading candidate applications: " + e.getMessage());
        request.getRequestDispatcher( path: "/admin/candidates.jsp").forward(request, response);
    }
}
```

Figure 108 doGet code

ii. doPost

protected void doPost(jakarta.servlet.http.HttpServletRequest request,
jakarta.servlet.http.HttpServletResponse response)
throws jakarta.servlet.ServletException,
IOException

Overrides:

doPost in class jakarta.servlet.http.HttpServlet

Throws:

jakarta.servlet.ServletException

IOException

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    String action = request.getParameter("action");
    String applicationIdStr = request.getParameter("applicationId");

    try {
        if (action != null && applicationIdStr != null) {
            int applicationId = Integer.parseInt(applicationIdStr);

            switch (action) {
                case "approve":
                    candidateService.approveApplication(applicationId);
                    request.getSession().setAttribute("success",
                        "Candidate application has been approved successfully.");
                    break;

                case "reject":
                    candidateService.rejectApplication(applicationId);
                    request.getSession().setAttribute("success",
                        "Candidate application has been rejected.");
                    break;

                default:
                    request.getSession().setAttribute("error", "Invalid action specified.");
                    break;
            }
        } else {
            request.getSession().setAttribute("error", "Missing action or application ID.");
        }
    } catch (NumberFormatException e) {
        request.getSession().setAttribute("error", "Invalid application ID format.");
    } catch (Exception e) {
        request.getSession().setAttribute("error", "Error performing action: " + e.getMessage());
    }

    response.sendRedirect(request.getContextPath() + "/admin/candidates");
}
```

Figure 109 doPost code

6 Test Cases

6.1 Admin Login

Table 14 Admin Login

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Auth (Admin)	Admin login with valid credential	Enter valid email and password than, click on login	Admin should be successfully logged in and redirected to dashboard.	After entering valid credentials admin is successfully logged in and redirected to admin dashboard.	Pass

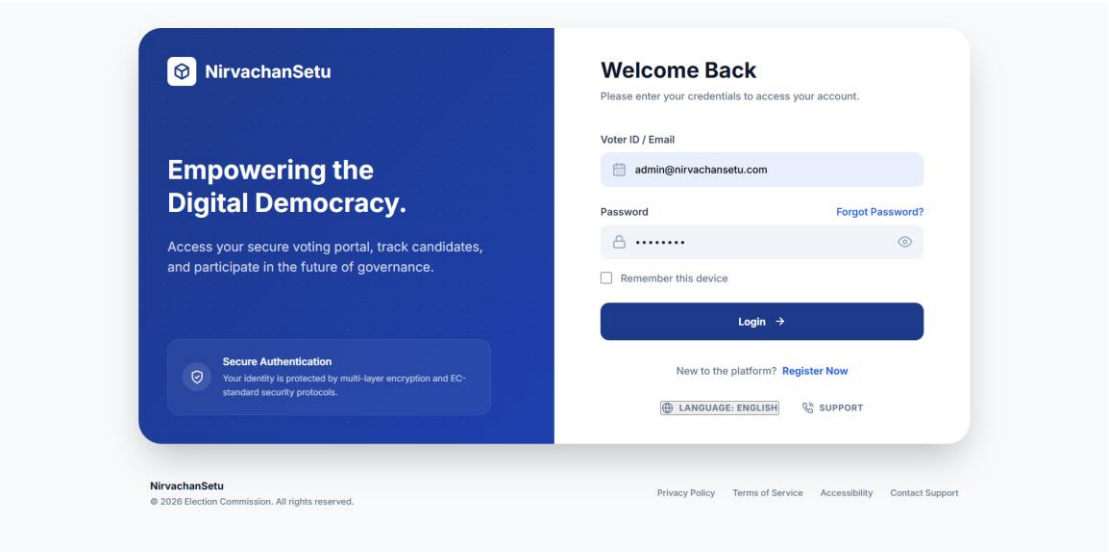


Figure 110 Login Page Admin

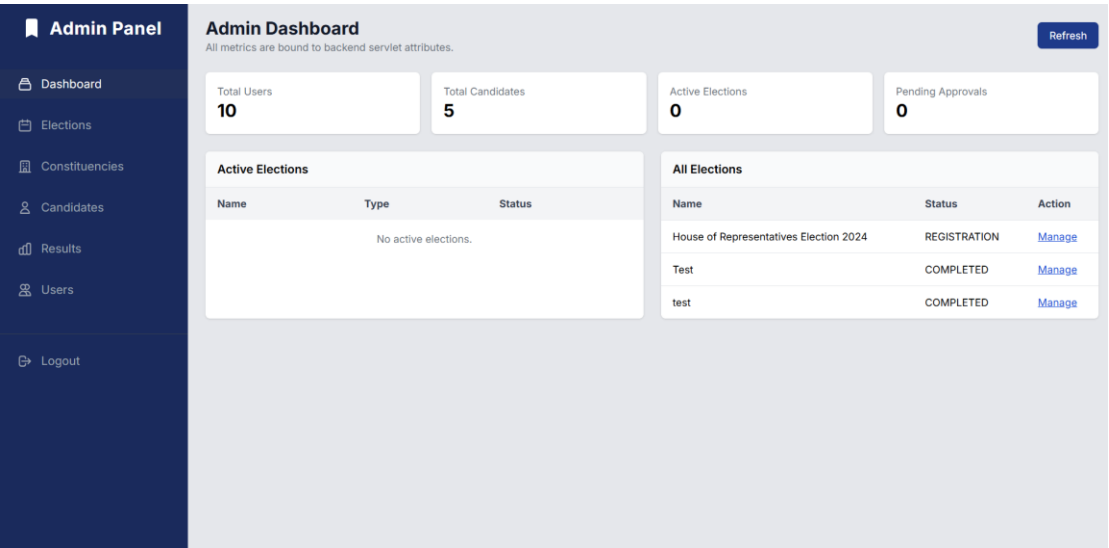


Figure 111 Admin Dashboard

6.2 Dashboard Refresh Functionality

Table 15: Dashboard refresh function

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Dash	To verify the functionality of the refresh button in dashboard .	First click on Refresh button in dashboard	The data in the dashboard data should reload.	After refreshing, the dashboard displayed the latests data successfully.	Pass

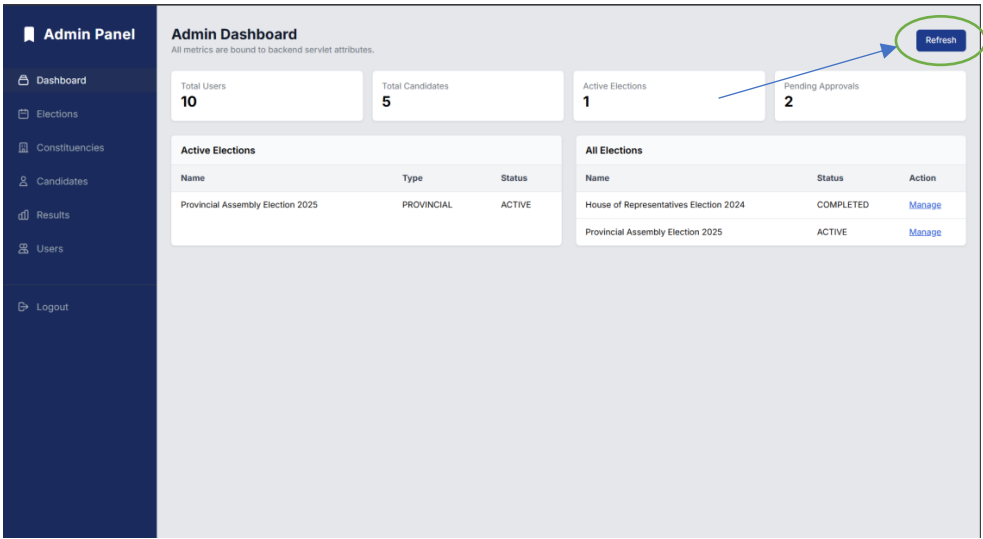


Figure 112: dashboard before refreshing

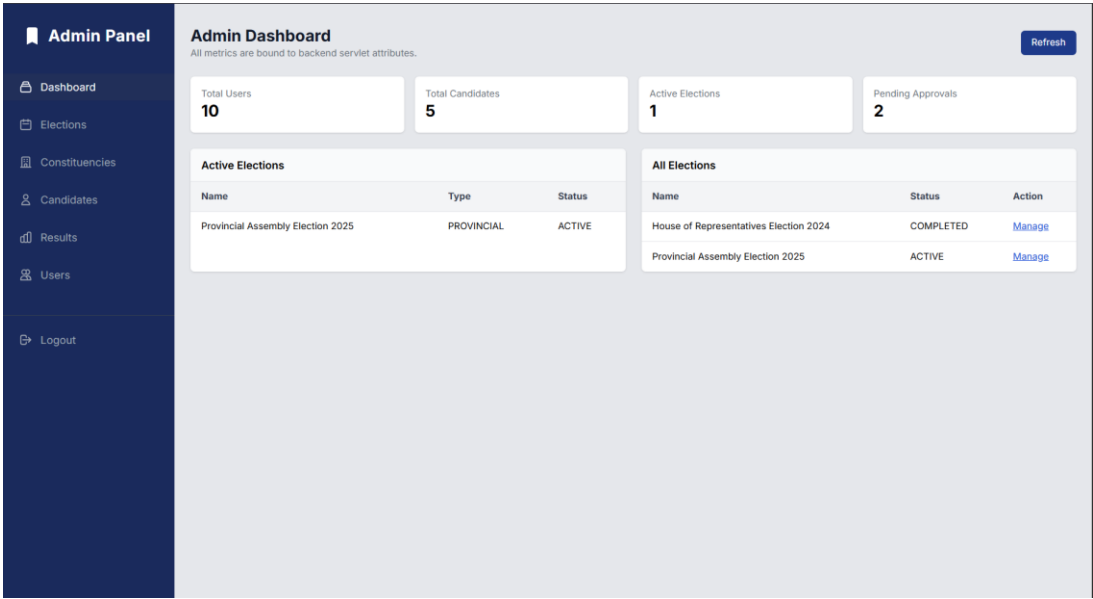


Figure 113: dashboard after refreshing

6.3 Add New Election

Table 16: add new election

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Elec	To verify the admin can create a new election	Initially, enter election details (valid ones) and then click Create/Save Election button	New election should be created successfully and displayed in election list	Election was created successfully and displayed in the elections table	Pass

Admin Panel

Dashboard

Elections

Constituencies

Candidates

Results

Users

Logout

Election Management

Configure, schedule, and monitor nationwide and local elections.

Refresh List

Create New Election

ELECTION NAME

General Election 2027

ELECTION TYPE

PROVINCIAL

STATUS

REGISTRATION

START DATE & TIME

05/03/2026 08:00 AM

END DATE & TIME

05/16/2026 10:00 AM

SPECIFIC CONSTITUENCY

All Constituencies (National)

ELECTION DESCRIPTION

The General Election 2027 is a provincial-level election conducted to allow eligible citizens to vote for their representatives across all constituencies. This election aims to ensure a fair, transparent, and secure democratic process through the online voting system. Registered candidates and verified voters will participate within the scheduled election period.

Launch New Election

All Registered Elections

ELECTION NAME	SCOPE	SCHEDULE	STATUS	ACTIONS
House of Representatives Election 2024 HOR	National	Starts: 2024-05-12T13:45 Ends: 2024-05-12T22:45	COMPLETED	Edit
Provincial Assembly Election 2025	National	Starts: 2025-06-15T13:45 Ends: 2025-06-15T22:45	ACTIVE	Edit

Figure 114: creating new election

Admin Panel

Dashboard

Elections

Constituencies

Candidates

Results

Users

Logout

Election Management

Configure, schedule, and monitor nationwide and local elections.

Refresh List

Create New Election

ELECTION NAME

e.g. General Election 2025

ELECTION TYPE

Select Type

STATUS

DRAFT

START DATE & TIME

mm/dd/yyyy --:-- --

END DATE & TIME

mm/dd/yyyy --:-- --

SPECIFIC CONSTITUENCY

All Constituencies (National)

ELECTION DESCRIPTION

Provide details about the election objectives, rules, and scope...

Launch New Election

All Registered Elections

ELECTION NAME	SCOPE	SCHEDULE	STATUS	ACTIONS
House of Representatives Election 2024 HOR	National	Starts: 2024-05-12T13:45 Ends: 2024-05-12T22:45	COMPLETED	Edit
Provincial Assembly Election 2025 PROVINCIAL	National	Starts: 2025-06-15T13:45 Ends: 2025-06-15T22:45	ACTIVE	Edit
General Election 2026 LOCAL	National	Starts: 2026-05-10T10:27 Ends: 2026-05-11T10:27	ACTIVE	Edit
General Election 2027 PROVINCIAL	National	Starts: 2026-05-03T08:00 Ends: 2026-05-16T10:00	REGISTRATION	Edit

Figure 115: created new election successfully

6.4 Approve Candidate Application

Table 17: approve candidate application

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Cand	To verify admin can approve candidate application successfully	First click the Approve button in the dashboard for application which are pending	The candidate application should successfully change to APPROVED status	The candidate application was approved and the status was updated successfully	Pass

The screenshot shows the 'Admin Panel' with a sidebar containing links to Dashboard, Elections, Constituencies, Candidates, Results, Users, and Logout. The main content area is titled 'User Management' and includes a 'Refresh' button. A green notification bar states 'Election created successfully.' Below this, summary statistics show 'Total Users: 10', 'Pending: 2', and 'Approved: 8'. A table lists users with columns for Name, Email, Role, Status, Constituency, and Actions. The 'Actions' column contains buttons for 'Reject', 'Approve', and 'Suspend'. The user 'Anita Devi' is in a 'PENDING' status, and her 'Approve' button is highlighted with a red circle and an arrow pointing to it from the right.

Name	Email	Role	Status	Constituency	Actions
System Administrator	admin@nirvachansetu.com	ADMIN	APPROVED	-	Reject Suspend
Rajesh Sharma	rajesh@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Sita Kumari	sita@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Hari Bahadur	hari@email.com	VOTER	APPROVED	Kathmandu-2	Reject Suspend
Anita Devi	anita@email.com	VOTER	PENDING	Kathmandu-3	Approve Reject Suspend
Ram Prasad	ram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Dr. Anjali Verma	anjali@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Prakash Hegde	prakash@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Vikram Aditya	vikram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Savitri Devi	savitri@email.com	VOTER	PENDING	Kathmandu-2	Approve Reject Suspend

Figure 116: approve option

Admin Panel

User Management
Live voter/admin/candidate users from backend.

Refresh

User has been approved successfully.

Total Users: 10, Pending: 1, Approved: 9

Name	Email	Role	Status	Constituency	Actions
System Administrator	admin@nirvachansetu.com	ADMIN	APPROVED	-	Reject Suspend
Rajesh Sharma	rajesh@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Sita Kumari	sita@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Hari Bahadur	hari@email.com	VOTER	APPROVED	Kathmandu-2	Reject Suspend
Anita Devi	anita@email.com	VOTER	APPROVED	Kathmandu-3	Reject Suspend
Ram Prasad	ram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Dr. Anjali Verma	anjali@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Prakash Hegde	prakash@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Vikram Aditya	vikram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Savitri Devi	savitri@email.com	VOTER	PENDING	Kathmandu-2	Approve Reject Suspend

Figure 117: approved successfully

6.5 Reject Candidate Application

Table 18: reject candidate application

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Cand	To verify admin are able to reject the application of the candidate	First click the Reject button from the dashboard for application which are rejected	The candidate application should change to REJECTED status correctly	The candidate application was rejected and status was also updated correctly and successfully	Pass

Admin Panel

- Dashboard
- Elections
- Constituencies
- Candidates
- Results
- Users**
- Logout

User Management
Live voter/admin/candidate users from backend.

Refresh

User has been approved successfully.

Total Users: 10, Pending: 1, Approved: 9

Name	Email	Role	Status	Constituency	Actions
System Administrator	admin@nirvachansetu.com	ADMIN	APPROVED	-	Reject Suspend
Rajesh Sharma	rajesh@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Sita Kumari	sita@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Hari Bahadur	hari@email.com	VOTER	APPROVED	Kathmandu-2	Reject Suspend
Anita Devi	anita@email.com	VOTER	APPROVED	Kathmandu-3	Reject Suspend
Ram Prasad	ram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Dr. Anjali Verma	anjali@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Prakash Hegde	prakash@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Vikram Aditya	vikram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Savitri Devi	savitri@email.com	VOTER	PENDING	Kathmandu-2	Approve Reject Suspend

Figure 118: reject option

Admin Panel

- Dashboard
- Elections
- Constituencies
- Candidates
- Results
- Users**
- Logout

User Management
Live voter/admin/candidate users from backend.

Refresh

User has been rejected.

Total Users: 10, Pending: 0, Approved: 9

Name	Email	Role	Status	Constituency	Actions
System Administrator	admin@nirvachansetu.com	ADMIN	APPROVED	-	Reject Suspend
Rajesh Sharma	rajesh@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Sita Kumari	sita@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Hari Bahadur	hari@email.com	VOTER	APPROVED	Kathmandu-2	Reject Suspend
Anita Devi	anita@email.com	VOTER	APPROVED	Kathmandu-3	Reject Suspend
Ram Prasad	ram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Dr. Anjali Verma	anjali@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Prakash Hegde	prakash@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Vikram Aditya	vikram@email.com	VOTER	APPROVED	Kathmandu-1	Reject Suspend
Savitri Devi	savitri@email.com	VOTER	REJECTED	Kathmandu-2	Approve Reject Suspend

Figure 119: rejected successfully

6.6 Add New Constituency

Table 19: add new constituency

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Const	To verify admin are able to add new constituencies	First enter the details of constituency details(valid ones) and then click save Constituency	The new constituency should be added successfully in the dashboard	The constituency was added and displayed correctly in the directory successfully	Pass

The screenshot displays the 'Admin Panel' interface. On the left is a sidebar with navigation links: Dashboard, Elections, Constituencies (selected), Candidates, Results, Users, and Logout. The main content area is titled 'Constituencies' with a subtitle 'Manage geographical voting regions and voter population data.' and a 'Refresh Data' button. Below this is the 'Add New Constituency' form, which includes input fields for 'CONSTITUENCY NAME' (Sunsari-02), 'DISTRICT' (Sunsari), 'PROVINCE' (Koshi), and 'TOTAL VOTERS' (100000). A 'DESCRIPTION' field contains the text 'Constituency covering Sunsari'. A 'Save Constituency' button is at the bottom right of the form. Below the form is a 'Constituency Directory' table with the following data:

REGION	PROVINCE	VOTER BASE	DESCRIPTION
Kathmandu-1 Kathmandu District	Bagmati Province	125000	Constituency covering central Kathmandu
Kathmandu-2 Kathmandu District	Bagmati Province	118000	Constituency covering eastern Kathmandu
Kathmandu-3 Kathmandu District	Bagmati Province	132000	Constituency covering northern Kathmandu

Figure 120: adding new constituency

Admin Panel

Dashboard

Elections

Constituencies

Candidates

Results

Users

Logout

Save Constituency

Constituency Directory

REGION	PROVINCE	VOTER BASE	DESCRIPTION
Kathmandu-1 Kathmandu District	Bagmati Province	125000	Constituency covering central Kathmandu
Kathmandu-2 Kathmandu District	Bagmati Province	118000	Constituency covering eastern Kathmandu
Kathmandu-3 Kathmandu District	Bagmati Province	132000	Constituency covering northern Kathmandu
Lalitpur-1 Lalitpur District	Bagmati Province	95000	Constituency covering central Lalitpur
Bhaktapur-1 Bhaktapur District	Bagmati Province	88000	Constituency covering Bhaktapur district
Pokhara-1 Kaski District	Gandaki Province	110000	Constituency covering Pokhara metropolitan
Biratnagar-1 Morang District	Koshi Province	105000	Constituency covering Biratnagar area
Chitwan-1 Chitwan District	Bagmati Province	97000	Constituency covering Chitwan district
Sunsari-02 Sunsari District	Koshi	100000	Constituency covering Sunsari

Figure 121: new constituency added successfully

6.7 Constituency Required Field Validation

Table 20: constituency required field validation

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Const	To verify the required field validation in constituency form works properly	Initially, leave the required fields empty and click Save	System should prevent submission and show validation message	Form submission was prevented and validation worked correctly	Pass

Admin Panel

- Dashboard
- Elections
- Constituencies**
- Candidates
- Results
- Users
- Logout

Constituencies

Manage geographical voting regions and voter population data.

[Refresh Data](#)

Add New Constituency

CONSTITUENCY NAME:

DISTRICT:

PROVINCE:

TOTAL VOTERS:

DESCRIPTION:

[Save Constituency](#)

Constituency Directory

REGION	PROVINCE	VOTER BASE	DESCRIPTION
Kathmandu-1 Kathmandu District	Bagmati Province	125000	Constituency covering central Kathmandu
Kathmandu-2 Kathmandu District	Bagmati Province	118000	Constituency covering eastern Kathmandu
Kathmandu-3 Kathmandu District	Bagmati Province	132000	Constituency covering northern Kathmandu

Figure 122: empty field on purpose

Admin Panel

- Dashboard
- Elections
- Constituencies**
- Candidates
- Results
- Users
- Logout

Constituencies

Manage geographical voting regions and voter population data.

[Refresh Data](#)

Add New Constituency

CONSTITUENCY NAME:

DISTRICT:

PROVINCE:

TOTAL VOTERS:

DESCRIPTION:

[Save Constituency](#)

Constituency Directory

REGION	PROVINCE	VOTER BASE	DESCRIPTION
Kathmandu-1 Kathmandu District	Bagmati Province	125000	Constituency covering central Kathmandu
Kathmandu-2 Kathmandu District	Bagmati Province	118000	Constituency covering eastern Kathmandu
Kathmandu-3 Kathmandu District	Bagmati Province	132000	Constituency covering northern Kathmandu

Figure 123: form submission prevented

6.8 View Election Results

Table 21: view election result

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Result	To verify all the completed election results displayed on the dashboard	Simple, open Results page	The System should display all the completed election results and vote counts correctly.	The election results and vote counts were displayed successfully	Pass

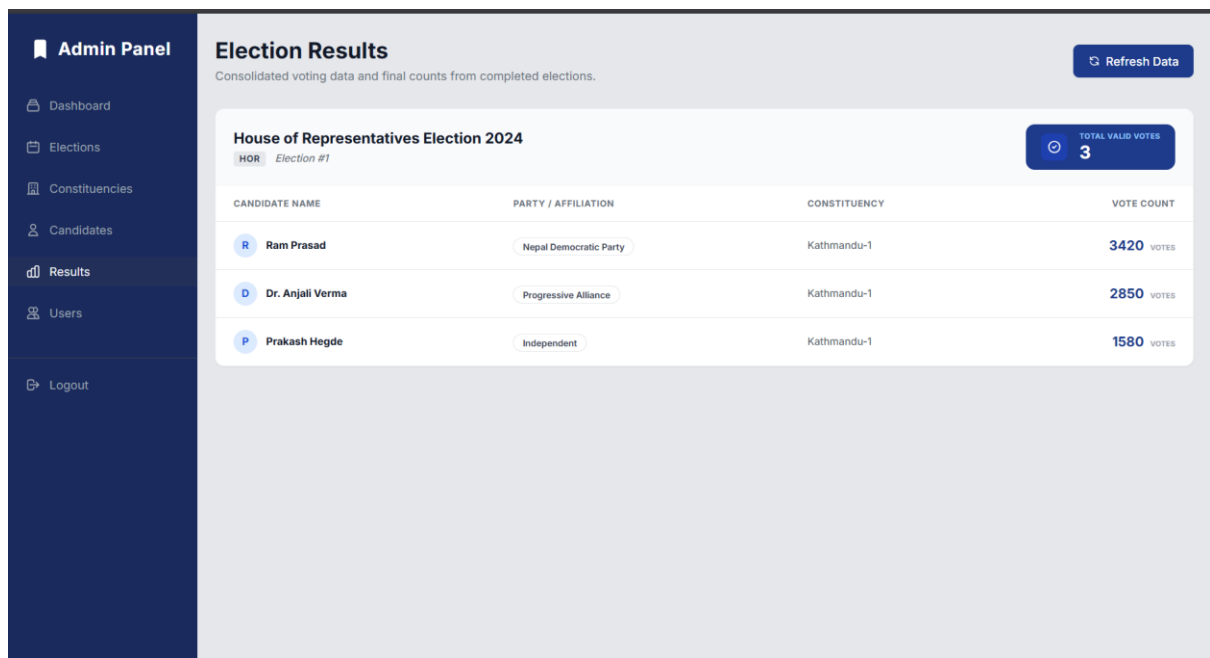


Figure 124: election result page

6.9 Voter Login

Table 22 Voter Login

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Auth (Voter)	Voter login with valid credential	Enter valid email and password than, click on login	Voter should be successfully logged in and redirected to dashboard.	After entering valid credentials Voter is successfully logged in and redirected to Voter dashboard.	Pass

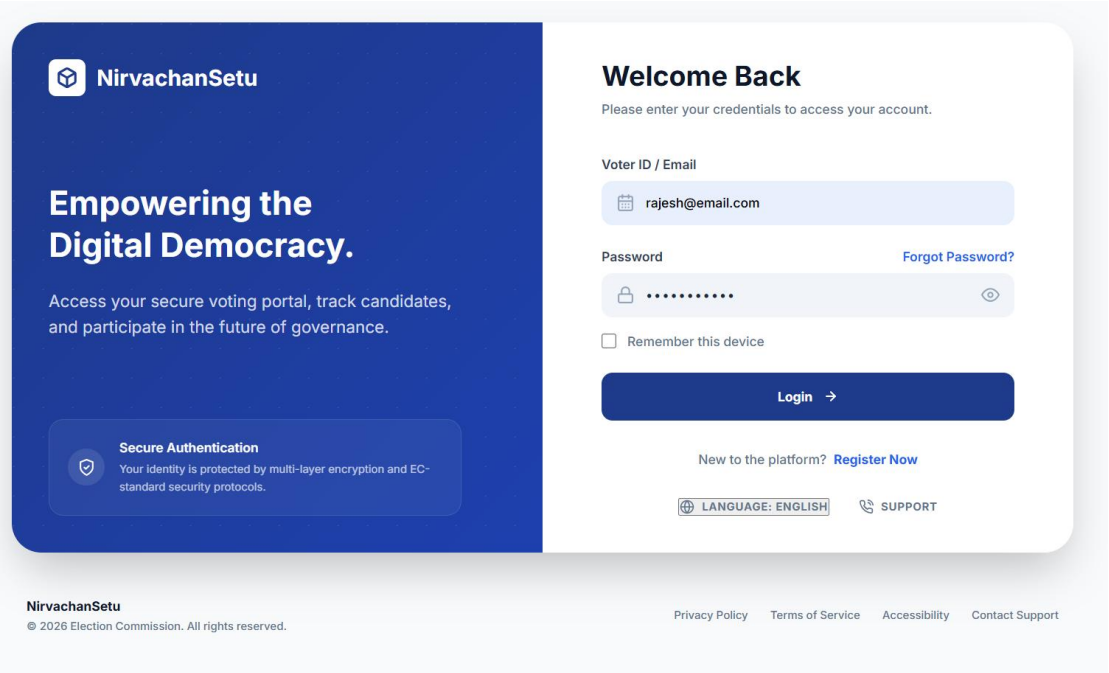
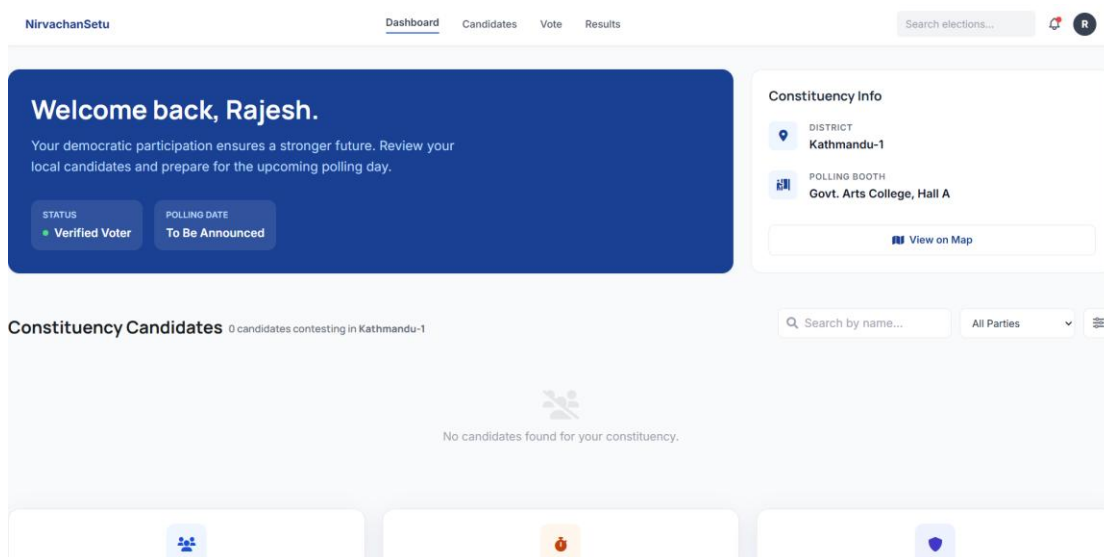


Figure 125 Login Page Voter

*Figure 126 Voter Dashboard*

6.10 Candidate Login

Table 23 Candidate Login

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Auth (Candidate)	Candidate login with valid credential	Enter valid email and password than, click on login	Candidate should be successfully logged in and redirected to dashboard.	After entering valid credentials Candidate is successfully logged in and redirected to Candidate dashboard.	Pass

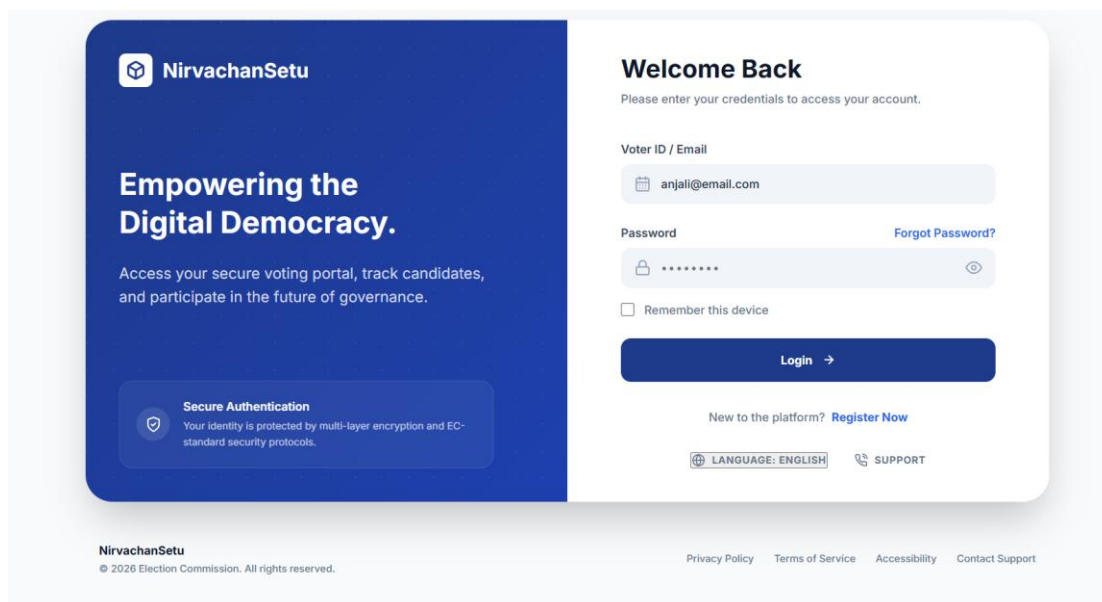
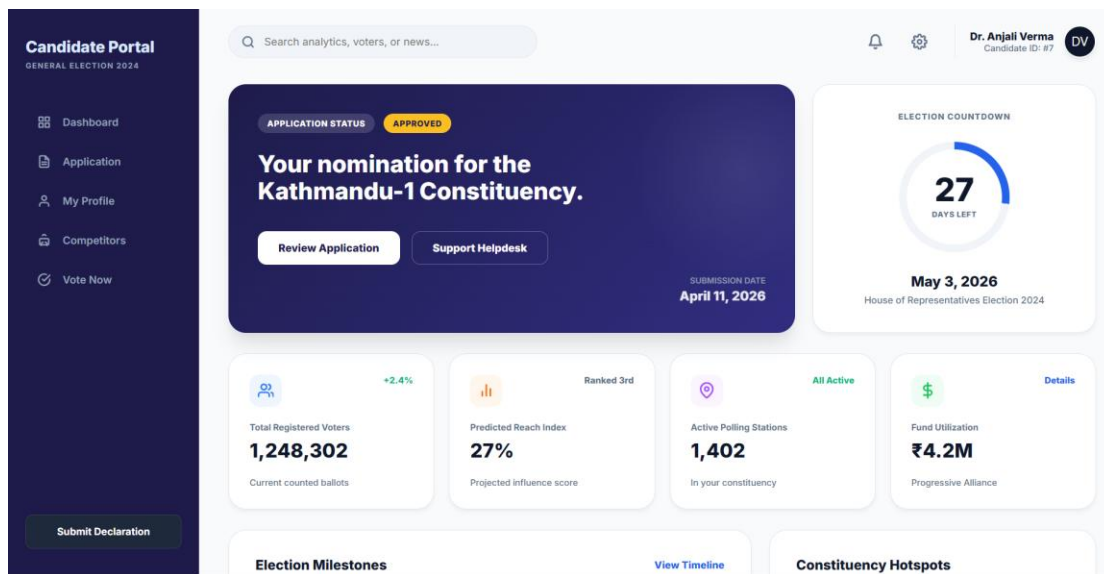


Figure 127 Login Page Candidate

*Figure 128 Candidate Dashboard*

6.11 Election Creation

Table 24 Election Creation

Test ID	Description	Steps	Expected Result	Actual Result	Status
TC_Election	Admin creates a new Election	Enter valid details than, click on launch new election	New Election should be created and listed in all election records	After entering valid details the new election is created and listed in election records	Pass

The screenshot shows the 'Election Management' interface. On the left is an 'Admin Panel' sidebar with links to Dashboard, Elections, Constituencies, Candidates, Results, Users, and Logout. The main area is titled 'Election Management' with a subtitle 'Configure, schedule, and monitor nationwide and local elections.' and a 'Refresh List' button. The 'Create New Election' form contains the following fields:

- ELECTION NAME:** House of Representatives Election 2026
- ELECTION TYPE:** HOR
- STATUS:** REGISTRATION
- START DATE & TIME:** 05/05/2026 02:18 PM
- END DATE & TIME:** 05/06/2026 02:20 PM
- SPECIFIC CONSTITUENCY:** All Constituencies (National)
- ELECTION DESCRIPTION:** This is HoR Election

A 'Launch New Election' button is at the bottom of the form.

Figure 129 Adding Election

The screenshot shows a table titled 'All Registered Elections' with the following data:

ELECTION NAME	SCOPE	SCHEDULE	STATUS	ACTIONS
House of Representatives Election 2024 HOR	National	Starts: 2026-05-03T16:05 Ends: 2026-05-03T16:05	CANCELLED	Edit
Test HOR	National	Starts: 2026-04-11T13:08 Ends: 2026-04-11T14:02	COMPLETED	Edit
test HOR	National	Starts: 2026-04-11T13:03 Ends: 2026-04-11T13:10	COMPLETED	Edit
House of Representatives Election 2026 HOR	National	Starts: 2026-05-05T14:18 Ends: 2026-05-06T14:20	REGISTRATION	Edit

Figure 130 Election Record

6.12 Cast vote

Table 25 Cast vote

Test ID	Description	Step	Expected- result	Actual- Result	status
Vote(Candidate)	User can vote the candidate	Login into NirvachanSetu and click on vote navbar. Voter page will apper and many candidates are available only one candidates can be vote by one user.	User will see tick icon and can't vote again	User will see tick icon.	pass

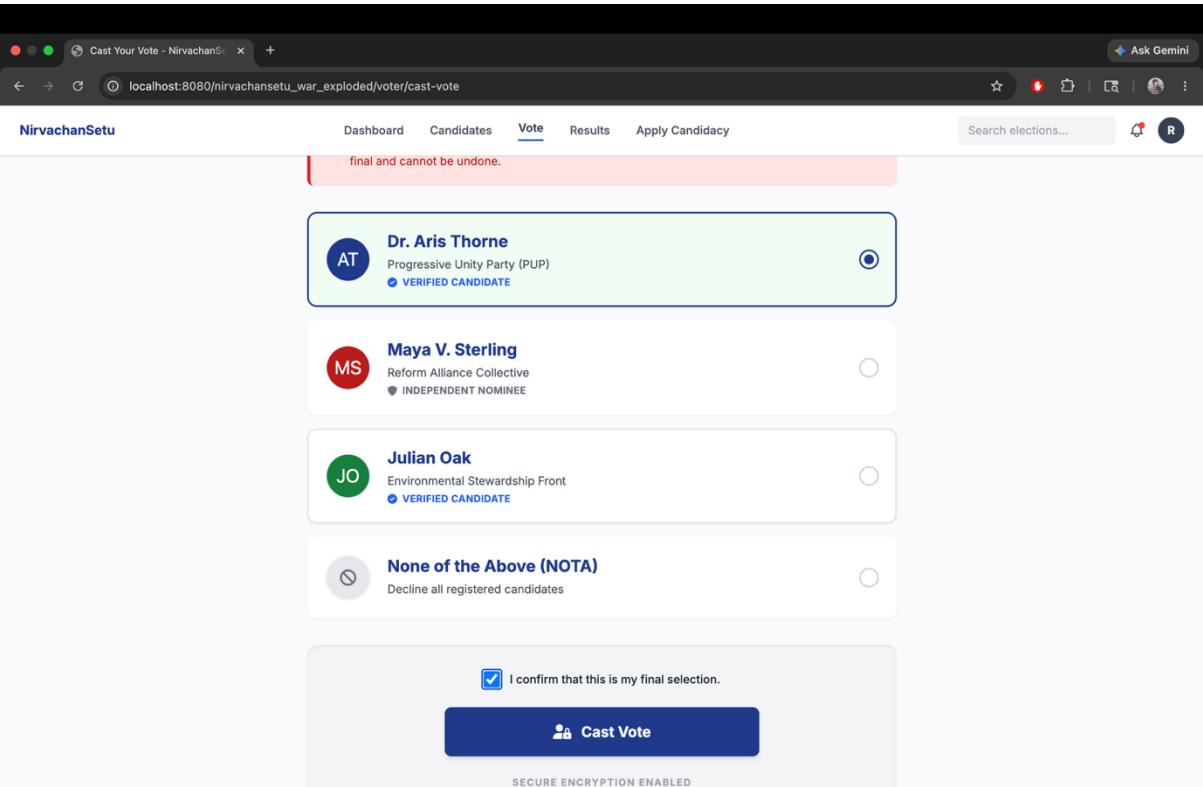


Figure 131 cast vote page

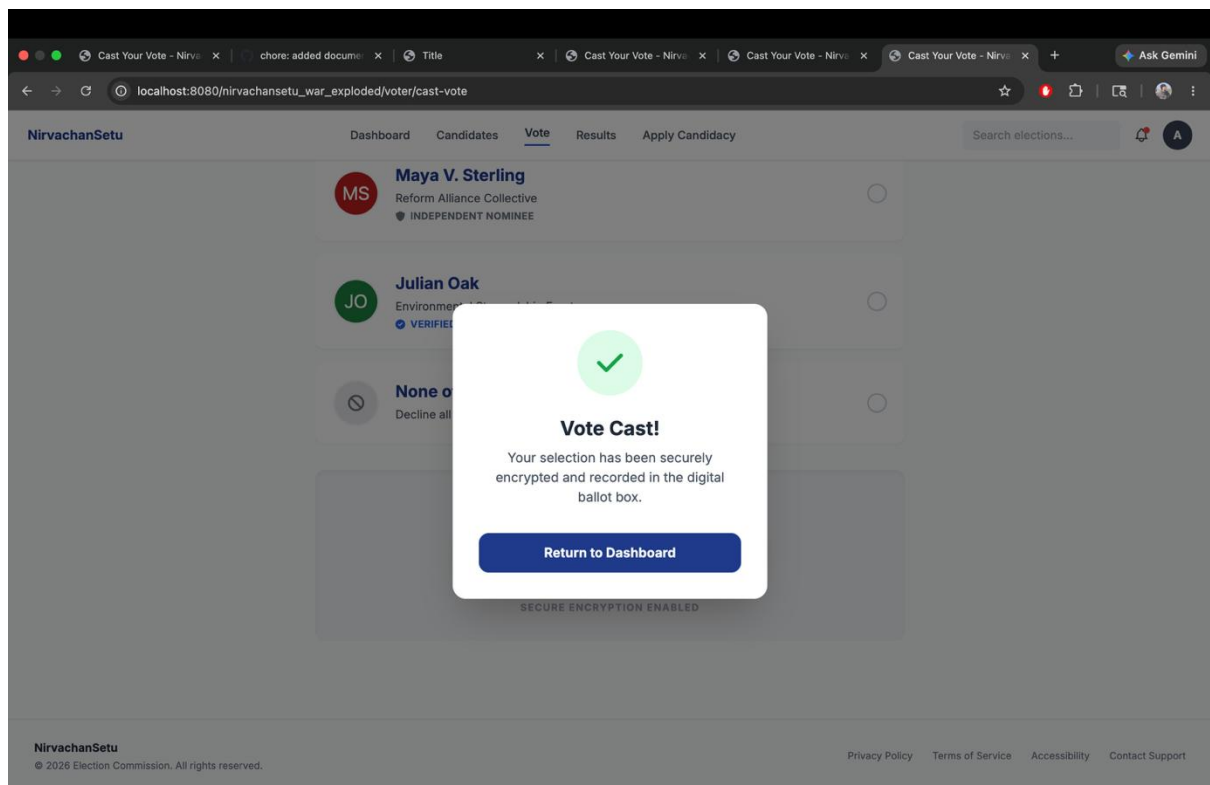


Figure 132 vote cast done

6.13 apply for candidacy*Table 26 Apply for Candidacy*

Test ID	Description	Step	Expected- result	Actual- Result	status
Vote(apply-candidacy)	User can apply for candidacy and take part in election .	After login as user in navbar apply0 candidacy an is seen click it. After click add remaining details to submit nomination request. And click in submit nomination request.	Tick icon will appare after submitting nomination request.	User will see tick icon Nomination Submitted.	pass


NirvachanSetu

Error casting vote: Error casting vote: Invalid candidate for this election.

X CANCEL APPLICATION

Candidate Nomination

Please provide accurate information for your official candidacy. Your nomination will be reviewed by the Election Commission.



VERIFIED PORTAL ACCESS

REQUIRED FILES

- Government Issued ID (PDF)
- Nomination Certificate
- Proof of Party Affiliation

Identity Details

FULL NAME

PARTY AFFILIATION

CONSTITUENCY SELECTION

ELECTION EVENT

Document Verification



ID Proof
Aadhaar, Voter ID, or Passport (PDF/JPG)

BROWSE FILES



Nomination Paper
Signed Form (A / B) (PDF)

BROWSE FILES

CAMPAIGN MANIFESTO

Briefly describe your vision and promises...

☐ I hereby declare that the information provided is true to the best of my knowledge and I understand that any discrepancy may lead to immediate disqualification as per EC Protocol V4.2.


Submit Nomination Request

SECURE TRANSMISSION ENABLED - EC PROTOCOL V4.2 - END-TO-END ENCRYPTED

© 2024 NIRVACHANSETU
 ELECTION COMMISSION DIGITAL INFRASTRUCTURE

Figure 133 apply-candidacy page

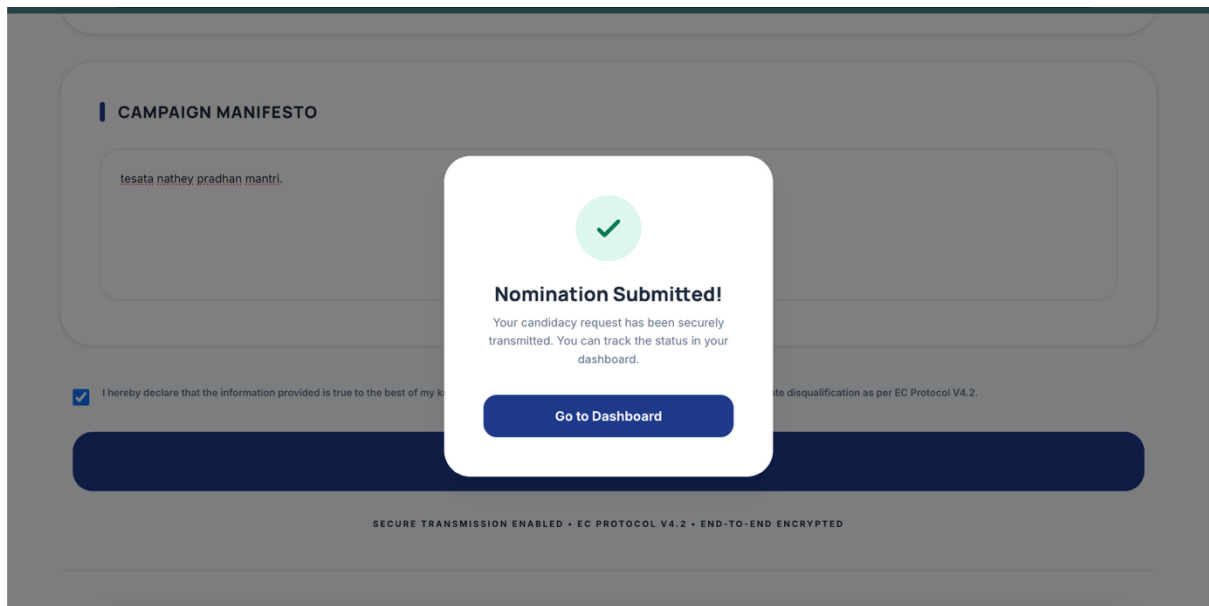


Figure 134 Nomination Submitted done.

6.14 view candidates

Table 27 View candidates

Test ID	Description	Step	Expected- result	Actual- Result	status
View candidates	User can view candidates that are standing in election.	Login into NirvachanSetu and click on candidate in navbar. Candidate page will appear and many candidates are available and clicking on candidate can read about candidate.	User will view the candidate page and candidate standing in election.	User will see all candidate that are standing in election.	pass

NirvachanSetu Dashboard **Candidates** Vote Results Apply Candidacy Search elections...

Electoral Candidates

Browse and evaluate candidates participating in the upcoming general assembly. Informed voting starts with understanding every manifesto.

CONSTITUENCY: All Constituencies PARTY: All Parties

No Candidates Found
There are no candidates registered matching your criteria.

< 1 2 3 ... >

NirvachanSetu
© 2026 Election Commission. All rights reserved. Privacy Policy Terms of Service Accessibility Contact Support

Figure 135 Candidates page

6.15 view Results

Table 28 View Results

Test ID	Description	Step	Expected- result	Actual- Result	status
View Result s	User can view Result of candidates how much vote they get in live.	Login into NirvachanSetu and click on result s in navbar. Result page will apper and all candidates vote count can been seen.	User will view the live vote counting in live time in results page .	User will see all candidate vote counting in live.	pass

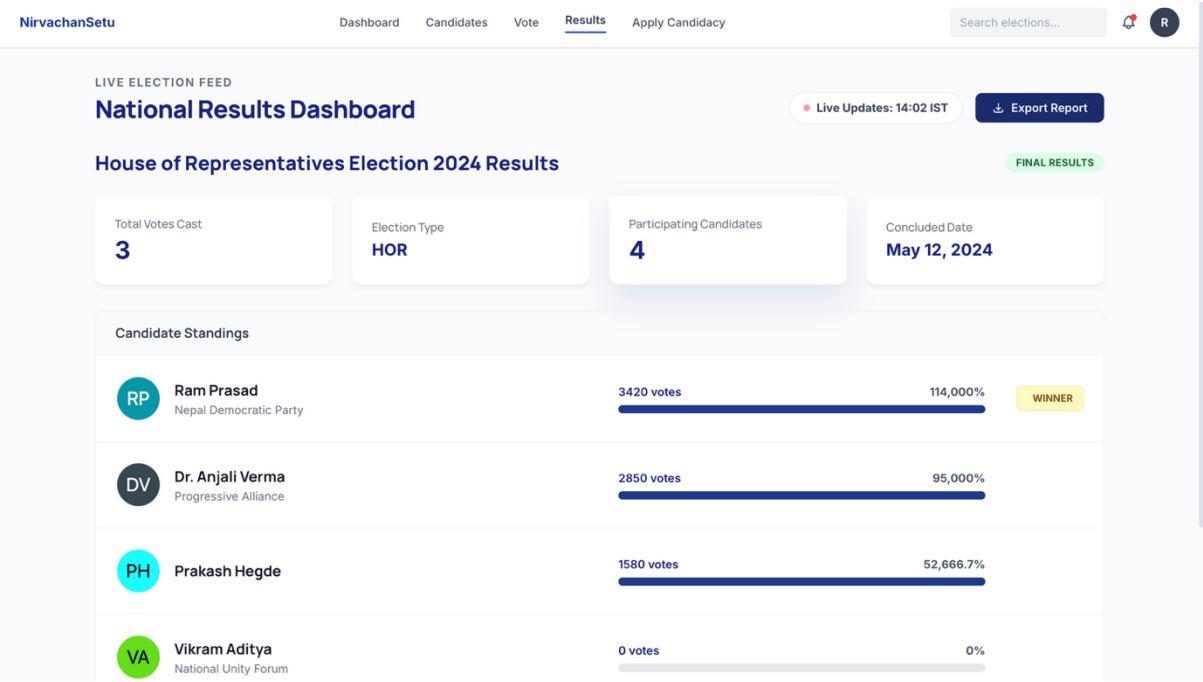


Figure 136 Results page

6.16 Register as user

Table 29 Register as User

Test ID	Description	Step	Expected- result	Actual- Result	status
Register as user	Register as user by providing valid data to use Nirvachansetu.	Fill all details on register and wait to approve. After the approved user can login and use Nirvachansetu.	After filling all details and login it and user dashboard can be seen.	After register login it and user dashboard.	pass

NirvachanSetu [Support](#)

Join the Digital Bastion

Create your secure identity to participate in the democratic process. Your data is protected by sovereign-grade encryption.

VERIFIED
Official Election Portal
SECURE

Create Account

Please provide your official details to register as a voter.

FULL NAME:

EMAIL ADDRESS:

PASSWORD:

CITIZENSHIP NUMBER:

CONSTITUENCY:

LEGAL IDENTIFICATION PROOF

PDF, PNG or JPG (max. 10MB)

Figure 137 Register form

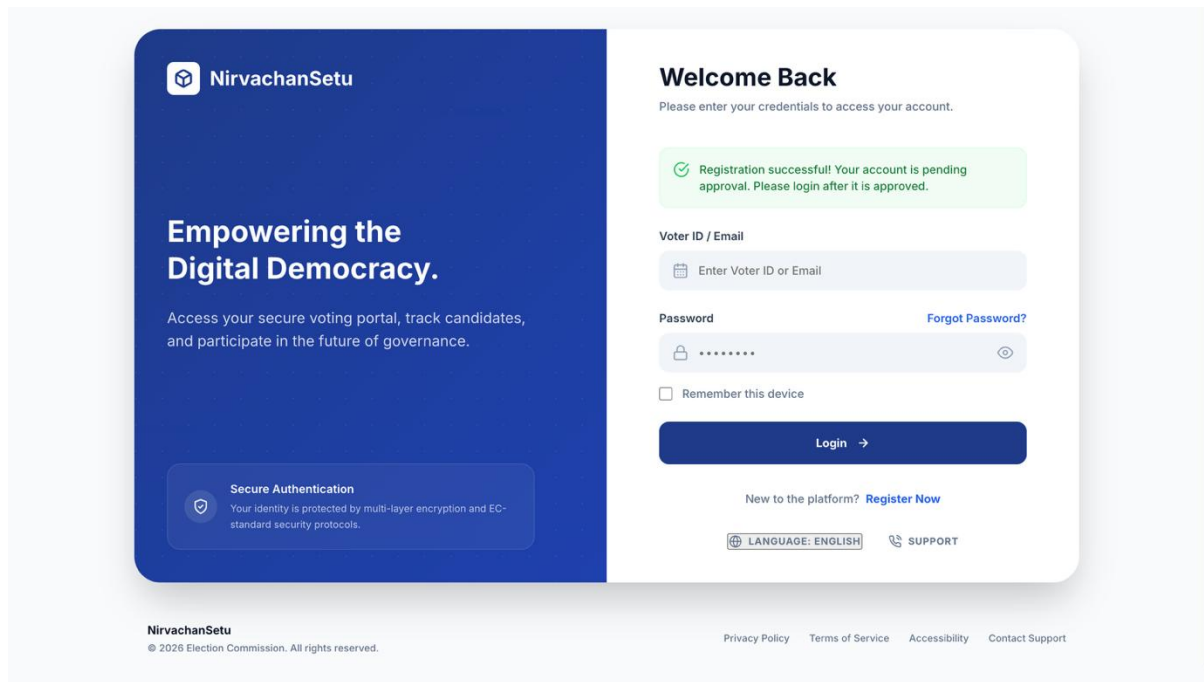


Figure 138 login after register

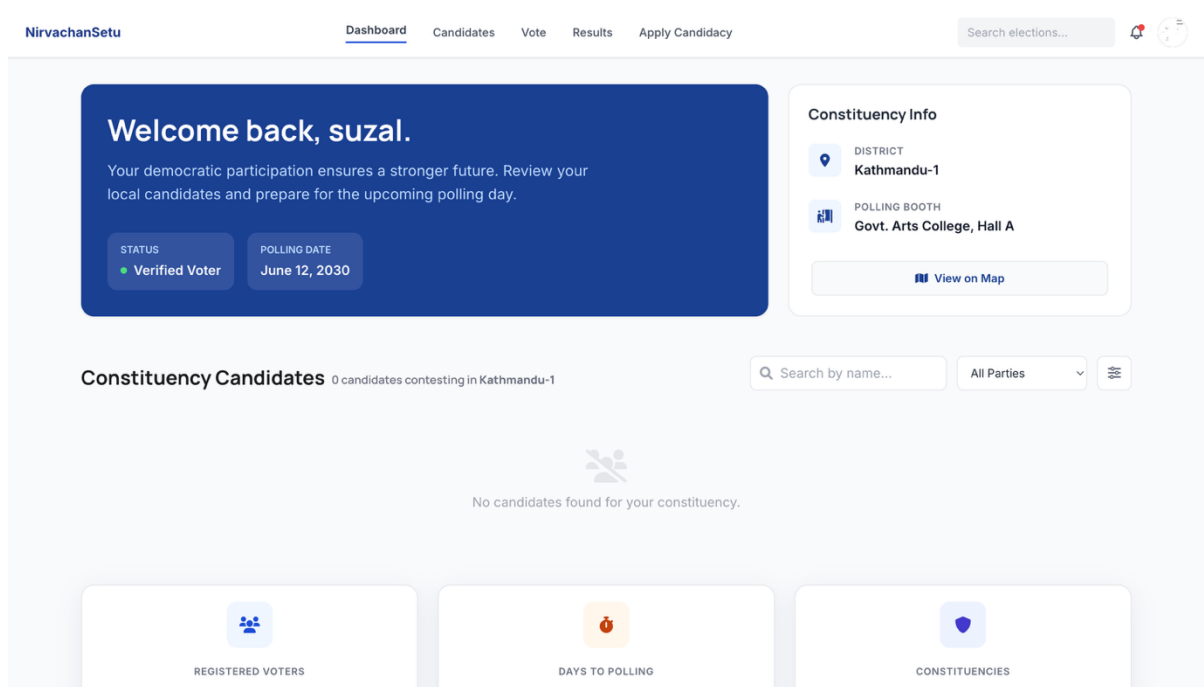


Figure 139 User dashboard after login

6.17 Candidate Nomination

Table 30 Candidate Nomination

Test ID	Description	Step	Expected- result	Actual- Result	status
Candidate Nomination	Candidate can get nominated for election.	The forms of the candidate nomination are filled up with candidate's proof of id and declaration.	After filling all details and providing the necessary documents confirmation test appears or error may appear saying application already submitted in case the candidate already submitted the application before.	error appear saying application already submitted.	pass

Candidate Portal
GENERAL ELECTION 2024

Dashboard

Application

My Profile

Competitors

Vote Now

Submit Declaration

NirvachanSetu

Candidate DashboardVote

Search elections...

Candidate Nomination

Provide accurate information for your official candidacy. Your nomination will be reviewed by the Election Commission.

Required Files

Government issued ID (PDF)

Signed nomination paper

Party evidence

NOMINATION FORM

Cancel

Identity Details

FULL NAME

Rajesh Sharma

PARTY AFFILIATION

Independent

CONSTITUENCY

Kathmandu-1

ELECTION CYCLE

House of Representatives Election 2024 (May 31, 2026 to Jun 3, 2026)

PARTY TYPE

National

SYMBOL

SUN

CAMPAIGN SUMMARY

test1

Verification Documents

ID Proof

Selected: 1 (5).pdf

Upload

Declaration

Selected: images (1).jpg

Upload

Party Evidence

Affiliation Dec

Upload

☒ I declare that the information provided is true and I am eligible for nomination.

Submit Nomination

Figure 140 Candidate nomination fill up

Candidate Portal
GENERAL ELECTION 2024

Dashboard

Application

My Profile

Competitors

Vote Now

Submit Declaration

NirvachanSetu

Candidate DashboardVote

Search elections...

Error submitting application: You have already submitted an application for this election.

APPLICATION STATUS

APPROVED

Your nomination for the Kathmandu-1 Constituency.

Review Application

Support Helpdesk

SUBMISSION DATE
May 1, 2026

ELECTION COUNTDOWN

11
DAYS LEFT

30 May 2026

TIST

Total Registered Voters

1,248,302

Current counted ballots

Predicted Reach Index

50%

Projected influence score

Active Polling Stations

1,402

In your constituency

Fund Utilization

₹4.2M

People First Party

Election Milestones

View Timeline

Document Upload Completed

Identity proof, educational certificates, and residential proof have been successfully verified by the portal's OCR system.

Oct 12, 11:40 AM

Physical Verification Scheduled

Oct 20, 09:00 AM

Constituency Hotspots

Figure 141 candidate nomination error since application was already submitted prior

154

6.18 Edit candidate profile

Table 31 Edit Candidate profile

Test ID	Description	Step	Expected- result	Actual- Result	status
Edit candidate profile and manifesto	Candidates can edit their profile as per their need.	Click on edit profile on my profile page.	The changes appear on the profile.	Accutal changes appears in profile	pass

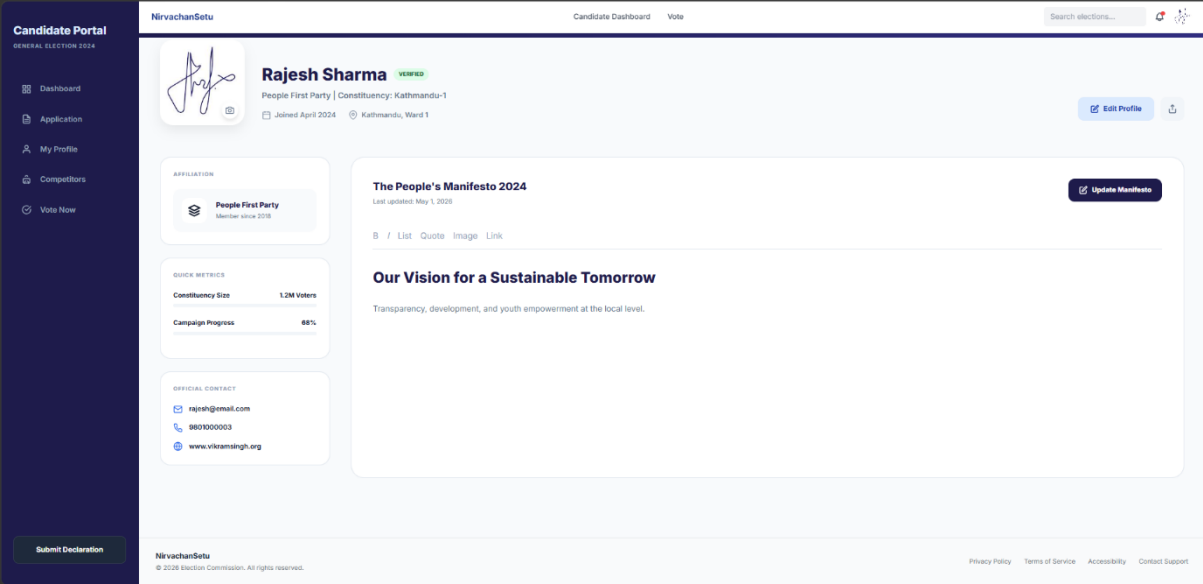


Figure 142 Before editing profile

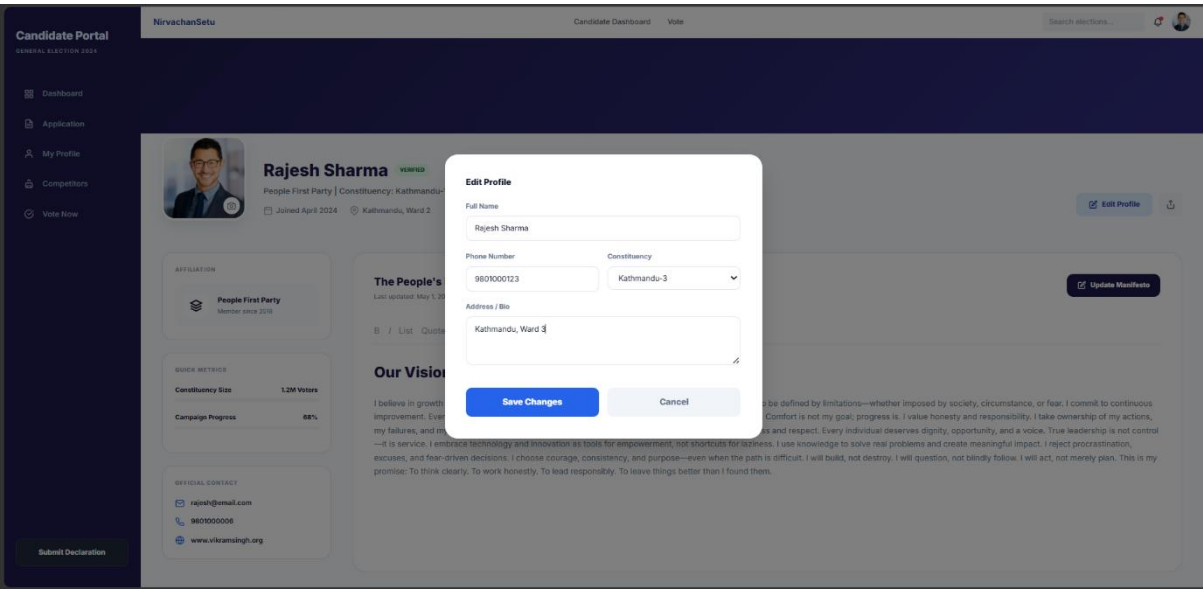


Figure 143 Changes made in profile

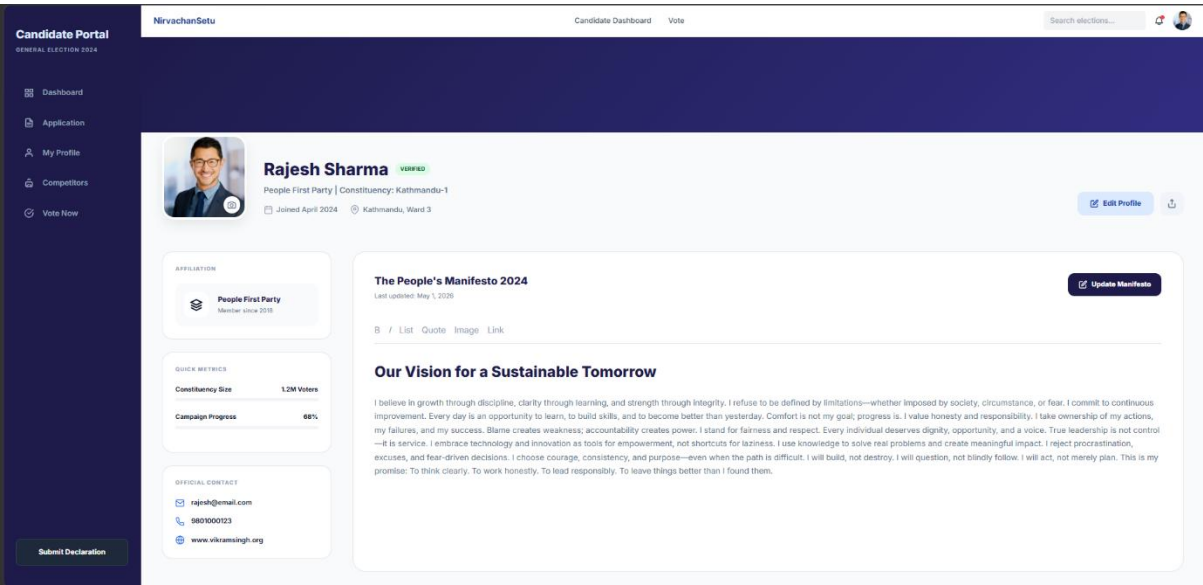


Figure 144 result after changes are made

6.19 Edit manifesto

Table 32 Edit manifesto

Test ID	Description	Step	Expected- result	Actual- Result	status
Edit candidate manifesto	Candidates can edit their manifesto as per their need.	Click on update manifesto on my profile page and write as per needed.	The changes appear on the profile.	Accutal changes appears in profile	pass

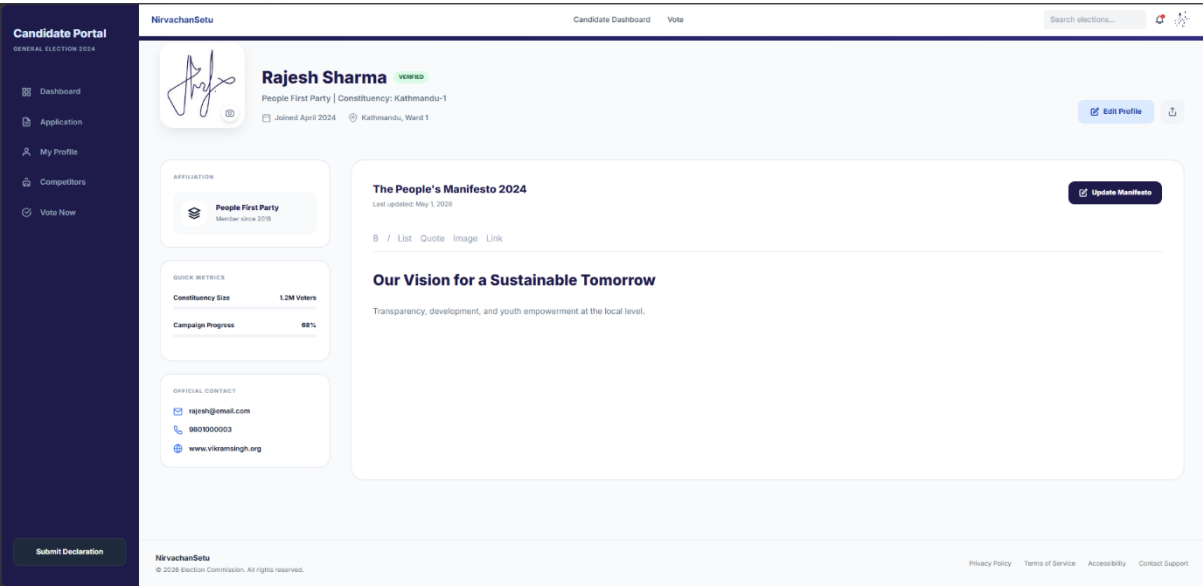


Figure 145 before editing manifesto

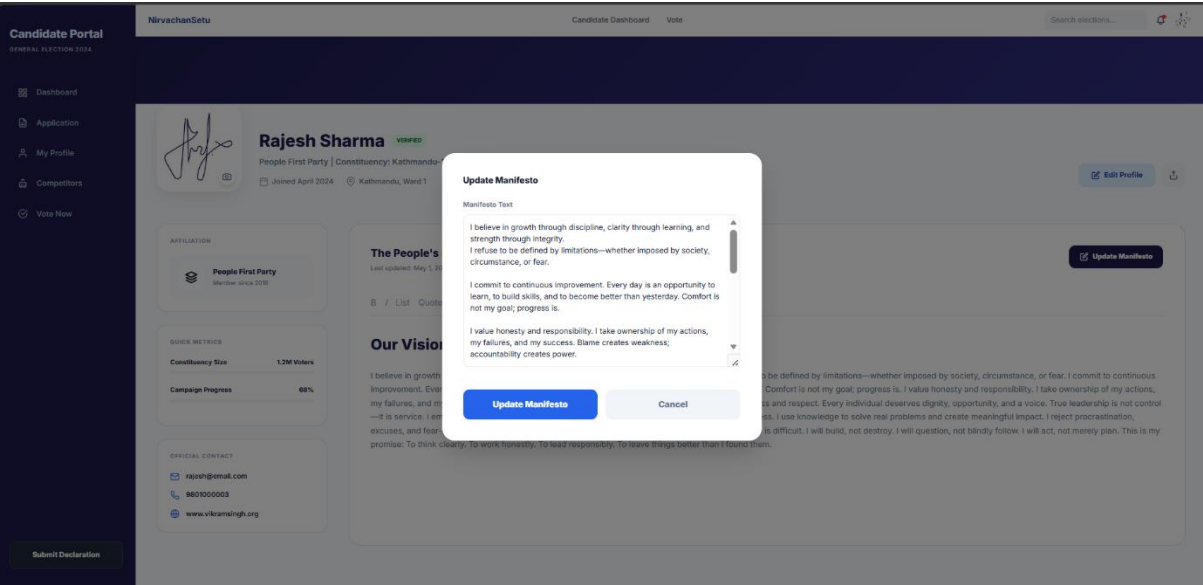


Figure 146 changes made in manifesto

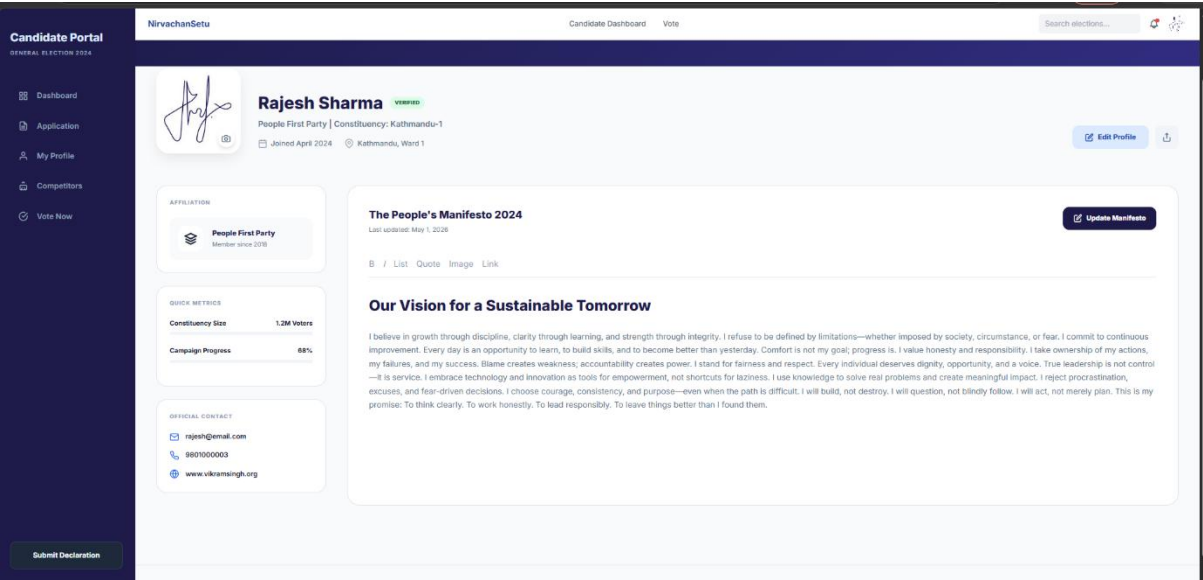


Figure 147 result after changes are made

6.20 Update profile picture

Table 33 update profile picture

Test ID	Description	Step	Expected- result	Actual- Result	status
Update profile picture	Candidates can update their profile picture on NirvachanSetu.	Click on camera logo on the profile picture in my profile and then select the picture you want to set as a profile picture.	The profile picture changes.	The profile picture actually changed.	pass

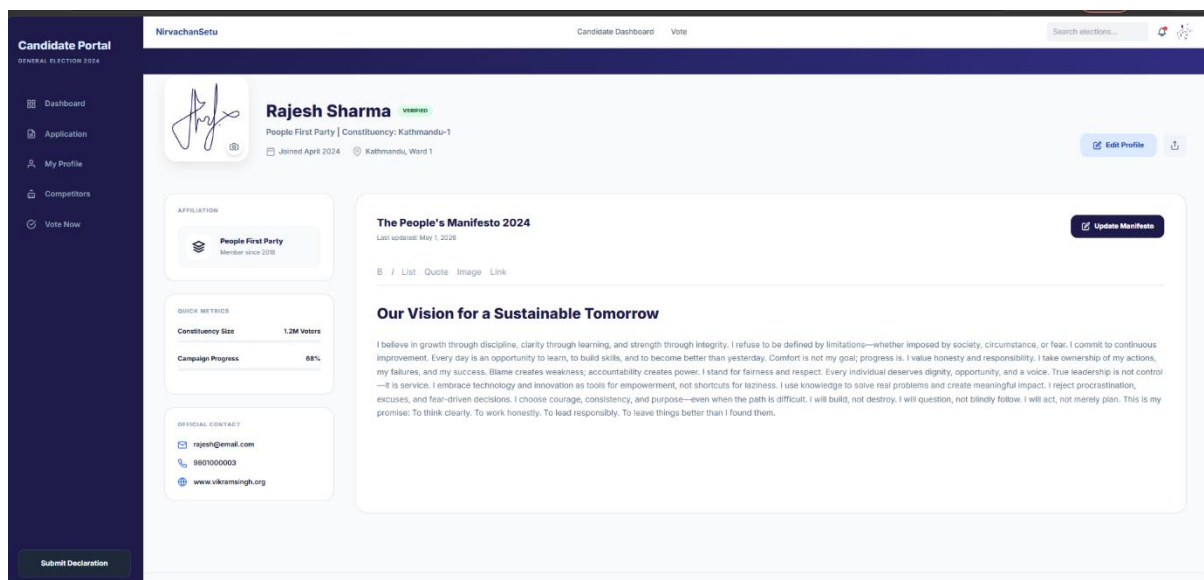


Figure 148 before changing profile picture

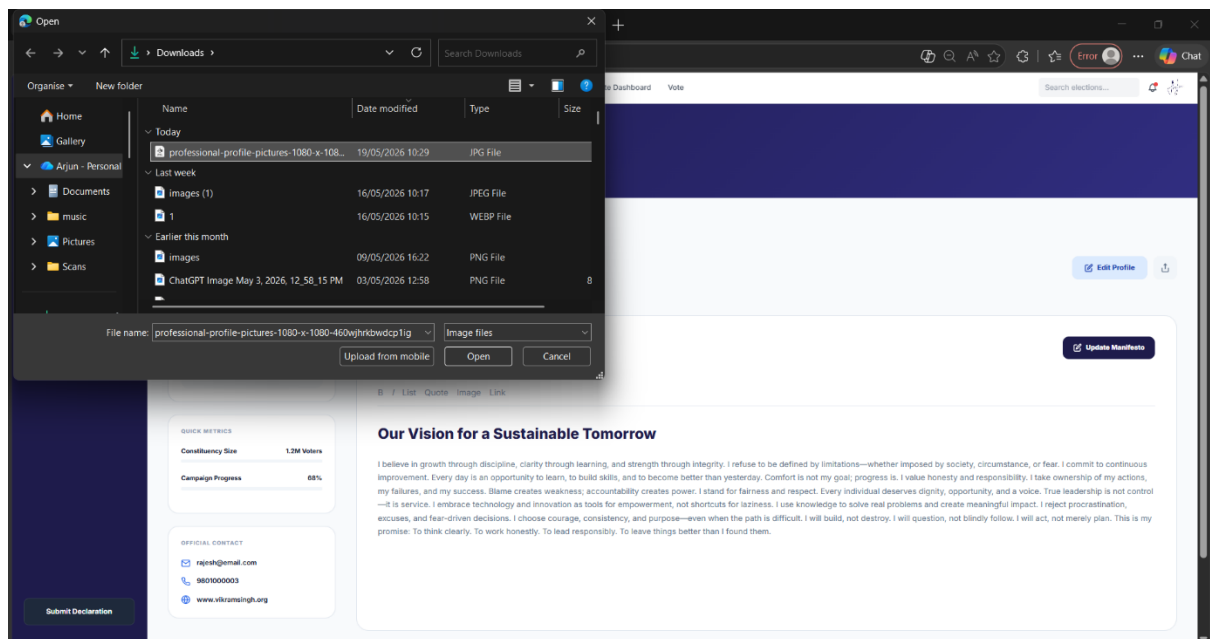


Figure 149 changing profile picture

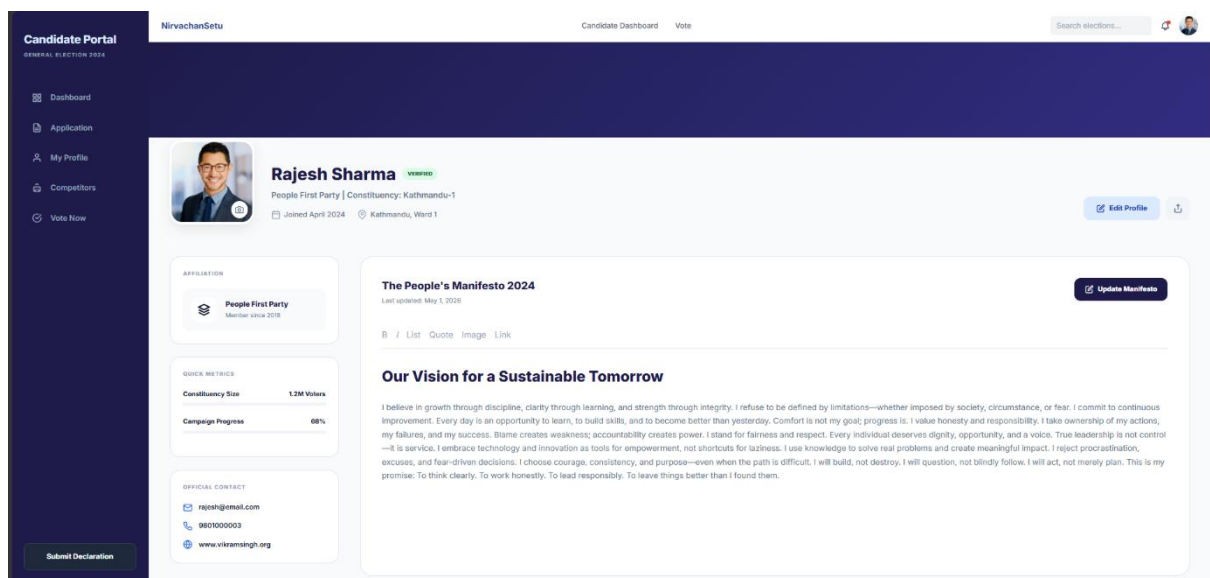


Figure 150 result after profile picture is changed

6.21 View Competitors.

Table 34 View Compitators

Test ID	Description	Step	Expected- result	Actual- Result	status
View competitors	Candidates can view the statistics of the competitors in NivachanSetu	On a side bar click on the competitors.	Competitors appears in the page or no competitors can be seen if there is no compitators in the area.	No competitors appears since there is no competitors in the area.	pass

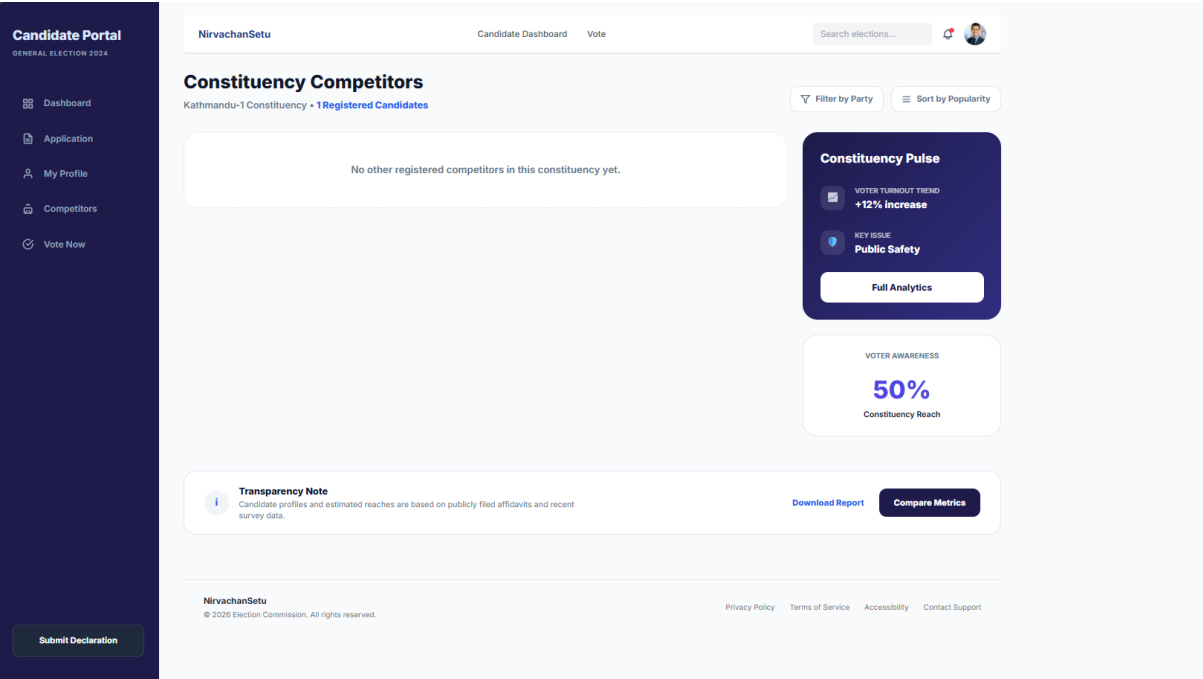


Figure 151 view competitors

7 Coursework Development with Analysis

This section documents the development contributions of each team member in the NirvachanSetu Digital Voting System. Each sub-section covers the features developed, the tools utilized, the implementation process, and the challenges encountered along with how they were resolved.

7.1 Team Contribution

The table below summaries responsibilities assigned to each team member:

Table 35 Contribution Tabless

Team Member	Task/Responsibilities
Sujal Kumar Majhi	Voter Portal – VoterDashboardServlet, VoterProfileServlet, VoterCandidateServlet, VoterVoteServlet, VoterResultServlet, VoterApplicationServlet and all voter JSP pages
Nitika Mandal	Admin Portal – AdminDashboardServlet, AdminUserServlet, AdminCandidateServlet, AdminElectionServlet, AdminConstituencyServlet, AdminResultServlet and all admin JSP pages
Arjun Rai	Candidate Portal – CandidateDashboardServlet, CandidateProfileServlet, CandidateManifestoServlet, CandidateApplicationServlet and all candidate JSP pages
Bijay Khawas	Authentication & Layout – AuthServlet, AuthFilter, layout templates (header, footer, sidebars), AppConstants utility, PasswordUtil utility
Sugam Khakiwada	Full Backend – all Model entities (User, Candidate, CandidateApplication, Election, Constituency, Vote), all DAO classes, all Service classes (UserService, CandidateService, ElectionService, VoteService), EMFProvider, persistence.xml, Database.sql schema

7.1.1 Sujal Kumar Majhi – Voter Portal

Sujal Kumar Majhi completed the Voter Portal - the main user interface for the registered and approved voters to access the NirvachanSetu system. This involved the creation of six front-end (JSP view) pages and their back-end controllers (Java Servlets).

a. Voter Dashboard (VoterDashboardServlet)

The VoterDashboardServlet (mapped to /voter/dashboard) responds to GET requests for the voter's dashboard. It loads the logged-in user from the HTTP session, and then loads: elections currently underway in the voter's constituency, list of candidates standing for election in the voter's constituency, whether the voter has already voted in the current election, and elections open for candidate nomination. Attributes are passed to voter/dashboard.jsp for display. We initially injected mock data into the servlet (pseudo elections, candidates and constituency info) to allow the JSP front-end to be developed and tested without requiring a live database connection. Later, when the services were available, the service calls (ElectionService.getActiveElections(), CandidateService.getCompetingCandidates(), VoteService.hasVoted()) were implemented and the mock block was left commented-out for future referencing in case of any issues.

Tool Used: The IntelliJ IDEA IDE was used for servlet development and debugging. Apache Tomcat 10 server was used locally. XAMPP was used for the MySQL database server.

b. Voter Profile Management (VoterProfileServlet)

The VoterProfileServlet (mapped to /voter/profile) processes GET and POST requests. The GET method retrieves the voter's profile from the session and all constituencies from ConstituencyDAO to allow the voter to change their constituency. The POST method receives the form data (full name, phone number, address, constituency), updates the User object, invokes UserService.updateProfile() to save changes to the database, and updates the session user. The success/error message is held in the session scope and displayed following a redirect.

c. Candidates View (VoterCandidateServlet)

The VoterCandidateServlet (mapped to /voter/candidates) displays all of the candidates contesting in a voter's constituency in read-only mode. It first checks that the voter is associated with a constituency. It then gets the election to view from query parameter (electionId) or the first active election from ElectionService.getActiveElections().

CandidateService.getCompetingCandidates() is invoked with the resolved election ID and constituency ID to return the list of candidates. The JSP is also provided with all active elections to handle elections with multiple elections.

d. Cast Vote Feature (VoterVoteServlet)

The VoterVoteServlet (mapped to /voter/cast-vote) is responsible for voting. The GET handler verifies the voter has a constituency, there is an active election, and by calling VoteService.hasVoted() to ensure the voter has not voted, then loads the candidates. The POST handler checks the candidateId and electionId, does another duplicate vote check for security, then invokes VoteService.castVote() with the election ID, candidate ID, constituency ID, and voter user ID. The voter is redirected to the dashboard on success, with a success message. All errors such as invalid IDs and double-voting are indicated by session-scoped error messages.

e. Election Results View (VoterResultServlet)

The VoterResultServlet (mapped to /voter/results) displays the results of completed elections. It calls ElectionService.findByStatus("COMPLETED") to get completed elections and for each election, VoteService.getElectionResults() to get the ranked candidates and VoteService.getVoteCount() for the vote count. The results are stored in two LinkedHashMaps (election to list of candidates and election ID to vote count) and passed to voter/results.jsp.

f. Apply for Candidacy (VoterApplicationServlet)

The VoterApplicationServlet (mapped to /voter/apply-candidacy) allows voters to apply to contest in an election. The GET handler reloads the user from the database (to avoid JPA detached entity issues), loads elections in the REGISTRATION phase, loads all constituencies and checks for existing applications. It excludes applications from past elections, enabling voters who have run in previous elections to apply again. The POST handler creates a new CandidateApplication object with data from the request, and invokes CandidateService.applyForElection() to save the application as PENDING for review.

g. Critical Analysis

Problem 1 - JPA Detached Entity Exception: When we initially set up the voter profile update, we received a JPA DetachedObjectException because the User object stored in the HTTP session was no longer attached to the EntityManager (the EntityManager is new for each request, making entities from the HTTP session detached). We rectified this issue by reloading

the user from the database at the beginning of each POST request using the `UserService.findById(sessionUser.getId())`, so that it can be updated in a managed state. This approach was then followed in `VoterApplicationServlet`.

Challenge 2 - Dashboard Development Without a Database: In the initial stages, the voter dashboard could not be tested as the database and services were being developed by other team members. To overcome the dependency on the database and services, we created mock data objects within the servlet and added them as request attributes. The JSP page could then be developed with "real" data. When the back-end was available, service calls were added and the mock block was commented out, showcasing a good team development strategy.

Problem 1: Preventing Duplicate Votes: This test found that if an individual voter submitted the vote form twice in quick succession (e.g., double-clicked), then the POST request might be processed twice before the first vote was recorded in the database. The original solution only prevented duplicate votes on the GET request. The solution was to have a second `VoteService.hasVoted()` check prior to `castVote()` in the POST handler, thus achieving server-side duplicate prevention of any client-side actions.

7.1.2 Nitika Mandal – Admin Portal

Nitika Mandal implemented the whole Admin Portal, allowing system administrators to manage users, candidate registrations, elections, constituencies, and election results. Six servlet controllers and JSP views were developed.

a. Admin Dashboard (AdminDashboardServlet)

The AdminDashboardServlet (mapped to /admin/dashboard) offers an admin view of the system. Every time the page is requested, it updates the session user from the database to get current data, then loads: registered users (UserService.getActiveElections()), all elections for the dashboard table (ElectionService.getActiveElections()), and all elections for the dashboard table (ElectionService.countAllCandidates()), total election count (ElectionService.countAllElections()), and total number of users (UserService.countAll()), and total pending users (UserService.countByStatus("PENDING")). These statistics are sent as request attributes to admin/dashboard.jsp.

b. User Management (AdminUserServlet)

The AdminUserServlet (mapped to /admin/users) provides administrators with the ability to manage all voter accounts. The GET handler calls UserService.findAll() and passes the list to users.jsp. The POST handler takes an action parameter (approve, reject, or suspend) and a userId, and calls UserService.updateStatus() with the action status (APPROVED, REJECTED, or SUSPENDED). The Post-Redirect-Get pattern is implemented - success or error messages are set in the session and a redirect is done to avoid a form resubmission when the page is refreshed.

c. Candidate Application Management (AdminCandidateServlet)

The AdminCandidateServlet (mapped to /admin/candidates) is responsible for the review of candidate applications. The GET handler returns a list of all applications via CandidateApplicationDAO.findAll(). The POST handler handles the approval or rejection of applications: CandidateService.approveApplication() "approves" the application (sets it to APPROVED), creates a Candidate record, and elevates the applicant's role from VOTER to CANDIDATE; CandidateService.rejectApplication() updates to REJECTED, but doesn't create a Candidate record. Both methods redirect with messages stored in the session.

d. Election Management (AdminElectionServlet)

The AdminElectionServlet (mapped to /admin/elections) handles all CRUD operations on elections. The GET handler checks if the id query parameter is set: if not, it displays all elections; if so, it loads the election for editing. The POST handler creates and updates. A private method, parseDateTime(), tries several DateTimeFormatter patterns (with and without seconds) to cope with different browser formats of the datetime-local input. A business rule is implemented: when the election state changes to ACTIVE or COMPLETED, the CandidateService.revertCandidatesToVoters() is invoked to change all candidate users to VOTER role.

e. Constituency Management (Admin Constituency Servlet)

The AdminConstituencyServlet (mapped to /admin/constituencies) is used for creating and listing constituencies. The GET handler calls ConstituencyDAO.findAll(). The POST handler creates a new Constituency from form fields (name, district, province, total voters count, description) and persists it with ConstituencyDAO.save(). Optional fields totalVoters are checked for null and empty to avoid NumberFormatException.

f. Election Results (AdminResultServlet)

The AdminResultServlet (mapped to /admin/results) shows election results to administrators. It can display the result of a particular election (electionId query parameter, used to publish results) or all elections. For each election, VoteService.getElectionResults() returns ranked candidates list and VoteService.getVoteCount() provides total votes. Two LinkedHashMaps (maintaining insert order) containing the results are returned to admin/results.jsp.

g. Critical Analysis

Challenge 1 - Election Status Business Logic: An interesting design challenge was encountered with managing candidate user roles as elections progress. Once an election is in ACTIVE status, CANDIDATE users should no longer have this role as candidate registration has ended. The approach was to invoke CandidateService.revertCandidatesToVoters() in the POST handler of the AdminElectionServlet when the status is changed to ACTIVE or COMPLETED. This leaves the Candidate record (for tracking votes) intact and reverts the User record role to VOTER for role-based security during the election process.

Problem 1 - Variable Date-Time Formats: Testing revealed that datetime-local input fields in HTML input forms can cause different date/time formats to be submitted to the server (some with seconds, some without). Using one `DateTimeFormatter` resulted in parsing errors across different browsers. The solution was to provide a private `parseDateTime()` method that tries several formatters, returning the first one that succeeds. This allowed browser-independent creation/updates of elections.

7.1.3 Arjun Rai – Candidate Portal

ArjunRai implemented the entire Candidate Portal, giving approved candidates an overview dashboard, profile and manifesto editing, and applying to the job. Four servlet controllers and JSP views were created.

a. Candidate Dashboard (CandidateDashboardServlet)

The CandidateDashboardServlet (mapped to /candidate/dashboard) is the main page for logged-in candidates. The GET handler loads the logged-in user from session, invokes CandidateService.findByUser() to retrieve the corresponding candidate record, and renders the view with: the candidate's application status, their contesting election, the constituency they are assigned to, the list of other candidates contesting in the same election/constituency, their current votes, and a vote percentage. The vote percentage is calculated by adding up the total votes of all competitors (including the current candidate) and dividing the current candidate's votes by this total. The code includes null checks in case no candidate record has been created.

b. Candidate Profile Management (CandidateProfileServlet)

The CandidateProfileServlet (mapped to /candidate/profile) handles the candidate's profile. The GET handler loads the current user from the session, and retrieves all constituencies. The POST handler updates full name, phone, address and constituency, saves changes through UserService.updateProfile(), updates the session and redirects with a message. This enables candidates to maintain their details during the election.

c. Manifesto Management (CandidateManifestoServlet)

The CandidateManifestoServlet (mapped to /candidate/manifesto) lets candidates edit their election manifesto. The GET handler retrieves the candidate record from CandidateService.findByUser() and returns the current manifesto. The POST handler checks the manifesto is not blank, gets the candidate ID and calls CandidateService.updateManifesto() to update the text. If no candidate (application not yet approved) or an empty manifesto is present, a suitable message is displayed.

d. Candidate Application (CandidateApplicationServlet)

The CandidateApplicationServlet (mapped to /candidate/application) provides a way for users to apply for candidacy. The GET handler retrieves elections with statuses REGISTRATION

and ACTIVE, all constituencies and the user's candidate application (if any). The POST handler creates a CandidateApplication entity from the form submission (election ID, constituency ID, party name, party type, manifesto, symbol), reloads the user from the database to prevent "detached entity" problems, and invokes CandidateService.applyForElection() method to process the application. If successful, the user is taken to the candidate dashboard.

e. Critical Analysis

Challenge 1 - Divide by Zero in Vote Percentage: When I first added the vote percentage to the screen, I received an ArithmeticException when there were no votes casted (divide by zero error). Further, the current candidate's votes were not being included in the sum of other candidates, resulting in an exaggerated percentage. This was resolved in two ways: by adding the current candidate's totalVotes to the sum before division, and by checking that the total is not zero before performing the division.

Problem 1 - Null Candidate for New Users: Accessing the candidate dashboard immediately after role change (before a Candidate record had been created by the backend) resulted in a NullPointerException with CandidateService.findByUser() returned null. The solution was to check for null before accessing candidate properties. If this was the case, the dashboard displays the message "Please wait for the admins to approve your account or apply for candidacy", and does not crash the page.

7.1.4 Bijay Khawas – Authentication, Filter & Layout

Bijay Khawas was involved in developing the Authentication system, AuthFilter security component, common layout templates for all portals and the AppConstants and PasswordUtil utility classes. These are the core security and look and feel components for the NirvachanSetu project.

a. Authentication (AuthServlet)

The AuthServlet (mapped to the /login, /register and /logout paths) handles all three types of authentication and is marked with @MultipartConfig to allow file uploads for registration.

The handleLogin() method extracts the email and password from the request, invokes UserService.login() (which invokes PasswordUtil.verifyPassword()) and, if successful, places the User object and role string in the HttpSession and then forwards to the role-based dashboard (ADMIN, VOTER, or CANDIDATE). If it fails, an error message is passed to login.jsp.

The handleRegister() method handles server-side validation: it ensures that all fields (full name, email, password, citizenship number) are present; checks that the password is at least six characters and matches the confirmation; and ensures there are no duplicate emails (UserService.findByEmail()) and duplicate citizenship numbers (UserService.isCitizenshipNumberExists()). The user can also upload an image (PNG/JPEG, up to 5MB), which is stored on the server under images/profile/ with a random name generated by UUID. Failure to upload an image is not critical: the user is registered and an error message logged. Users are automatically given roles VOTER and PENDING.

The logout URL destroys the current HTTP session and re-directs to the login page.

b. AuthFilter – Security and Role-Based Access Control

The AuthFilter is a filter in Jakarta Servlet that processes all HTTP requests. It is responsible for two main tasks: authentication and authorisation.

Authentication: first, the filter checks if the request is for a public path (login, register, CSS, JS, images). If not, it checks for a valid user object in the session. During the UI development phase, a documented bypass was added: if no user is present in the session, then it injects a dummy VOTER user so that the front-end developer can test portal pages without having to implement login. This bypass is obviously commented out for production.

Authorisation: the `hasRoleAccess()` helper controls path access - only the ADMIN role may access `/admin/*` paths; only the CANDIDATE role may access `/candidate/*` paths; both VOTER and CANDIDATE roles may access `/voter/*` paths (candidates can vote too). If not, the user is redirected to his or her own dashboard via `getDashboardForRole()`.

c. Layout Templates

The layout JSPs (`header.jsp`, `footer.jsp`, `admin-sidebar.jsp`, `voter-sidebar.jsp`, `candidate-sidebar.jsp`, `admin-layout.jsp` and `admin-layout-close.jsp`) form the common structure for all portal pages. Sidebars are customised for each portal with appropriate menu items. The `admin-layout.jsp` and `admin-layout-close.jsp` files include wrappers that embed content pages in the admin sidebar layout without having to repeat markup. This guarantees a unified look-and-feel and makes maintenance easy - if a sidebar navigation item is to be changed, only a single file needs to be updated.

d. AppConstants Utility

`AppConstants` is a final class with a private constructor that defines all constants used in the system: attribute names used in the session (`SESSION_USER`, `SESSION_ROLE`), role constants (`ROLE_ADMIN`, `ROLE_VOTER`, `ROLE_CANDIDATE`), status constants for users, elections, candidates and applications, election type and party type constants, default values for pagination, URL path constants, the `PUBLIC_PATHS` array used by `AuthFilter` and message constants that represent success and error messages. Having constants centralised prevents typos when comparing strings, and makes refactoring easy.

e. PasswordUtil Utility

`PasswordUtil` has static methods for password security. `hashPassword()` performs SHA-256 hashing of the plain text (appending an application-specific salt ("NirvachanSetu2024")) and returns the hash as a hexadecimal string. `verifyPassword()` hashes the plain text again with the same salt and checks if the hash equals the stored hash (case-insensitive). A private

bytesToHex() function converts the MessageDigest byte array to hex string. The class has a private constructor to prevent instantiation.

f. Critical Analysis

Challenge 1 - Preventing Duplicate Registration: The first version of the code only prevented duplicate email addresses from registering. Testing revealed a citizen could use different email addresses to register multiple voter accounts with the same citizenship number. The solution was to include a call to `UserService.isCitizenshipNumberExists()` in the registration handler to check if a citizen is registered with that citizenship number and returns an error message if it exists.

Problem 1 - Servlet Configuration: When the image upload was first added to the registration form, the servlet was unable to process the file as it threw an `IllegalStateException` because it was not configured with `@MultipartConfig`. Without this, Jakarta EE doesn't process multipart form data, so `request.getPart()` unavailable. This was fixed by adding `@MultipartConfig` with suitable size restrictions (1MB threshold, 5MB max file size, 10MB max request size). We also added a try-catch to the upload processing so that a failed upload does not hinder registration - the user will be registered without a profile image and a log entry will be created.

7.2 Sugam Khakiwada – Backend (Models, DAOs, Services & Database)

Sugam Khakiwada took care of the entire backend design of NirvachanSetu: all JPA entity model classes, all DAO classes based on the GenericDAO pattern, all service classes, JPA configuration (EMFProvider and persistence.xml), and the complete MySQL database schema (Database.sql). Backend is the data backbone of all portals and controllers.

a. Model Classes (JPA Entities)

We designed and developed six JPA entity classes, which correspond to the main domain objects of the online voting system:

- User - stores full name, email (unique), password (hashed), citizenship number (unique), phone, address, profile image file path, role (ADMIN / VOTER / CANDIDATE), status (PENDING / APPROVED / REJECTED / SUSPENDED), and a many-to-one relationship to Constituency.
- Election – stores the name, description, type (HOR / PROVINCIAL / LOCAL), start and end dates and times, status (DRAFT / REGISTRATION / ACTIVE / COMPLETED / CANCELLED) of election and the optional many-to-one relationship with Constituency.
- Constituency - name, district, province, total registered voters, and description.
- Candidate Application - stores a voter's candidate application to contest an election, linking to User, Election and Constituency, along with party name, party type (NATIONAL / REGIONAL / INDEPENDENT), election symbol, manifesto, and candidate application status (PENDING / APPROVED / REJECTED / WITHDRAWN).
- Candidate - stores the record of an approved candidate, which is created when a Candidate Application is approved, with relationships to User, Candidate Application, Election, and Constituency, along with party name, symbol, manifesto, a running total of votes received, and status (ACTIVE / DISQUALIFIED / WITHDRAWN).
- Vote - a cast vote, relating to an Election, Candidate, Constituency and Voter (User). The one-voter-one-vote rule is enforced by this table.

b. GenericDAO and DAO Classes

A base class `GenericDAO<T, ID>` has been implemented using JPA's `EntityManager` to offer a reusable type-safe, generic CRUD implementation (create, read, update, delete and count). The domain DAO classes inherit from `GenericDAO` and include domain-specific queries:

- `UserDAO` - adds `findByEmail()`, `findByRole()`, `findByCitizenshipNumber()`, `countByStatus()`, `findAllWithPagination()`, and `findActiveByRole()`.
- `ElectionDAO` - adds `findByStatus()`, `findByType()`, `findActiveElectionByConstituency()`, and `count()`.
- `ConstituencyDAO` - extends `GenericDAO` with `findAll()` to list constituencies.
- `CandidateApplicationDAO` - adds `findByUser()`, `findByElection()`, `findByUserAndElection()` (to avoid duplicate applications) and `findByStatus()`.
- `CandidateDAO` - adds `findByElection()`, `findByConstituency()`, `findByElectionAndConstituency()`, `findByUser()`, `findActiveByElection()` (to reverse the role), and `countByElection()`.
- `VoteDAO` - adds `hasVoted()` (to prevent duplicate votes), `countByElection()` and `countByCandidate()` (for vote counting).

Each DAO creates its own `EntityManager` and calls `close()` in service layer finally blocks to avoid connection leaks.

c. Service Layer

Four service classes implement business logic and interact with DAOs:

- `UserService` - manages user registration (with password hashing via `PasswordUtil.hashPassword()`), user login, user profile update, role change, status update, and duplicate email and citizenship number check.
- `CandidateService` - orchestrates the candidate lifecycle: apply (with checks that user is approved, election is in REGISTRATION state and no duplicate vote), approve (creates Candidate record and changes user role to CANDIDATE), reject, withdraw, disqualify, update manifesto, and `revertCandidatesToVoters()` to change candidate users back to VOTER when elections are active.
- `ElectionService` - manages elections create, update, change status, and get elections by status, type and constituency.
- `VoteService` - provides a transactionally safe `castVote()`: checks election activity, duplicate votes, candidate eligibility, creates Vote record, and updates candidate's totalVotes, all within a JPA transaction. Also exposes `getElectionResults()`, `getWinner()` and `getVoterTurnout()`.

d. EMFProvider and Persistence Configuration

EMFProvider is a singleton which stores a single EntityManagerFactory instance for the duration of the application (read from persistence.xml - META-INF/), which defines the MySQL JDBC driver, the database URL, JPA provider (Hibernate) and entity classes. This is a common JPA practice, as creating EntityManagerFactory per request is very expensive and will consume a lot of resources very quickly.

e. Database Schema (Database.sql)

We developed the full MySQL schema in Database.sql. It has CREATE TABLE statements for the six entity tables with suitable data types, primary key auto-increment settings, NOT NULL constraints for mandatory fields, UNIQUE constraints for email and citizenship_number in the users table, foreign key constraints between tables (e.g., votes table to elections, candidates, constituencies, and users), and ENUM column types for status and role to restrict values at the database level. This schema was verified using XAMPP/MySQL and test data was entered to check all relationships and constraints.

f. Critical Analysis

Challenge 1 - EntityManager Lifecycle: A major design issue was the lifecycle management of JPA EntityManagers in a typical web environment. EntityManagers are not automatically managed by Jakarta EE. If not managed properly, EntityManagers would stay open after service calls (exhausting the connection pool and leaking memory). The answer was to make each DAO class its own EntityManager instance (the EntityManagerFactory is created from the shared EMFProvider class) and a close() method. Every service method includes a finally block that calls close() to ensure that EntityManagers are always closed even in the case of an exception.

Challenge 2 - Atomic Cast Vote: The castVote() method needed to perform multiple database updates to maintain consistency: writing the Vote record and updating the totalVotes field of the Candidate. If the latter write failed after the former was executed, then the vote count would be inconsistent with the votes. The fix was to update the Candidate's vote count within an explicit JPA transaction on the VoteService's own EntityManager (with a rollback in the catch block in case of an exception), allowing the set of steps to be atomic.

Problem 1 - Multiple Applications: When we tested the system, we found that a voter can make multiple applications for candidacy for an election by directly accessing the apply URL. The

solution was to do this in `CandidateService.applyForElection()`: it first calls `CandidateApplicationDAO.findByUserAndElection()` with the user and election IDs. If such an application is found, a `RuntimeException` is thrown with a suitable message, before a new application is attempted to be saved. The servlet catches this exception and passes the message to the user by setting the error attribute in the session.

8 Conclusion

NirvachanSetu is a significant move towards democratic process being brought up to date through technology. The implementation of this Java-based web application has shown how more complex programming concepts, such as MVC architecture, role-based access control, secure session management, and normalized database design can be used together to address a real life problem of national interest.

During the development process the team was able to design and implement a multi-role election management platform that caters to the needs of the administrators, voters, and the candidates through defined workflows. The admin module gives total control over the election life cycle, including user verification up to the publication of the results. The voter module provides a secure and anonymous voting experience, and the one-person-one-vote principle is faithfully adhered to. The candidate module enables candidates to control their profiles and match the elections. These modules, together constitute a well-integrated effective e-government system.

The project further enhanced the practical knowledge of the team in the world of Java EE technologies, such as, Java Servlets, JSP, JDBC, and MySQL. The features to be implemented like filter-based redirect-handling, session-controlled accessibility, password encryption, and responsive CSS design without use of external frameworks challenged the group to be innovative and effective in implementing the foundational knowledge of programming.

During development a number of challenges were experienced. Because it was necessary to design a schema representing complex relationships in the real world, such as constitutions and eligible candidate status, and the storage of anonymous votes, careful planning and iteration were needed. It also required extensive testing and debugging to ensure that the application worked correctly in various environments and to guarantee the security of sessions managed by the application. The team was able to overcome these challenges by involving itself in problem-solving together, allocating tasks to be completed in an organized manner, and engaging in a progressive development process.

Regarding the learning aspect, this project reasserted on the greater significance of planning prior to coding. The pre-writing of lines of application code in wireframes, ER diagrams, and class diagrams were invaluable in ensuring the project maintained coherence and direction throughout writing the first line of application code. The MVC architecture, specifically, led to

better organization and debuggability of the codebase and, in the long term, demonstrated the value of clean and modular design.

In the future, NirvachanSetu has many opportunities to grow and develop. This system could be scaled up to include local and provincial elections with the current HoR elections since the scalable architecture already supports this. It could later be enhanced with SMS or email notification support, real time vote tallying dashboards, multi-language support to enable accessibility and integration into official national identity databases to facilitate automated voter verification. Also, due to the freedom of the RESTful API design, it will be possible to have the platform support mobile application clients in future.

To sum up, NirvachanSetu was able to fulfill its core goals of building a secure, transparent and scalable digital election platform. The project is not only able to implement the technical requirement of the CS5054NT module, but also shows how the concepts of software engineering can be practically used to respond to significant societal concerns. The team also prides itself on the work done and is certain that the system will build a solid base of more robust and feature-rich e-governance solution in the future.

9 References

Ant Design Team. (2024) *Analytics Page with Area and Bar Charts* [Online]. Available from: <https://preview.pro.ant.design/dashboard/analysis> [Accessed 19 May 2026].

Ant Design Team. (2024) *Ant Design Pro — Enterprise UI Components Library* [Online]. Available from: <https://pro.ant.design> [Accessed 19 May 2026].

Ant Design Team. (2024) *Comparison Table* [Online]. Available from: <https://pro.ant.design/components/table/> [Accessed 19 May 2026].

Ant Design Team. (2024) *Enterprise MIS Dashboard with KPI Cards* [Online]. Available from: <https://preview.pro.ant.design/dashboard/analysis> [Accessed 19 May 2026].

Ant Design Team. (2024) *List and Card Page* [Online]. Available from: <https://pro.ant.design/components/list/> [Accessed 19 May 2026].

Ant Design Team. (2024) *List Table with Paginated Data* [Online]. Available from: <https://pro.ant.design/components/list/> [Accessed 19 May 2026].

British Broadcasting Corporation. (2024) *BBC UK General Election Results Page* [Online]. Available from: <https://www.bbc.co.uk/news/election/2024/uk/results> [Accessed 19 May 2026].

British Broadcasting Corporation. (2024) *Constituency Breakdown Page* [Online]. Available from: <https://www.bbc.co.uk/news/election/2024/uk/constituencies> [Accessed 19 May 2026].

Dribbble Holdings Ltd. (2024) *Government Digital Service Style* [Online]. Available from: <https://dribbble.com/search/user-profile-ui> [Accessed 19 May 2026].

Dribbble Holdings Ltd. (2024) *Voter Portal and Election Dashboard UI Concepts* [Online]. Available from: <https://dribbble.com/search/voter-portal-ui> [Accessed 19 May 2026].

Election Commission of India. (2024) *Candidate Affidavit Portal* [Online]. Available from: <https://affidavit.eci.gov.in> [Accessed 19 May 2026].

Election Commission of India. (2024) *Election Commission of India* [Online]. Available from: <https://www.eci.gov.in/candidate-corner> [Accessed 19 May 2026].

Election Commission of India. (2024) *Official Candidate Profile Pages* [Online]. Available from: <https://www.eci.gov.in/candidate-corner> [Accessed 19 May 2026].

Election Commission of Nepal. (2024) *Official Login* [Online]. Available from: <https://election.gov.np> [Accessed 19 May 2026].

Election Commission of Nepal. (2024) *Official Election Results Portal — 2012 Election Results* [Online]. Available from: <https://election.gov.np/en/page/election-result> [Accessed 19 May 2026].

Envato. (2024) *Aura Admin Dashboard Template* [Online]. Available from: <https://themeforest.net/search/aura+admin+dashboard> [Accessed 19 May 2026].

GitHub Inc. (2024) *User Settings and Profile Page* [Online]. Available from: <https://github.com/settings/profile> [Accessed 19 May 2026].

Google LLC. (2024) *Kaggle Competition Leaderboard* [Online]. Available from: <https://www.kaggle.com/competitions> [Accessed 19 May 2026].

Google LLC. (2024) *Material Design Guidelines — Cards and Status Indicators* [Online]. Available from: <https://m3.material.io/components/cards/overview> [Accessed 19 May 2024].

Government Digital Service. (2024) *Apply for a Licence: Multi-Step Form Pattern* [Online]. Available from: <https://design-system.service.gov.uk/patterns/complete-multiple-tasks> [Accessed 19 May 2026].

Government Digital Service. (2024) *GOV.UK Design System — Form Components and Step-by-Step Journeys* [Online]. Available from: <https://design-system.service.gov.uk/patterns/step-by-step-navigation> [Accessed 19 May 2026].

Government Digital Service. (2024) *GOV.UK Design System — User Account Pages* [Online]. Available from: <https://design-system.service.gov.uk> [Accessed 19 May 2026].

Guardian News and Media Ltd. (2024) *Election Results Visualisation 2024* [Online]. Available from: <https://www.theguardian.com/politics/ng-interactive/2024/jul/04/uk-general-election-results-2024> [Accessed 19 May 2026].

HubSpot Inc. (2024) *Sales and Campaign Analytics* [Online]. Available from: <https://app.hubspot.com/reports-dashboard> [Accessed 19 May 2026].

LinkedIn Corporation. (2024) *LinkedIn People Search Results Page* [Online]. Available from: <https://www.linkedin.com/search/results/people> [Accessed 19 May 2026].

LinkedIn Corporation. (2024) *LinkedIn Public Profile Page* [Online]. Available from: <https://www.linkedin.com> [Accessed 19 May 2026].

Massachusetts Institute of Technology. (2024) *TrueVote Research Project — Electronic Ballot Usability* [Online]. Available from: <https://vote.mit.edu> [Accessed 19 May 2026].

Medium Corp. (2024) *Article Page Typography and Reading Layout* [Online]. Available from: <https://medium.com> [Accessed 19 May 2026].

Meta Platforms Inc. (2024) *Instagram Registration* [Online]. Available from: <https://www.instagram.com/?hl=en> [Accessed 19 May 2026].

Meta Platforms Inc. (2024) *Facebook Login Interface* [Online]. Available from: <https://www.facebook.com/login> [Accessed 19 May 2026].

Nielsen Norman Group. (2024) *10 Usability Heuristics for User Interface Design* [Online]. Available from: <https://www.nngroup.com/articles/ten-usability-heuristics> [Accessed 19 May 2026].

Overseas Vote Foundation. (2024) *Digital Ballot Interface — Civic Technology Reference* [Online]. Available from: <https://www.overseasvotefoundation.org> [Accessed 19 May 2026].

Recharts Group. (2024) *Redefined Chart Library Built with React and D3* [Online]. Available from: <https://recharts.org/en-US> [Accessed 19 May 2026].

shadcn. (2024) *Bar Charts and Data Tables* [Online]. Available from: <https://ui.shadcn.com/charts> [Accessed 19 May 2026].

shadcn. (2024) *Charts Collection (Area and Bar Charts)* [Online]. Available from: <https://ui.shadcn.com/charts> [Accessed 19 May 2026].

shadcn. (2024) *Dashboard Example* [Online]. Available from: <https://ui.shadcn.com/examples/dashboard> [Accessed 19 May 2026].

shadcn. (2024) *Dashboard Example* [Online]. Available from: <https://ui.shadcn.com/examples/dashboard> [Accessed 19 May 2026].

shadcn. (2024) *Data Table Component* [Online]. Available from: <https://ui.shadcn.com/docs/components/data-table> [Accessed 19 May 2026].

shadcn. (2024) *Form Components* [Online]. Available from: <https://ui.shadcn.com/docs/components/form> [Accessed 19 May 2026].

shadcn. (2024) *Leaderboard and Rankings Component* [Online]. Available from: <https://ui.shadcn.com/blocks> [Accessed 19 May 2026].

shadcn. (2024) *Settings Page Example* [Online]. Available from: <https://ui.shadcn.com/examples/authentication> [Accessed 19 May 2026].

shadcn. (2024) *Shadcn Admin — React and Tailwind Admin Dashboard* [Online]. Available from: <https://shadcn-admin.netlify.app> [Accessed 19 May 2026].

shadcn. (2024) *Tasks Example* [Online]. Available from: <https://ui.shadcn.com/examples/tasks> [Accessed 19 May 2026].

Typeform SL. (2024) *Multi-Step Application Interface* [Online]. Available from: <https://www.typeform.com> [Accessed May 2026 2024].